THOMAS MORE

PROGRAMMING ADVANCED

Applied Data Intelligence

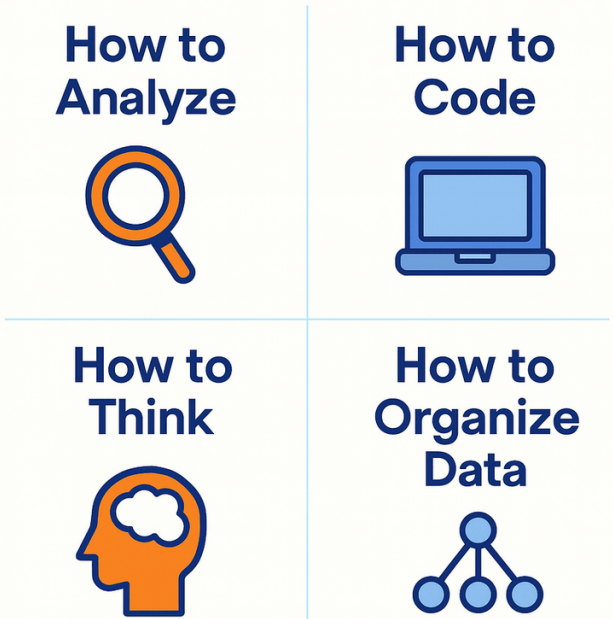
YP0822 - YP0847

This work is licensed under CC BY-NC-ND 4.0 by Elke Boonen (2025)

Learning Goals

- **Analyze** assignments and create algorithms
- **Apply** OO principles and recursion
- **Understand** performance using Big O Notation
- **Compare** and apply data structures
- **Experiment** with search and sorting algorithms
- **Apply** problem-solving strategies
- **Verify** functionality against requirements

Course overview

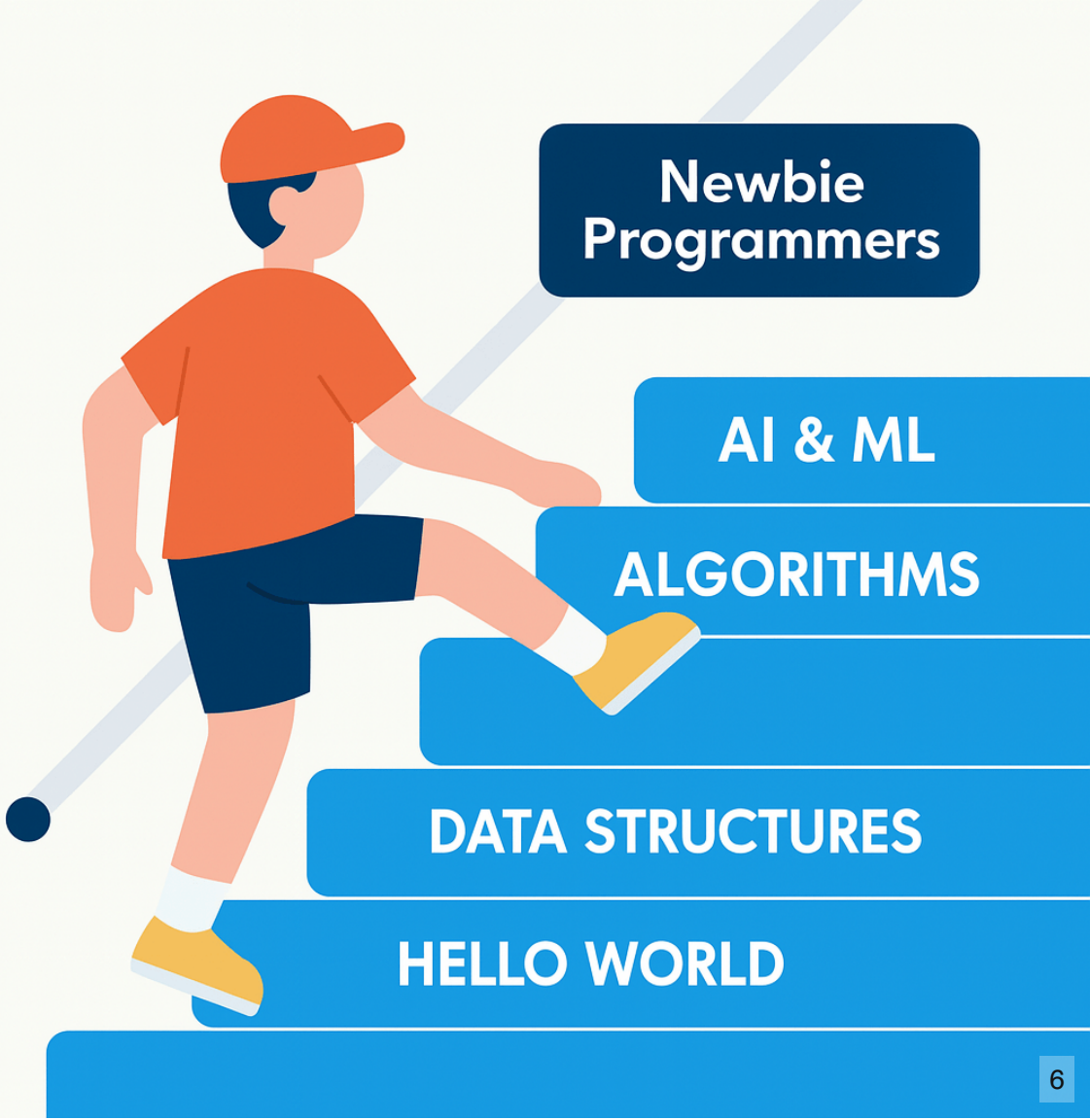


4 Pillars of Computational Problem-Solving

- **How to Analyze**: Searching, Big O Notation
- **How to Organize Data**: Stacks, Queues, Hashmaps, Trees, Graphs
- **How to Code**: Sorting, Recursion
- **How to Think**: Backtracking, Divide & Conquer, Dynamic Programming, Greedy Algorithms

Build your skills!

- Many new programmers jump straight into AI & ML... but skip essential fundamentals:
  - Object-Oriented Programming
  - Data Structures
  - Algorithms
- **Every step matters!**



Searching & intro to Big O

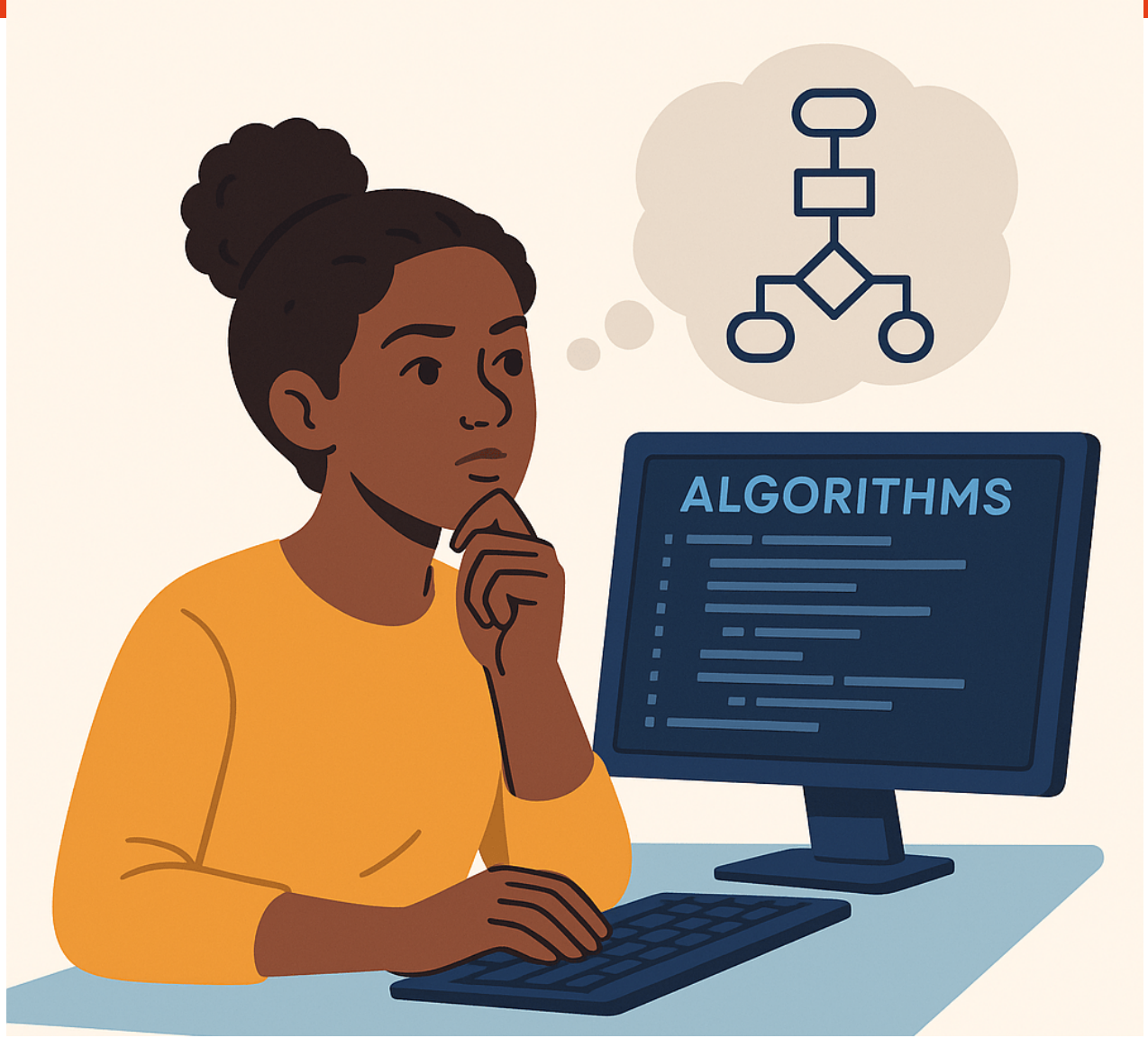
Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Searching & intro to Big O

- Basics of Algorithms
- Searching Methods
  - Coding
- Well hello Big O
- How to calculate Big O?


ALGORITHMS

What are Algorithms?

- An algorithm is a **set of step-by-step instructions** to **solve a specific problem**
  - Finding the Largest Number in a List
  - Sorting a Group of Names
  - Calculating the Shortest Route Between Two Cities
  - Searching for a Word in a Document
  - Creating a Playlist Based on Mood
  - ..
- The same **problem** can often be solved **in multiple ways using different algorithmic approaches.**

Why is Algorithm Choice Important?

- **Efficiency**: Some algorithms can solve a problem **faster** than others
  - Especially when **dealing** with vast **amounts** of **data**.
- **Space**: The amount of **memory** an algorithm uses can vary.
  - Especially in systems with **limited resources**.
- **Robustness**: Some algorithms can better **handle errors** or **unexpected inputs** than others.

The Impact of Algorithm Choices

- **Cost: Inefficient** algorithms can lead to **higher** operational **costs**
- **User Experience:** If an app or website loads slowly due to an inefficient algorithm, it can result in a **negative** user experience.
- **Future Scalability:** Choosing the right algorithm can mean the difference between a system that **scales** over **time** and one that **stagnates**.

Boost algorithm performance

- **What you already know: Brute force methods!**
  - The naive method
  - Try every possibility until you find the answer
  - Like trying every key to open a door
  - Simple but often slow!
- **What you'll learn:**
  - Why naive isn't always the best.
  - Ways to make our code run faster and smarter.
  - How to check if our code is fast or slow.
  - Tips for writing better code.

Searching & intro to Big O

- ~~Basics of Algorithms~~
- Searching Methods
  - Coding
- Well hello Big O
- How to calculate Big O?

What is Searching?

- **Searching means finding specific information or items** within a collection of data
  - Looking for a particular value, record, or element
  - Goal: locate the target as quickly as possible
- Searching is **locating specific information** within a larger set of data.
- Access the **information** we need, **when** we need it in the **most efficient** way

Why do we need Searching?

- A **cornerstone** of Computer Science!
- Why searching is **fundamental?**
  - **Information retrieval** - find what you're looking for in large datasets
  - **Decision making** - locate relevant data to make informed choicesμ
  - **Efficiency** - avoid manually checking every single item
  - **Automation** - computers can search millions of records instantly

- *The art of searching isn't just about finding what we're looking for, but doing so in the most efficient way possible, saving time and resources.*

Searching in real life

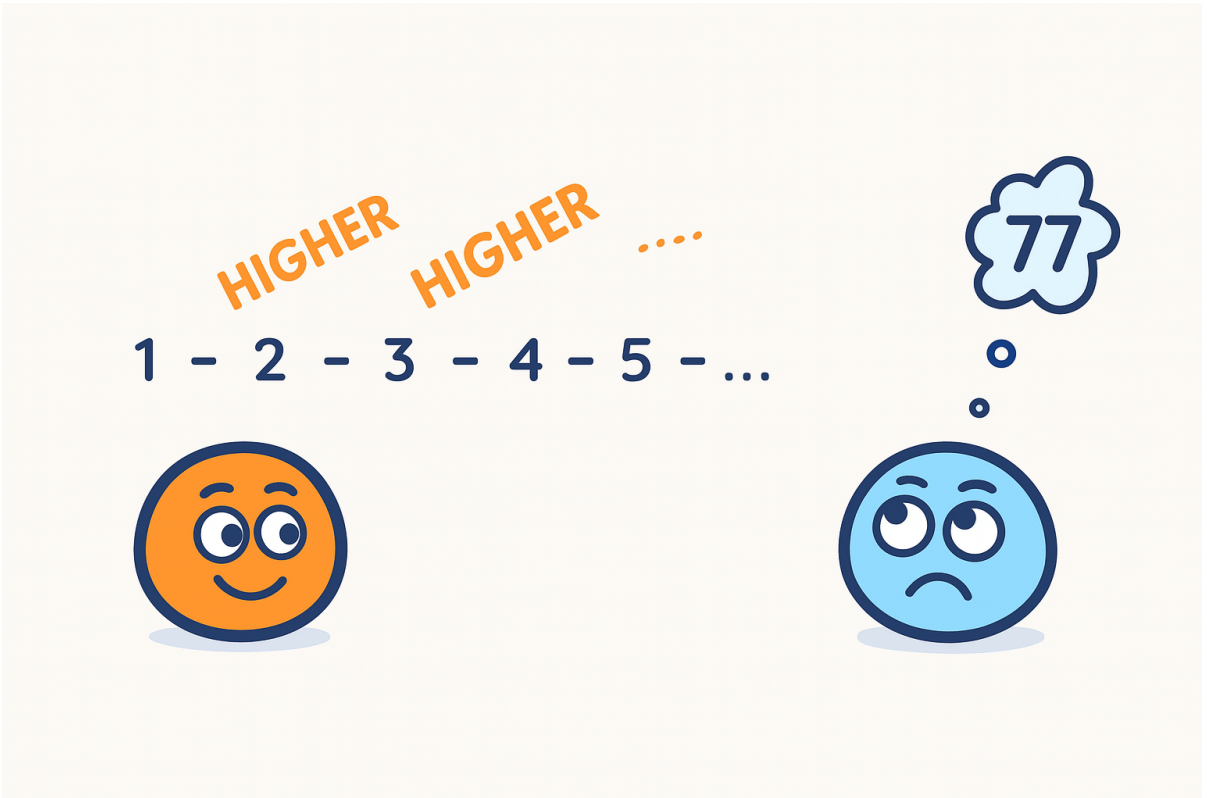
- **Phone contacts:** Find "Mom" in your contact list
- **Google search:** Type keywords to find relevant websites
- **Library catalog:** Find books by title, author, or subject
- **Netflix:** Search for movies by genre, actor, or title
- **Online shopping:** Find products by name, price range, or category
- **GPS navigation:** Search for restaurants, gas stations, or addresses
- **Email:** Find specific messages by sender, subject, or keywords
- **Social media:** Search for friends, hashtags, or posts
- **Medical records:** Doctors search patient history for symptoms or treatments
- …

Now, let's start searching

“ Suppose we play a guessing game; Someone has a number between 1 and 100 that we have to guess. (S)he only responds with 'higher' or 'lower' to our guesses.

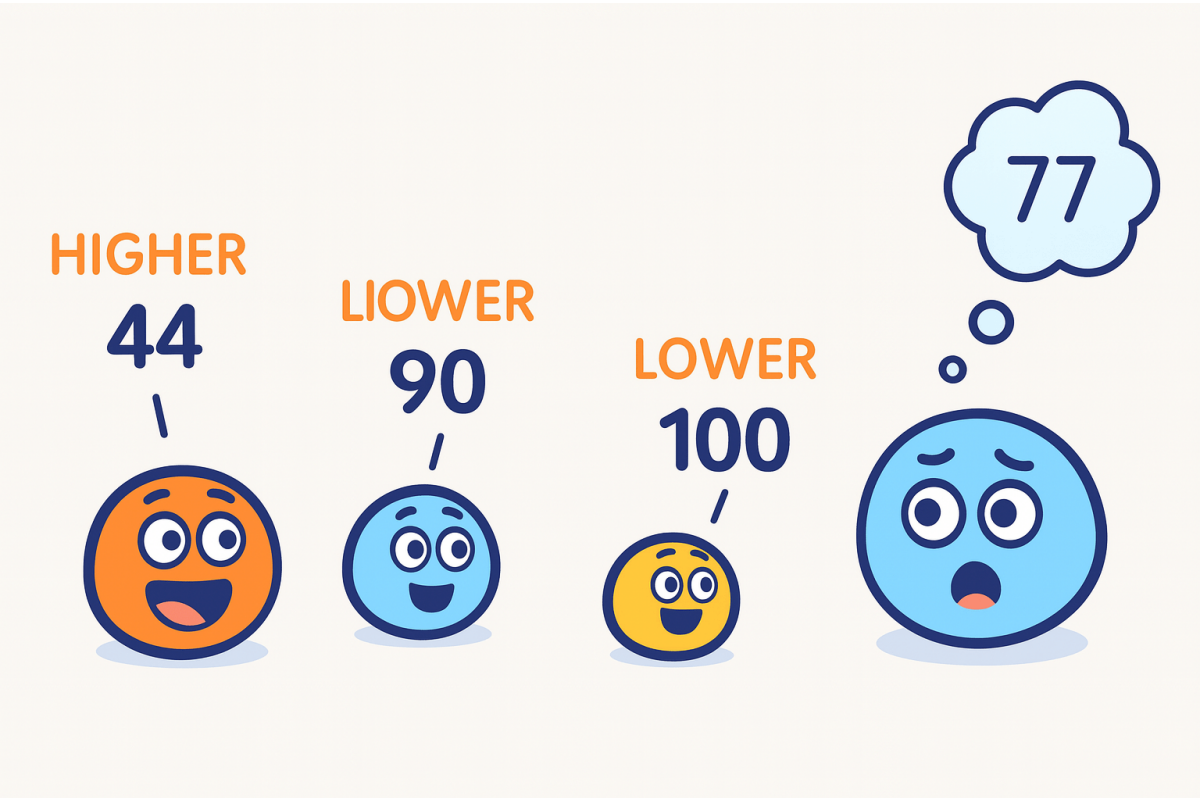
I guess this is too linear

We could solve it by counting up (or down) until we find it.



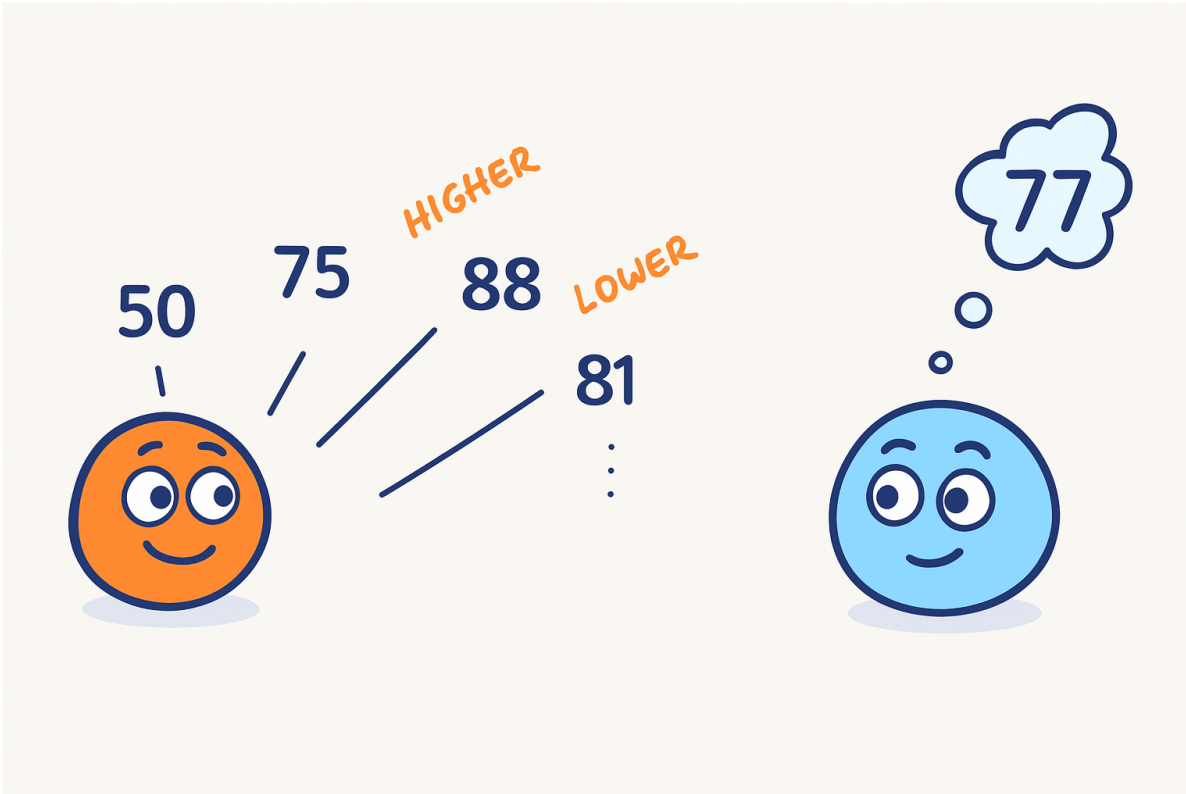
I guess this is just stupid

We could solve it by shouting out random numbers and not listening to the feedback (like my kids did when they were young ;))



I guess this is binary?

We could solve it by starting in the middle (= 50) and guess further by always splitting the range in half.



Yes, it is binary search!

- Intuitively we feel that the **last strategy** is the **most effcient!**
  - you also apply this when looking for a word in a dictionary, when looking for a pagenumer in a book etc..
- This method is called **binary search**
- Typical algorithm
- **Input** = **sorted** list
- **Output** = **position of element** we are looking for or **null/-1/...** (in case we don't find it)

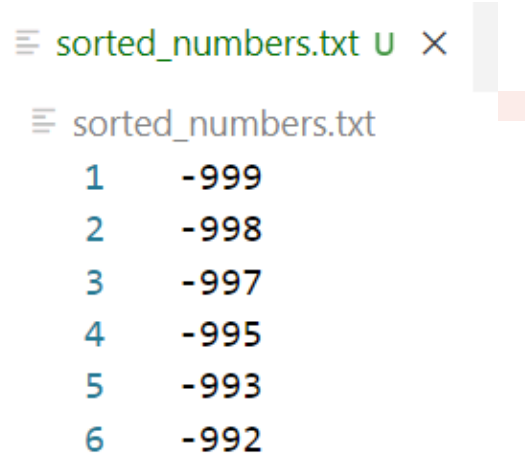
Yes, it is binary search!

- Intuitively we feel that the **last strategy** is the **most effcient!**
  - you also apply this when looking for a word in a dictionary, when looking for a pagenumer in a book etc..
- This method is called **binary search**
- Typical algorithm
- **Input** = **sorted** list
- **Output** = **position of element** we are looking for or **null/-1/...** (in case we don't find it)

Searching & intro to Big O

- ~~Basics of Algorithms~~
- Searching Methods
  - Coding
- Well hello Big O
- How to calculate Big O?

Code the search methods!



- Create a new project!
- Read in a file containing a sorted list of 1000 numbers between -1000 & 1000 (see canvas)
- Store the numbers in an array
- Apply each of the search methods to locate specific numbers.
  - **Linear Search**: Traverse through the dataset one element at a time until the target is found
  - **Stupid Search (Random Guessing)**: Randomly guess an index and check the corresponding element until the target is found.
  - **Binary Search**: Divide the sorted dataset in half repeatedly until the target is found.

Linear search

- **Comparison**: Check if the current element matches the target.
- **Return**: If a match is found, return the index of the element. If no match is found after checking all elements, return -1 or a similar indication of failure.
- **What about efficiency?**
- **Brute Force method:** checking everything!

```
1 FUNCTION LinearSearch(array, target):
2     FOR i = 0 TO length(array) - 1:
3         IF array[i] == target THEN
4             RETURN i  // target found at index i
5         END IF
6     END FOR
7     RETURN -1  // target not found
8 END FUNCTION
```

Binary search

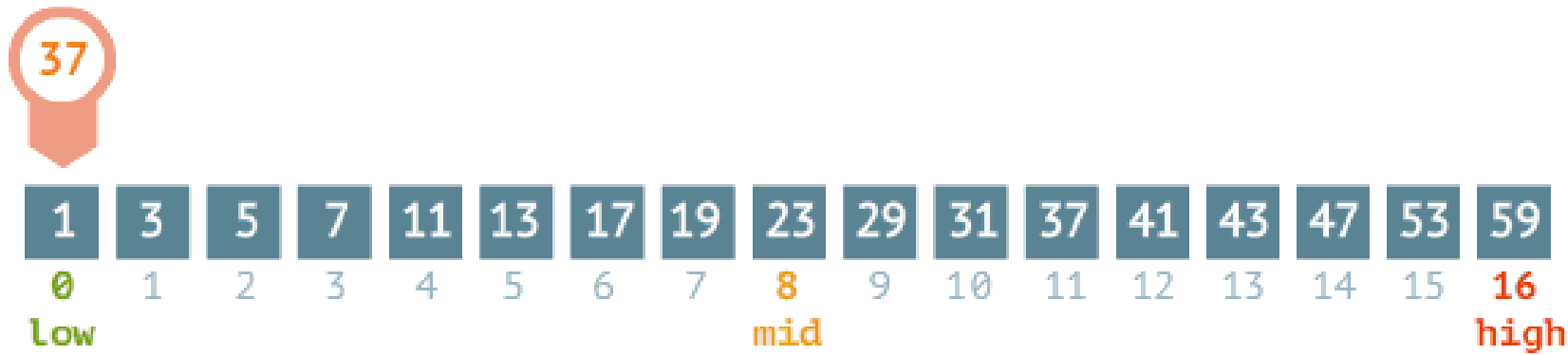
- **Precondition**: List must be sorted.
- **Comparison**: Compare target with middle element.
- **Narrowing Down**
  - If target is larger than middle element, search the right half.
  - If target is smaller, search left half.
- **Return**: If a match is found, return the index of element. If no match is found and list cannot be divided further, return -1

```
1  FUNCTION BinarySearch(array, target):
2      low = 0
3      high = length(array) - 1
4
5      WHILE low <= high:
6          mid = (low + high) / 2
7
8          IF array[mid] == target THEN
9              RETURN mid  // target found at index mid
10         ELSE IF array[mid] < target THEN
11             low = mid + 1
12         ELSE
13             high = mid - 1
14         END IF
15     END WHILE
16
17     RETURN -1  // target not found
18 END FUNCTION
```

What about efficiency?

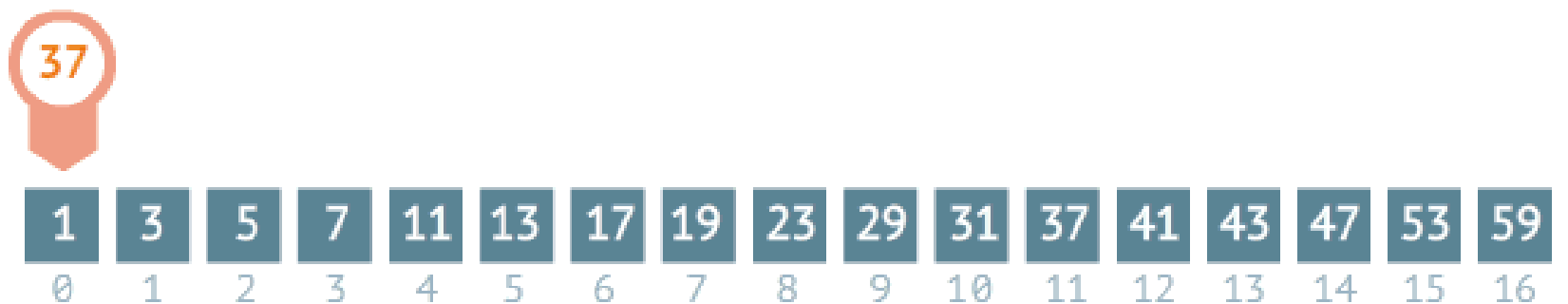
Binary search

steps: 0



Sequential search

steps: 0



Stupid search (for fun ;))

```
FUNCTION StupidSearch(list: ARRAY OF INT, target: INT) -> INT:
    MAX_ATTEMPTS = length of list * 10
    // Define a max number of tries (arbitrary choice)

    attempts = 0

    WHILE attempts < MAX_ATTEMPTS:
        randomIndex = RANDOM integer between 0 and (length of list - 1)

        IF list[randomIndex] == target:
            RETURN randomIndex  // Target found at randomIndex
        END IF

        attempts = attempts + 1
    END WHILE

    RETURN -1  // Target not found after max attempts
END FUNCTION
```

What about efficiency?

- **Random Selection**: Pick an index randomly from the list.
- **Comparison**: Check if the element at the chosen index matches the target.
- **Return**:
  - If match is found, return index of the element.
  - If no match is found after max attempts, return -1

Now which one is best?

- Why do we intuitively feel binary search is better?
- Let's back up our gut feeling by checking how many guesses every search method needs to find the correct position
- How?
  - Add a global variable
  - Or add a parameter to every method
  - Count the loops!
- What do you see? What do you notice?

```
SEARCH 100
LINEAR SEARCH -  550 guesses - INDEX 549
BINARY SEARCH -  8 guesses - INDEX 549
TUPID SEARCH -  871 guesses - INDEX 549

SEARCH 380
LINEAR SEARCH -  1000 guesses - NOT FOUND
BINARY SEARCH -  10 guesses - NOT FOUND
STUPID SEARCH -  10001 guesses - NOT FOUND
```

Number of guesses

- It depends on the method/algorithm how many guesses you need!
- In the worst case, if a number is not in the list/array
  - linear needs 1000 guesses
  - binary needs 10 guesses
  - *we leave stupid out, because it's too random!*
- This means that **a linear, simple search needs 1000 runs through the loop** to find that the number is not in the array, while **binary only needs 10 runs**!
- This is a **huge difference**!

Making the guesses visual

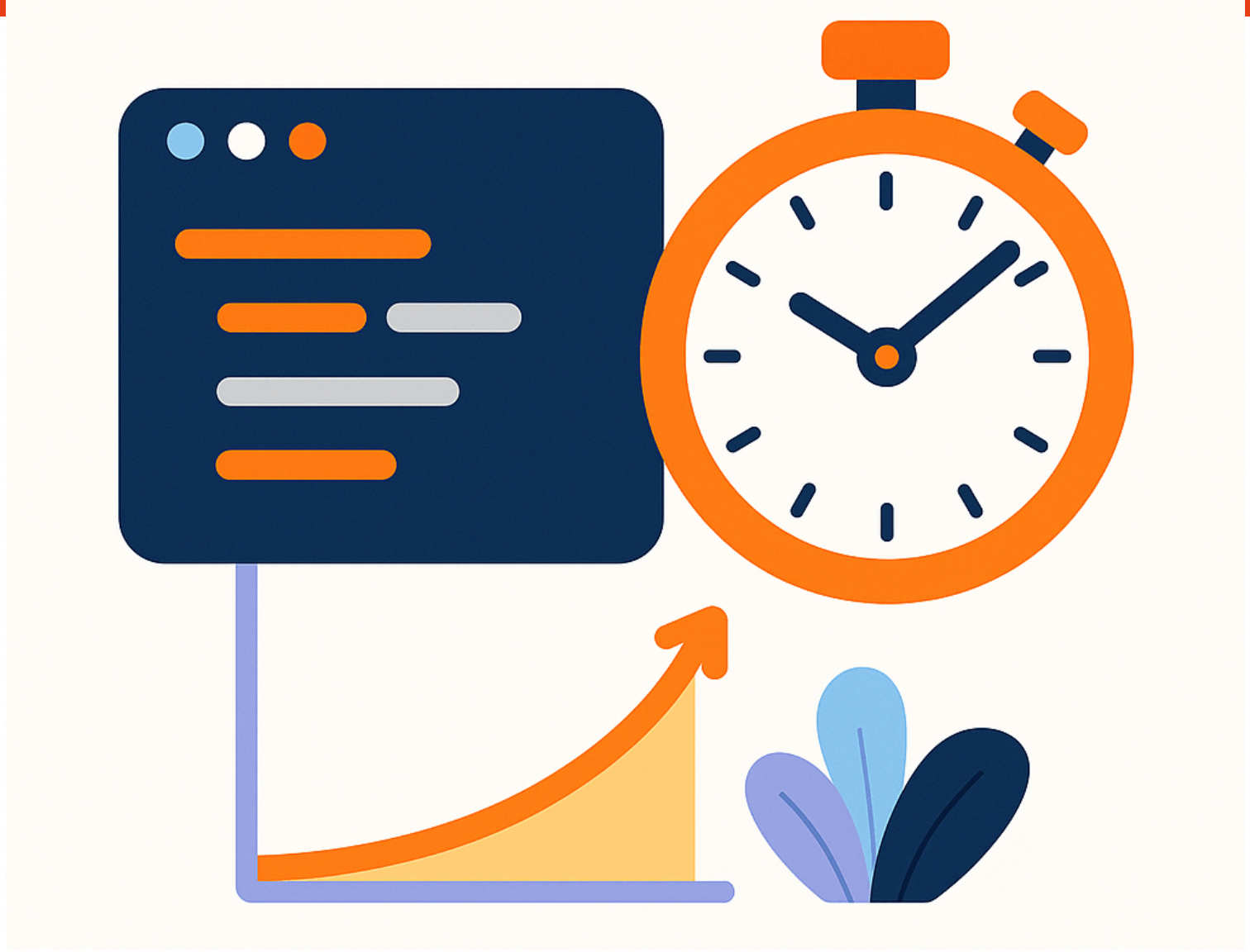
- x = position in array
- y = guesses needed
  - y1 = linear (black)
  - y2 = stupid (red)
  - y3 = binary (blue)

x	y_1	y_2	y_3
0	1	199	9
−1	1001	1000	11
−1	1001	1000	11
−1	1001	1000	11
−1	1001	1000	11
1	2	1000	10
−1	1001	1000	11
988 meer rijen Alles tonen			
494	495	366	10
−1	1001	1000	11
−1	1001	1000	11
−1	1001	1000	11

https://www.desmos.com/calculator/lraembjiy2?embed

Searching & intro to Big O

- ~~Basics of Algorithms~~
- ~~Searching Methods~~
  - ~~Coding~~
- Well hello Big O
- How to calculate Big O?



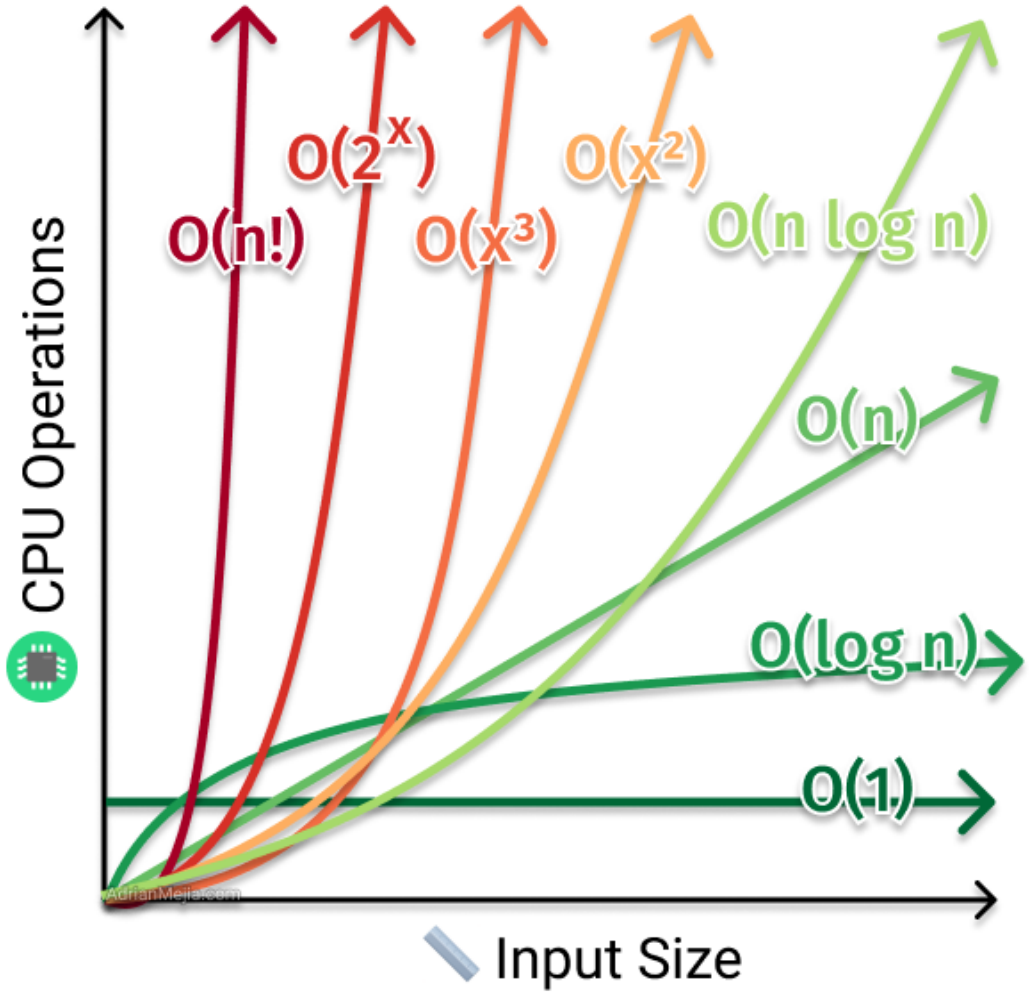
Well hello Big O

Big O notation = high-level understanding of the algorithm in terms of its **time complexity (or space) complexity as the size of the input grows**.

	constant	*logarithmic*	*linear*	*N-log-N*	*quadratic*	*cubic*	*exponential*
n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

Table relating size of data set to amount of computations

⏱️ Time Complexity



NAME	TIME COMPLEXITY
Constant time	O (1)
Logarithmic time	O (log n)
Linear time	O (n)
Quasilinear Time	O(n log n)
Quadratic Time	O(n^2)
Exponential Time	O(2^n)
Factorial Time	O(n!)

Euh... log in Big O?

- What do you remember of logarithms? Euh...
- OK... it is really not that hard, seriously!
- The **logarithm** is the **inverse** function to **exponentiation**

$$\log_b x = y \Leftrightarrow b^y = x \; and \; x > 0, b > 0, b <> 1$$

- Some examples

$$10^2 = 100 \Leftrightarrow \log_{10} 100 = 2 \; (base \; b = 10)$$
$$2^3 = 8 \Leftrightarrow \log_2 8 = 3 \; (base \; b = 2)$$
$$2^5 = 32 \Leftrightarrow \log_2 32 = 5 \; (base \; b = 2)$$

- In **Big O we use always 2 as our base b**, so $\log x = log_2 x$
  - *In computer science everything is binary ;) (base 2 = binary logarithm)*

Back to binary search

- So what's the runtime for simple & binary search?
  - O(log n) = log time (blue)
  - O(n) = linear time (green)
  - O(n * log n) (purple)
  - O(n^2) (black)
  - O(n!) (red)

x_2	y_2	z_2	u_2	v_2	w_2
1	0	1	0	1	1
2	1	2	2	4	2
3	1.584962501	3	4.754887502	9	6
4	2	4	8	16	24
5	2.321928095	5	11.60964047	25	120
6	2.584962501	6	15.509775	36	720

https://www.desmos.com/calculator/argbttvxxb?embed

So that's why binary is better!

- Simple search = O(n)
  - *linear time*
  - runtime grows proportionally to the size of the input data
- Binary search = O(log n)
  - *logarithmic time*
  - runtime grows proportionally to the logarithm of the input size
- **Binary search is faster!**

https://www.desmos.com/calculator/vmt7f1mwdc?embed

But is Big O really that important?

- Let's assume it takes 1 millisecond to check one element..
- Now... tell me what you would choose if you have tons of data to search?

SIMPLE SEARCH

elements to search...	runtime
10	10 ms
100	100 ms
1000	1 sec
10000	10 sec
100000	2 min
1000000	17 min
10000000	3 hours
100000000	1 day
1000000000	12 days

BINARY SEARCH

ms	runtime
10	3 ms
100	7 ms
1000	10 ms
10000	13 ms
100000	17 ms
1000000	20 ms
10000000	23 ms
100000000	27 ms
1000000000	30 ms

Time & Space Complexity

Time complexity

- **Amount of time** an algorithm takes relative to its input size.
- Influenced by **operations** such as loops, recursions, and computations.

Space complexity

- **Amount of memory** an algorithm uses relative to its input size.
- Influenced by **data structures**, variables, and allocations

Time vs. Space Complexity

- Both give **insight** into how **algorithms** scale
- **Expressed using Big O notation**
- Can impact each other: optimizing one might increase the other

  = time-space tradeoff
- Time focuses on duration, while space focuses on memory consumption.
- While both can be crucial, certain applications prioritize one over the other (e.g., embedded systems might prioritize space).

Some time & space complexity examples

- **Time** Complexity: *how execution time increases* with input size.
  - **O(1)**: Constant time. E.g., Accessing an array element by index.
  - **O(n)**: Linear time. E.g., Finding an item in an unsorted list (Linear Search).
  - **O(n^2)**: Quadratic time. E.g., Bubble Sort with nested loops.
  - **O(logn)**: Logarithmic time. E.g., Binary Search on a sorted list.
  - **O(nlogn)**: Linearithmic time. E.g., MergeSort or QuickSort.
- **Space Complexity:** *how memory usage grows* with input size.
  - **O(1)**: Constant space. E.g., Swapping two numbers using a temporary variable.
  - **O(n)**: Linear space. E.g., Creating a new list that's a copy of an input list.

Searching & intro to Big O

- ~~Basics of Algorithms~~
- ~~Searching Methods~~
  - ~~Coding~~
- ~~Well hello Big O~~
- How to calculate Big O?
  - Examples

How to calculate Big O?

- **Understand the Algorithm**: understand behavior and steps, especially in relation to its input.
- **Identify the Basic Operations**: recognize fundamental operations that depend on size of input, includes operations, comparisons, iterations, method calls
- **Count the Operations**: express number of operations in terms of n, where n is the size of the input.
- **Find the Dominant Term**: identify the term that grows fastest as n becomes large = this term will dominate the complexity.
- **Express in Big O Notation**: Once you've identified the dominant term, you can express the algorithm's complexity in Big O notation.

O(1) constant time

- **Constant time to run, regardless of the size of the input.**
- Examples
  - *The use of bookmarks! It allows the reader to find the last page in an efficient manner. It doesn't matter if you are reading a book of 30 pages or one of 1000 pages, you will always find the last page in **one single step**!*
  - math operations
  - accessing array via index
  - returning a value from a function
  - creating a variable

```
1 int[] array = new int[]{{ 78, 12, -3}
2
3 array[0]         //1 single step
4 int sum = 5 + 6 //1 single step
5
6 //O(1) + O(1) + 0(1) = 3*O(1)
7 //--> remove constants = O(1)
8 sum = array[0] + array[1]
```

O(n) linear time

- **Run-time increases at the same pace as the input size**
- Examples
  - *Reading a book: one minute per page means that it will take you 30 minutes for a book of 30 pages and 1000 minutes (16 hours) for a book of 1000 pages.*
  - iterating over elements: for, foreach
  - Creating a new array

```
1  int[] array = new int[]{{ 78, 12, -3}
2
3  int sum = 0;
4
5  foreach (var element in array)  //O(n)
6  {
7          sum += element;         //O(1) + O(1)
8  }
9
10 //Find the highest order term --> O(n)
```

O(n²) quadratic time

- **Runs in quadratic time, which is the squared size of the input data.**
- Examples:
  - two nested loops
    - three nested loops = O(n³) *Cubic time*
  - bubble sort
  - creating a matrix

```
1  int[] array = new int[]{ 78, 12, -3}
2
3  int sum = 0;
4
5  foreach (var element in array)  //O(n)
6  {
7          for (int i = 0; i < array.length; i++) //O(n)
8      {
9          // do something
10     }
11 }
12
13 //O(n)*o(n) = O(n²)
```

Searching & intro to Big O

- ~~Basics of Algorithms~~
- ~~Searching Methods~~
  - ~~Coding~~
- ~~Well hello Big O~~
- How to calculate Big O?
  - Examples

Example 1

- **Time complexity = O(n)**
  - The loop runs (n - 1) times,

    where n is the length of the array = O(n)
  - Within the loop:
    - Generating random number is O(1)
    - Swapping two numbers is O(1)
  - Dominant operation is the loop
- **Space Complexity = O(1)**
  - the input array is modified in place (no extra space)
  - some extra variables O(1)

```
1  public static void Example(int[] array)
2  {
3    int n = array.Length;
4    for (int i = n - 1; i > 0; i--)
5    {
6      int j = rng.Next(0, i + 1);
7      int temp = array[i];
8      array[i] = array[j];
9      array[j] = temp;
10   }
11 }
```

Example 2

```
1  public static int[] Example(int[] array)
2     {
3         int n = array.Length;
4         int[] result = new int[n];
5         result[0] = array[0];
6
7         for (int i = 1; i < n; i++)
8         {
9             int j = i;
10            while (j >= 0)
11            {
12                result[i] += array[j];
13                j--;
14            }
15        }
16        return result;
17    }
```

- **Time complexity = O(n^2)**
  - The outer `for` loop runs `n` times.
  - For each iteration `i` of the outer loop, the inner `while` loop runs `i` times (in the worst case, for `i = n`, it runs `n` times).
- **Space Complexity = O(n)**
  - new array `result` = same size as the input array = O(n).
  - Other variables = O(1)

Example 3

```
1  public static void Example(int n)
2  {
3    // Create a 2D matrix of size n * n
4    int[,] matrix = new int[n, n];
5
6    // Fill the main diagonal of the matrix
7    for (int i = 1; i < n; i *= 2)
8    {
9      matrix[i, i] = 1;
10   }
11 }
```

- **Time complexity = O(log n)**
  - The for-loop filling of the main diagonal runs in logarithmic time since we're doubling the index with each iteration.
- **Space Complexity = O(n^2)**
  - The matrix has n * n elements, resulting in O(n^2) space complexity.

- *Binary search is a lot faster than simple search.*
- *O(log n) is faster than O(n), but it gets a lot faster once the list of items you're searching grows*
- *Algorithm time & space complexity is written in Big O*
- *Big O notation = high-level understanding of the algorithm in terms of its time complexity (or space) complexity as the size of the input grows.*

How to Analyze

- **Purpose**: Measuring and evaluating algorithm performance
- **Why it matters**: Before building solutions, you need to understand what makes one algorithm better than another. This helps you choose the right approach for different problems.
- **Real-world impact**: Knowing why Instagram loads faster than a poorly designed app, or why Google search returns results in milliseconds.

Sorting

Programming advanced

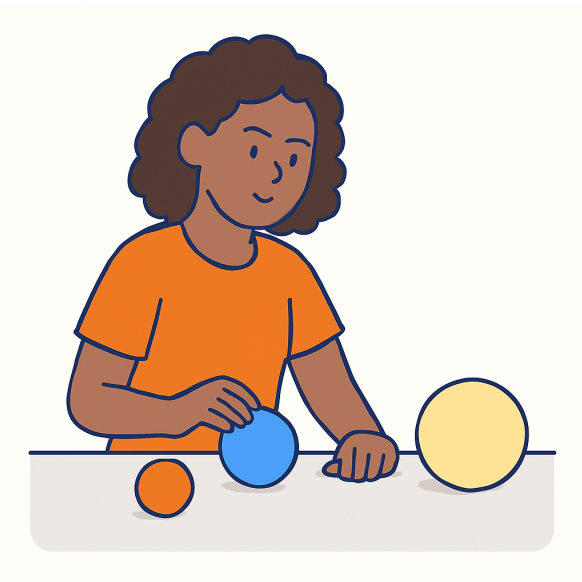
YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Sorting

- Why sorting?
- Bubble sort
- Selection sort
- Insertion sort
- What about Big O?

What is Sorting?



- Organizing items by rules
- Arranging items in a specific order
  - Most simplest form = **ascending** (smallest to largest) or **descending** (largest to smallest)
  - Numbers, letters, names, dates but also categories, colors, priority levels etc...

Why do we need Sorting?

- **Sorting is important in Computer Science**
  - **a cornerstone concept**

- Sorting **is** a **fundamental** operation!
  - **Searching** becomes **faster:** *easier to find items in organized data*
  - **Data** looks **cleaner:** *humans prefer organized information*
  - **Algorithms** work **better:** *many algorithms require sorted input*
  - **Data analysis** - patterns and trends become visible when data is ordered

Sorting in real life

- **Organizing your closet:** Arrange clothes by type (shirts, pants, jackets), then by color or season
- **Library books**: Arranged alphabetically by author's last name so you can find any book quickly
- **Grocery shopping**: Arrange your shopping list by store layout (produce, dairy, meat) to shop efficiently
- **Sports rankings**: Teams sorted by wins to determine rankings
- **Restaurant menus**: Dishes organized by category (appetizers, mains, desserts) and often by price
- ...

Sorting algorithms

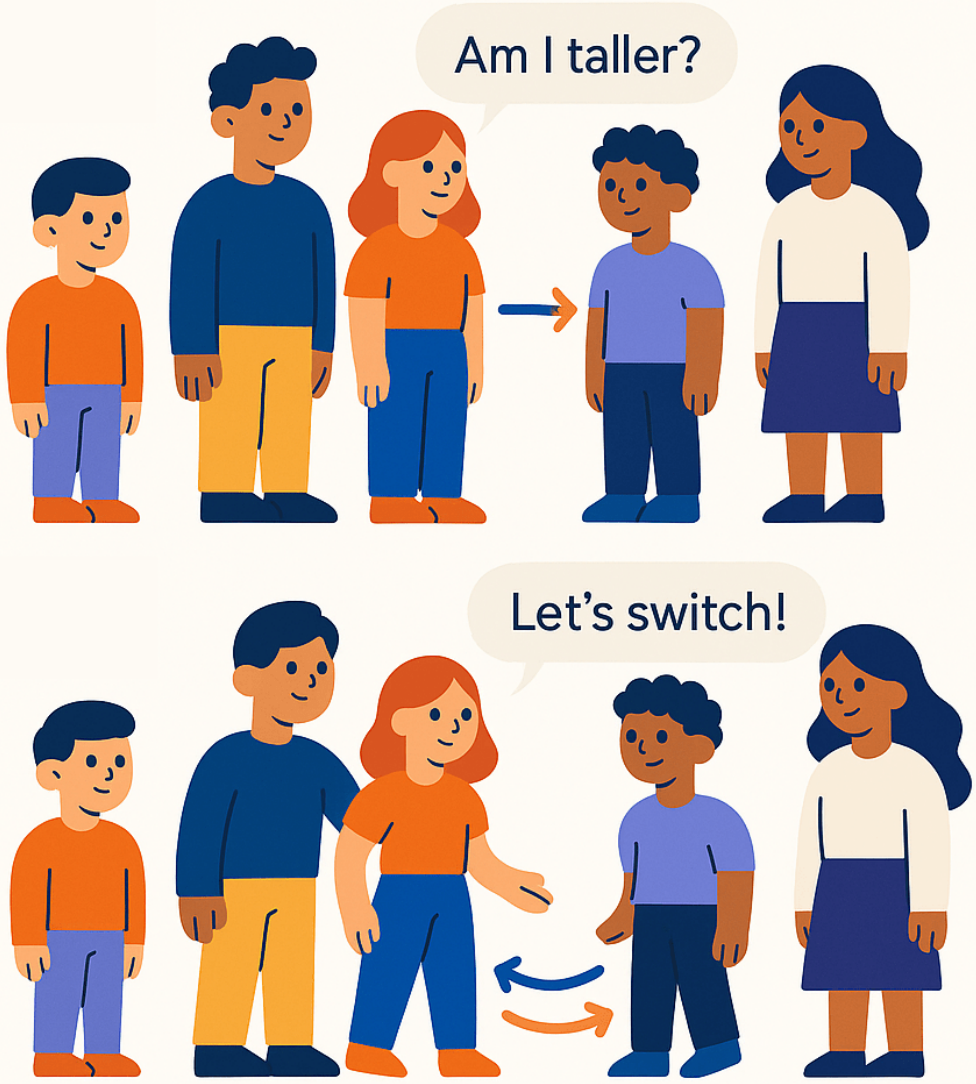
- Wikipedia lists 43 different sorting algorithms:
  - *Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Counting Sort, Radix Sort, Bucket Sort, Shell Sort, Tim Sort, Cocktail Shaker Sort, Comb Sort, Gnome Sort, Odd-even Sort, Cycle Sort, Pancake Sorting, Stooge Sort, Bogo Sort, Intro Sort, Patience Sorting, Permutation Sort, Strand Sort, Tournament Sort, Gravity Sort, Smoothsort, Block Sort, Pigeonhole Sort, Burstsort, Library Sort, Franceschini's Method, Flashsort, Binary Tree Sort, Cartesian Tree Sort, Bitonic Sorter, Batcher Odd-even Mergesort, Cube Sort, Spaghetti Sort, Splaysort, Squid Sort, American Flag Sort, Postman Sort, Brick Sort*
- Each algorithm has unique characteristics and implementations!

Why so many sorting algorithms?

- **Different Performance Trade-offs:** Some algorithms are faster but use more memory, others are slower but memory-efficient
- **Learning vs. Real-world use**
- **Data-Specific Optimizations**: nearly sorted data, small datasets <> large datasets
- **Different Constraints and Requirements:**
  - Stable vs. unstable: Some preserve original order of equal elements
  - In-place vs. external: Some sort without extra memory
- **No single "best" algorithm exists** - the right choice depends on your specific data, constraints, and requirements!

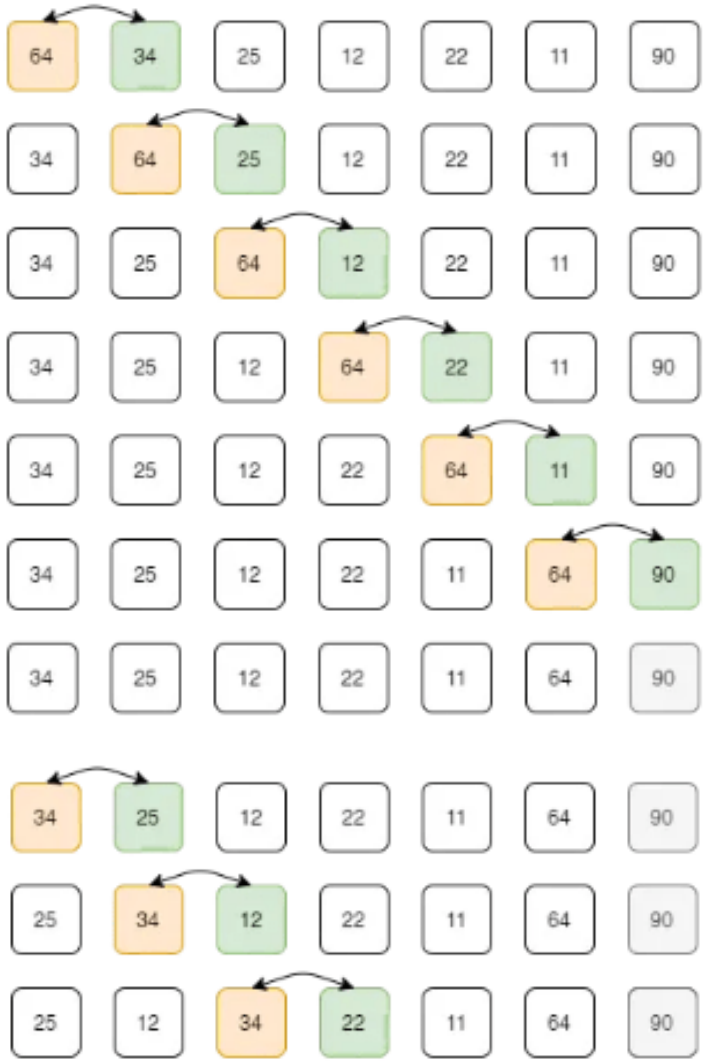
Sorting

- ~~Why sorting?~~
- Bubble sort
- Selection sort
- Insertion sort
- What about Big O?


Am I taller?
Let's switch!

Bubble sort

- Scan the array, swapping adjacent pair of elements if they are not in order. This **bubbles up** the **largest element to the end**.
- Scan the array again, bubbling up the second largest element.
- etc... repeat until all elements are sorted



Bubble sort

- Pseudo code, try to convert in to C#-code
  - Input array = [32, 7, 45, 23, 41, 19, 28, 50, 1, 37]
- See in action: https://visualgo.net/en/sorting

```
1  Procedure BubbleSort(A, n):
2      Input: An array A of n elements.
3      Output: Sorted array A in non-decreasing order.
4
5      For i from 0 to n - 2 Do:
6          For j from 0 to n - 2 - i Do:
7              If A[j] > A[j + 1] Then
8                  Swap A[j] and A[j + 1]
9              End If
10         End For
11     End For
12 End Procedure
```

Bubble sort

- Why start at array.length-1?
- Output intermediate steps and see the magic!

```
32 7 45 23 41 19 28 50 1 37
7 32 23 41 19 28 45 1 37 50
7 23 32 19 28 41 1 37 45 50
7 23 19 28 32 1 37 41 45 50
7 19 23 28 1 32 37 41 45 50
7 19 23 1 28 32 37 41 45 50
7 19 1 23 28 32 37 41 45 50
7 1 19 23 28 32 37 41 45 50
1 7 19 23 28 32 37 41 45 50
1 7 19 23 28 32 37 41 45 50
```

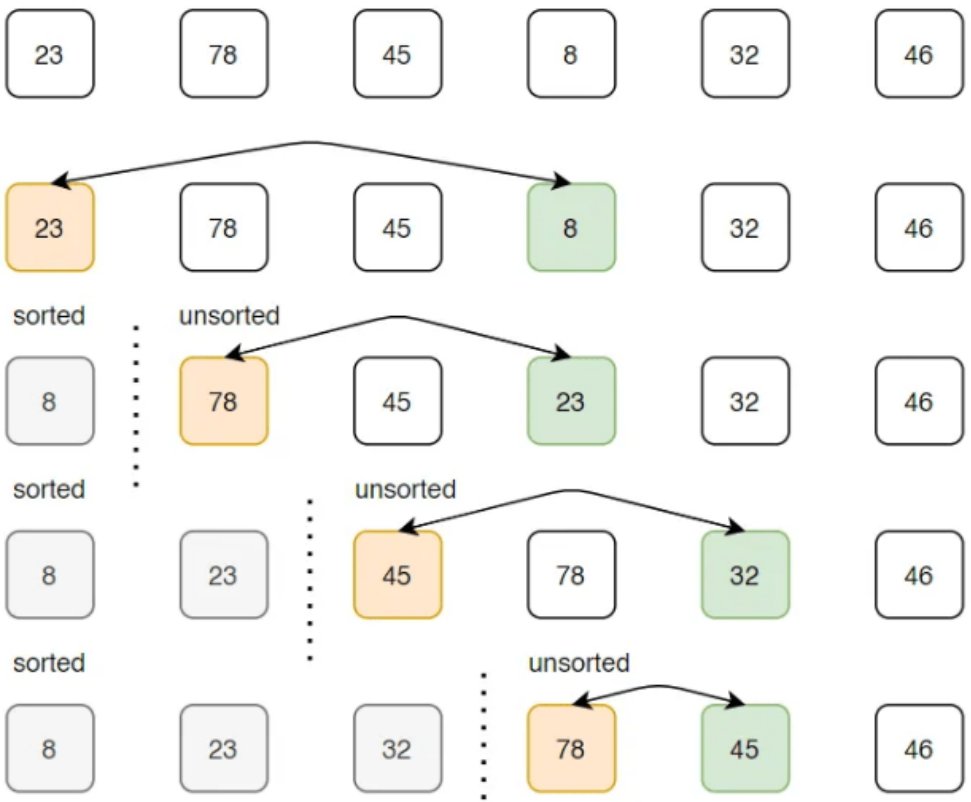
Sorting

- ~~Why sorting?~~
- ~~Bubble sort~~
- Selection sort
- Insertion sort
- What about Big O?

Who's the shortest?
You go first!

Selection sort

- **Find the smallest element**, and put it to the first position.
- Find the next smallest element, and put it to the second position.
- Repeat until all elements are sorted



Selection sort

- Pseudo code, try to convert in to C#-code
  - Input array = [32, 7, 45, 23, 41, 19, 28, 50, 1, 37]
- See in action: https://visualgo.net/en/sorting

```
1  Procedure SelectionSort(A, n):
2      Input: An array A of n elements.
3      Output: Sorted array A in non-decreasing order.
4
5      For i from 0 to n - 2 Do:
6          minIndex = i
7          For j from i + 1 to n - 1 Do:
8              If A[j] < A[minIndex] Then
9                  minIndex = j
10             End If
11         End For
12         If minIndex ≠ i Then
13             Swap A[i] and A[minIndex]
14         End If
15     End For
16 End Procedure
```

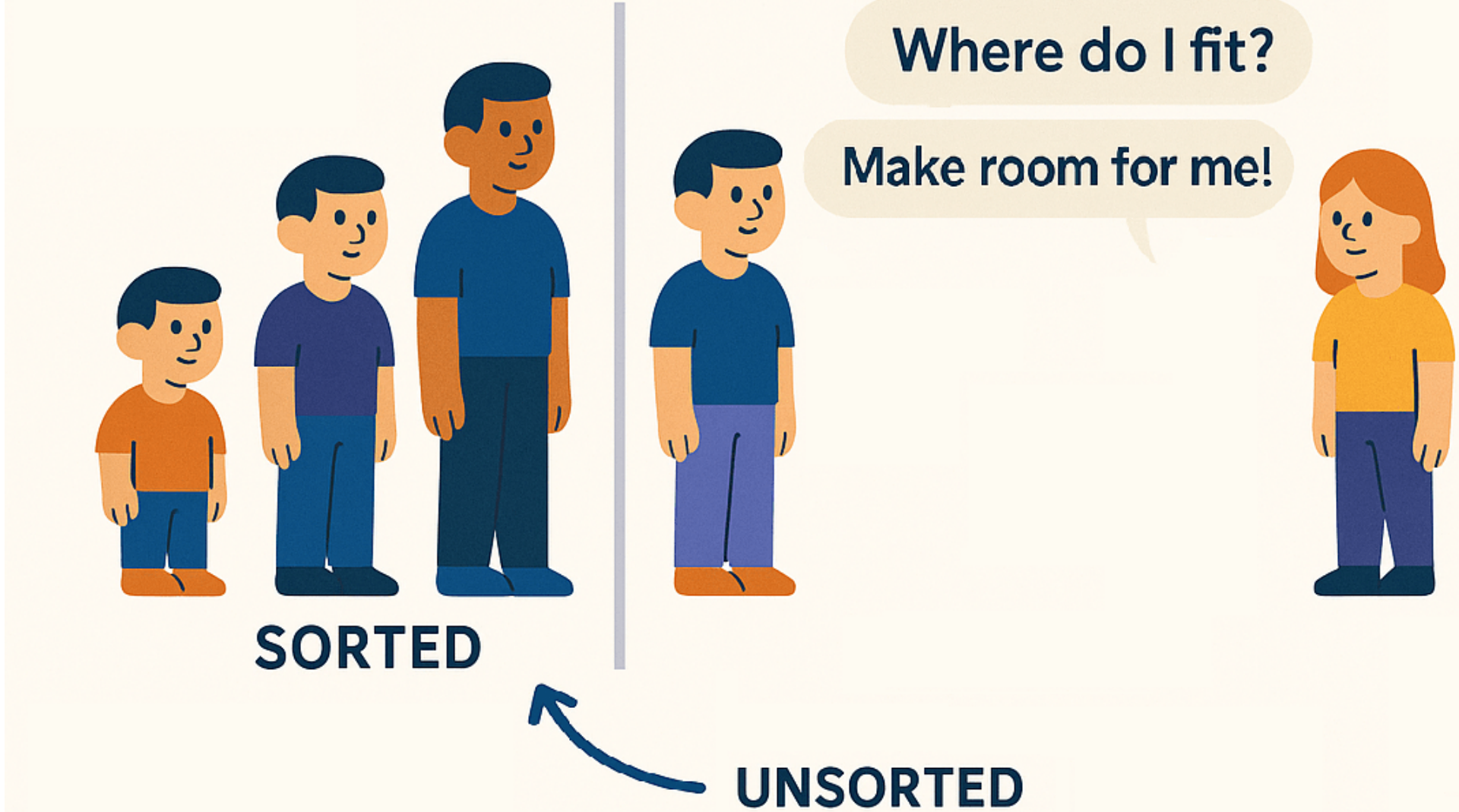
Selection sort

- Why start at i+1?
- Output intermediate steps and see the magic!

```
32 7 45 23 41 19 28 50 1 37
1 7 45 23 41 19 28 50 32 37
1 7 45 23 41 19 28 50 32 37
1 7 19 23 41 45 28 50 32 37
1 7 19 23 41 45 28 50 32 37
1 7 19 23 28 45 41 50 32 37
1 7 19 23 28 32 41 50 45 37
1 7 19 23 28 32 37 50 45 41
1 7 19 23 28 32 37 41 45 50
```

Sorting

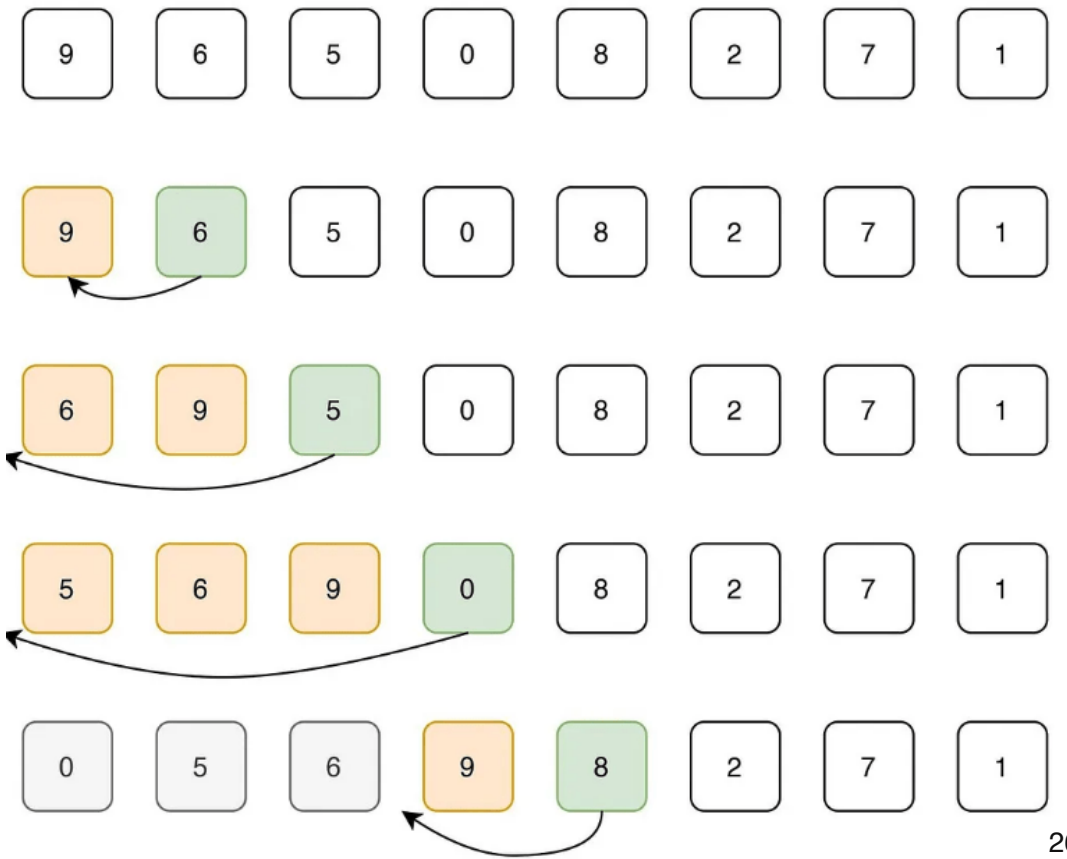
- ~~Why sorting?~~
- ~~Bubble sort~~
- ~~Selection sort~~
- Insertion sort
- What about Big O?


Where do I fit?
Make room for me!
SORTED
UNSORTED

Insertion sort

“ Human way of sorting!

- Take the first element as a sorted subarray
- **Insert** the second element **into the sorted subarray** (shift elements if needed)
- Insert the third element into the sorted subarray
- Repeat until all elements are inserted.



Insertion sort

- Pseudo code, try to convert in to C#-code
  - Input array = [32, 7, 45, 23, 41, 19, 28, 50, 1, 37]
- See in action: https://visualgo.net/en/sorting

```
1  Procedure InsertionSort(A, n):
2      Input: An array A of n elements.
3      Output: Sorted array A in non-decreasing order.
4
5      For i from 1 to n - 1 Do:
6          key = A[i]
7          j = i - 1
8
9          // Shift elements of A[0..i-1], that are greater than key,
10         // to one position ahead of their current position
11         While j >= 0 and A[j] > key Do:
12             A[j + 1] = A[j]
13             j = j - 1
14         End While
15
16         A[j + 1] = key
17     End For
18 End Procedure
```

Insertion sort

- Why start at j = i?
- Why break?
- Output intermediate steps and see the magic!

```
32 7 45 23 41 19 28 50 1 37
32 7 45 23 41 19 28 50 1 37
7 32 45 23 41 19 28 50 1 37
7 32 45 23 41 19 28 50 1 37
7 23 32 45 41 19 28 50 1 37
7 23 32 41 45 19 28 50 1 37
7 19 23 32 41 45 28 50 1 37
7 19 23 28 32 41 45 50 1 37
7 19 23 28 32 41 45 50 1 37
1 7 19 23 28 32 41 45 50 37
```

Sorting

- ~~Why sorting?~~
- ~~Bubble sort~~
- ~~Selection sort~~
- ~~Insertion sort~~
- What about Big O?
- Let's do some pruning!

Count the steps!

- Sort an array of 1000 random elements (numbers.txt, canvas)

```
1 int[] array = Array.ConvertAll(File.ReadAllLines("numbers.txt"), Convert.ToInt32);
```

  - Count the steps
    - Bubble: 499500
    - Selection: 499500
    - Insertion: 250904

- Sort the sorted_numbers file from last lesson (sorted_numbers.txt, canvas)

```
1 int[] array = Array.ConvertAll(File.ReadAllLines("sorted_numbers.txt"), Convert.ToInt32);
```

  - Count the steps
    - Bubble & selection: 499500
    - **Insertion: 999**

Count the loops!

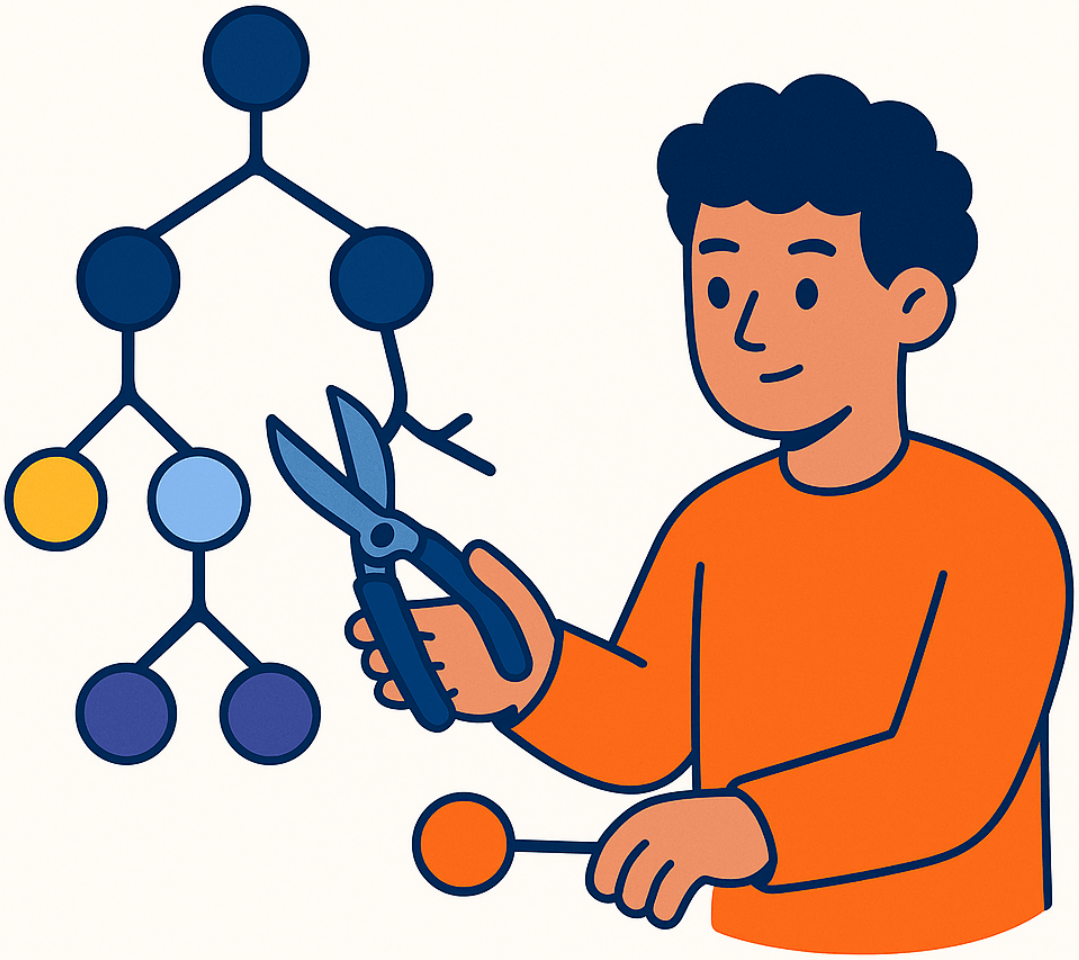
- 2 for-loops in every method... so it means quadratic (aka squared)
- Or just to be sure calculate
  - $O(n) * (O(n)) + O(1))$
    - constants are ignored in Big O
  - $= O(n * (n + 1)) = O(n^2 + n)$
  - $= O(n^2)$
- It's the same for all 3 sorting methods, but in the previous slide we see a difference between worst-case scenario and best case scenario!
  - Worst case = random numbers
  - Best case = sorted numbers

Bubble <> selection <> insertion

- **Bubble, Selection, and Insertion = quadratic time complexity**
- **All naive approaches, but some better than others**
- **Selection & Bubble: always do maximum work**
  - Selection: always scans entire remaining array
  - Basic bubble: always does n-1 passes, even if already sorted
- **Insertion: more intelligent - adapts to the data**
  - Fewer comparisons needed when data is partially sorted
  - If list is completely sorted, complexity drops to O(n)
  - Natural "early stopping" when element finds its place
- **All 3 algorithms sort "in-place", so space complexity = O(1)**

Sorting

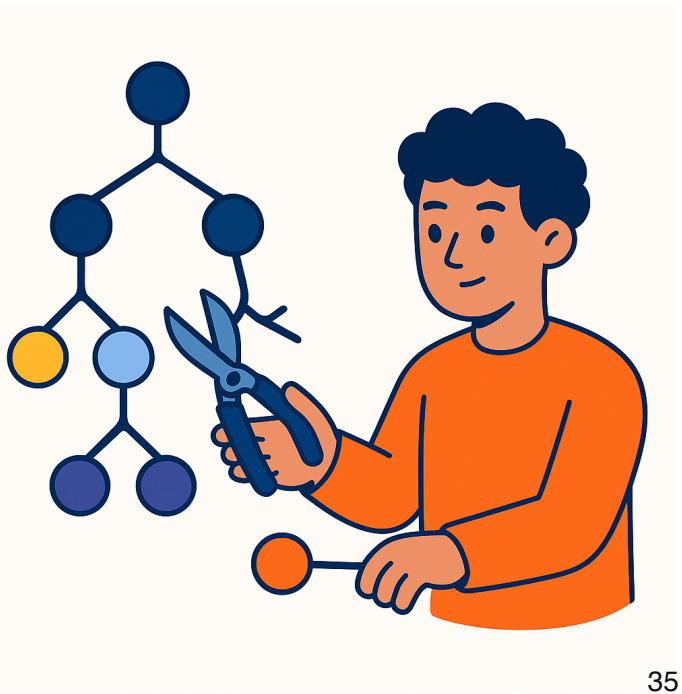
- ~~Why sorting?~~
- ~~Bubble sort~~
- ~~Selection sort~~
- ~~Insertion sort~~
- What about Big O?
- Let's do some pruning!



Pruning: make algorithms smarter!

How to Code

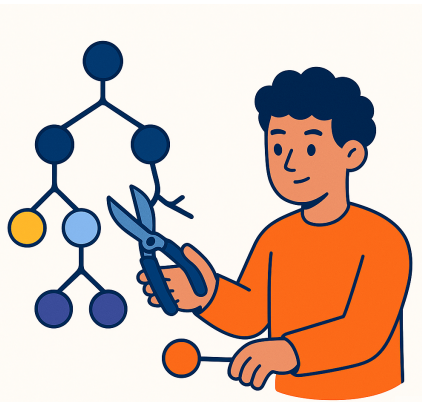
- **Pruning = cutting away unnecessary work**
  - Stop processing when you know the answer won't change
  - Skip iterations, branches, or calculations that add no value
- **Why is Pruning Important?**
  - Transforms brute force into intelligent algorithms
  - Dramatically improves best-case and average-case performance
  - Saves computational resources (time, memory, energy)



Pruning: make algorithms smarter!

How to Code

- **Common Pruning Techniques:**
  - **Early termination**: Stop loops when goal is reached
  - **Flags**: Track completion status to avoid redundant work
  - **Bounds checking**: Skip impossible or unnecessary ranges
  - **Memoization**: Remember results to avoid recalculation
- **Pruning in Real Life:**
  - GPS navigation: stop searching when destination is found
  - Web search: stop after finding enough relevant results
  - Game AI: abandon losing moves early
- ***Work smarter, not harder!***



Speeding up the bubble: pruning

How to Code

- **Can we optimize Bubble sort by making it smarter?**
  - Take a look at bubble sort! Could we maybe plant a flag somewhere?
- **Pruning with Flags**
  - Pruning = cutting away unnecessary work
  - **Flag = boolean variable to track state/status**
  - Add a flag to detect when array is sorted
  - If no swaps happen in a pass → flag stays false → we're done!
  - This transforms worst-case into best-case performance
- From naive to intelligent algorithm!
  - Basic bubble: brute force approach (always n-1 passes)
  - Optimized bubble: adaptive approach (stops early when sorted)

Speeding up the bubble: plant a flag!

How to Code

- **Flag usage = fundamental coding pattern**
  - Start with flag = false (assuming work needs to be done)
  - Set flag = true when condition is met
  - If flag stays false after a pass → exit early
- **Flag applications in algorithms:**
  - Track completion status
  - Control loop termination
  - Optimize algorithms by avoiding unnecessary iterations

Speeding up the bubble: plant a flag!

How to Code

- **Add the flag to your code**
- **Count the steps**
  - Bubble: 999
  - Selection: 499500
  - Insertion: 999
- **Flag transforms**

  $O(n^2)$ worst-case into

  O(n) best-case!

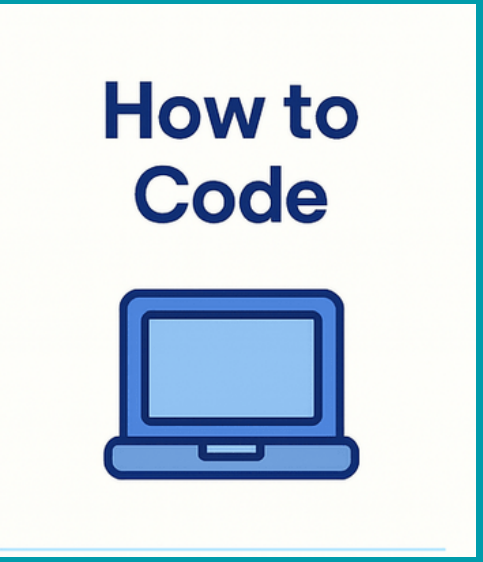
```
1  REPEAT:
2    flag = false
3    FOR each element:
4      IF work_needed THEN:
5        do_work()
6        flag = true
7      END IF
8    END FOR
9  UNTIL flag == false
10 // No work was done, we're finished!
```

Worst case <> best case!

Algorithm	Time Complexity (Best)	Time Complexity (Average)	Time Complexity (Worst)	Space Complexity
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

- **Worst-Case**: longest time an algorithm might take, no matter the input
- **Best and Average Cases**: understand usual performance, but prepare for the worst!
- **Pruning**: Simple optimizations can dramatically change algorithm behavior!
- *So whatever you do, don't pick selection, because it's always slow, no matter the case.*

- *Simple sorting algorithms: bubble, insertion & selection!*
- *Insertion = human way of sorting*
- *Big(O) = quadratic = very slow!*
- *Pruning = cutting away unnecessary work, use flags!*



- **Purpose**: Implementation techniques and coding patterns
- **Why it matters**: Having the right coding tools lets you turn your problem-solving ideas into working programs efficiently.
- **Real-world impact**: Writing clean, maintainable code that other developers can understand and extend.

Data structures

Programming advanced

YP0822 - YP0847

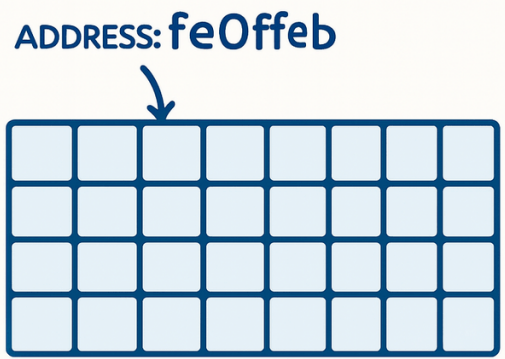
Elke Boonen & Rik Tonnelier

Data structures

- How memory works
- Data structures
- Build your linked list!
- And a stack...

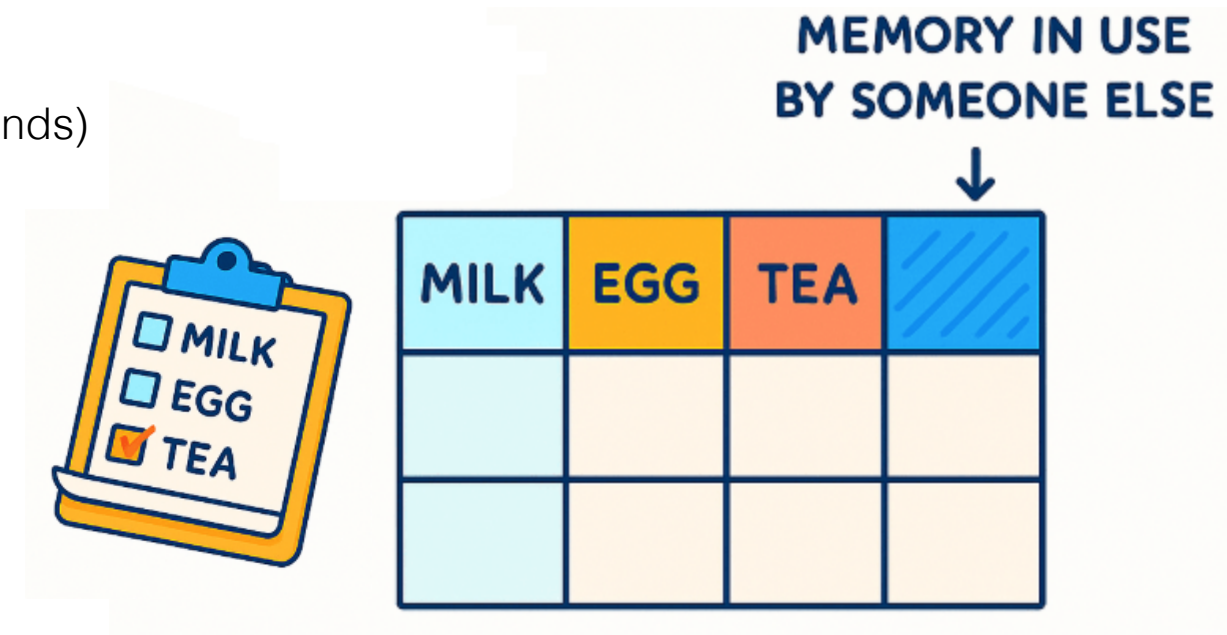
How memory works...

- Memory = **giant set of drawers**
- Each drawer has an **address** and can **hold** an **element**
- Every time you want to store an item: **ask** your computer for some **space**!
- If we want to store **multiple items**: we use **arrays or lists**!



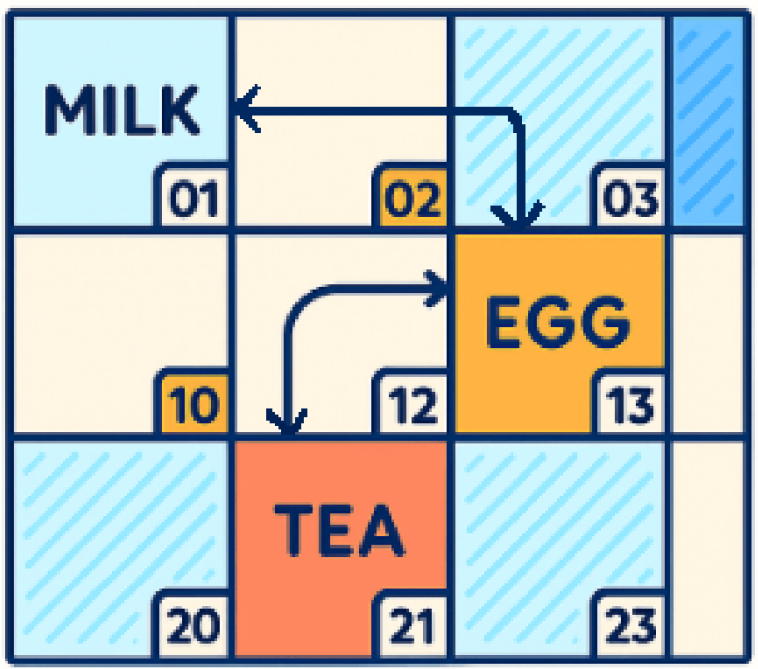
Arrays

- All items are stored right next to each other!
- What with a fourth task?
  - move all items?
  - or hold 'seats' at the beginning?
    (like going to the movies with friends)



Linked lists

- Your items can be anywhere!
- Each item stores also the address of the next item in the list!
  - find next in list = treasure hunt ;)
- A bunch of addresses are linked together!
- With linked lists = never have to move your items around!

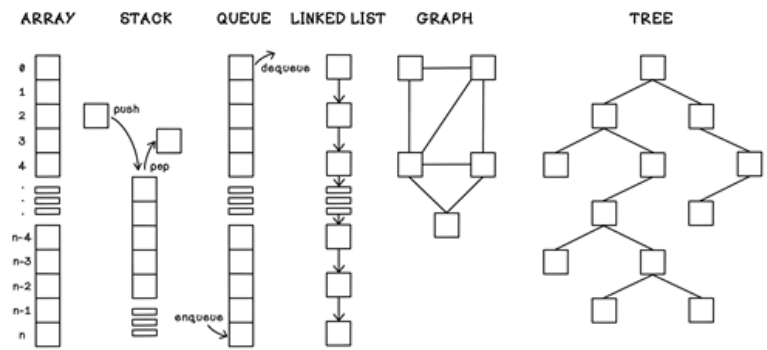


Data structures

- ~~How memory works~~
- Data structures
- Build your linked list!
- And a stack...

What are Data Structures?

- Linked lists & arrays are data structures
- **Data structures**: collection of values organized for efficient access and modification**:**
  - ***Linear: arrays, lists, stack, queue...***
  - **Tree**: binary, heaps, space partitioning...
  - **Hash**: distributed hash table, hash tree...
  - **Graphs**: decision, directed, acyclic...



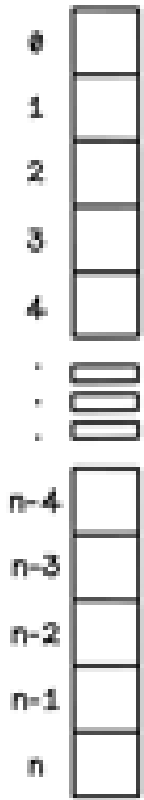
- It always **depends on problem & desired goal** which one to take
- Data structures are a big field in CT (like almost everything ;))

Linear Data Structures?

- **What are linear data structures?**
  - Data elements arranged in a sequence, one after another
  - Each element has a predecessor and successor (except first/last)
  - Examples: arrays, linked lists, stacks, queues
- **Why do we need different linear data structures?**
  - **Different access patterns** - some need random access, others sequential
  - **Different performance requirements** - fast insertion vs fast searching
  - **Different memory constraints** - fixed size vs dynamic size
  - **Different usage patterns** - LIFO, FIFO, or random access

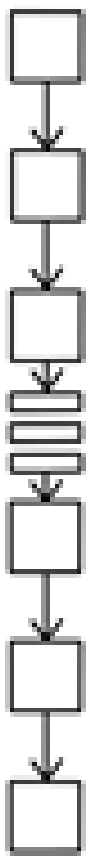
Array

Linear Data Structure



- Stores **homogeneous elements** in adjacent locations, size must be specified before saving data
- **Time needed** for
  - Access = O(1) (specific locations)
  - Search = O(n) simple search, O(log n) binary search if sorted
  - Insert = O(n) (in case of shifting)
  - Delete = O(n)
- **Real life**: Contact list, student grades, inventory items

Linked list

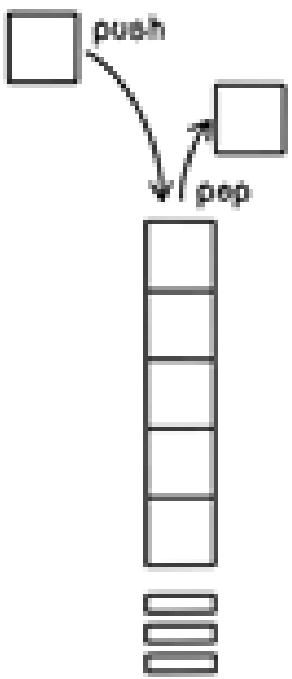


Linear Data Structure

- Each element is a **separate** object
- Element (**node**) consists out of the data and reference to next node
  - singly linked: only reference to next node
  - doubly linked: reference to next and previous node
  - circular: last node links to first - list makes circle
- Time needed for
  - Access = O(n)
  - Search = O(n)
  - Insert = O(1)
  - Delete = O(1)
- **Real life**: Music playlist, train cars, chain of command

Stack

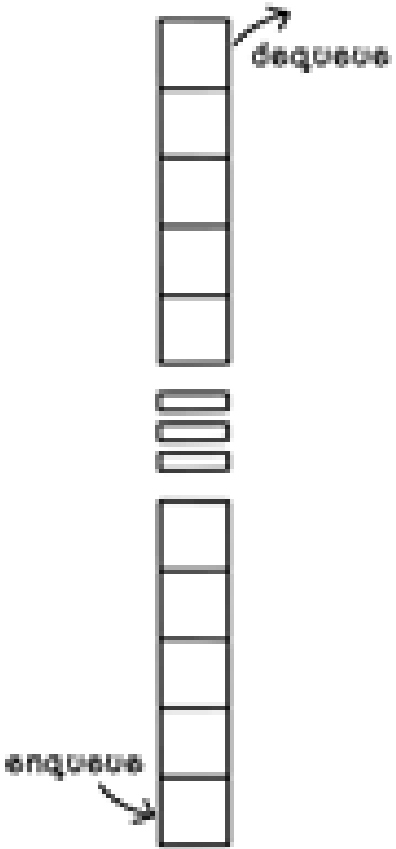
Linear Data Structure



- LIFO (last in, first out)
- Two prinipal operations at top of stack
  - **push:** adds element to the collection
  - **pop**, removes last element that was added
- It can be implemented by using both array and linked list.
- Time needed for
  - Access = O(n)
  - Insert = O(1)
  - Delete = O(1)
- **Real Life**: Browser back button, undo operations, function calls

Queue

Linear Data Structure



- FIFO (first in, first out)
- Two prinipal operations
  - **dequeue:** adds element to the collection (end of queue)
  - **enqueue**, removes first element added (front queue)
- It can be implemented by using both array and linked list.
- Time needed for
  - Access = O(n)
  - Insert = O(1)
  - Delete= O(1)
- **Real Life:** Printer queue, customer service line, task scheduling

Why code the Data Structures?

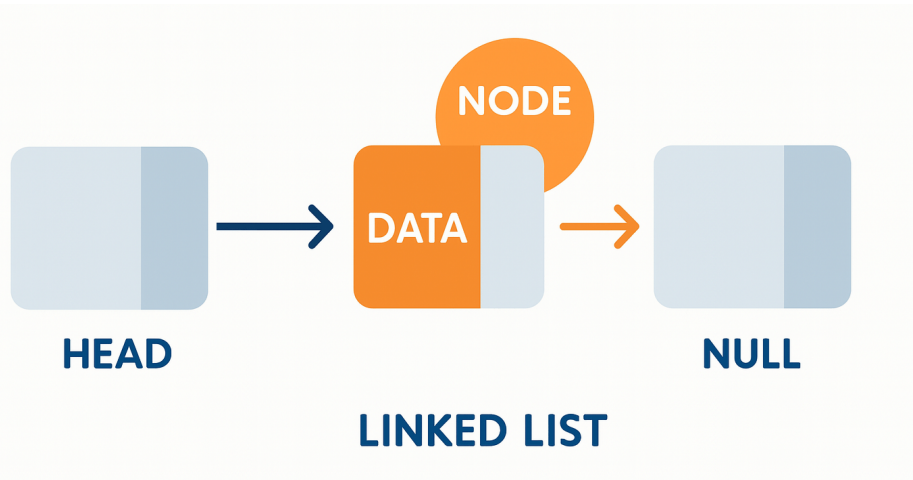
- Why Learn Data Structures When Libraries Exist?
  - **Understanding the fundamentals:** better usage of the tools
  - Know when to use what
  - Performance awareness - understand why some operations are fast/slow
  - Debug effectively - know what's happening under the hood when things go wron
- Real-world development:
  - **Use built-in libraries - C# has Stack<T>, Queue<T>, List<T>, Array**
  - Don't reinvent the wheel - these are tested, optimized, and maintained
  - Focus on your application logic instead of low-level implementation

Data structures

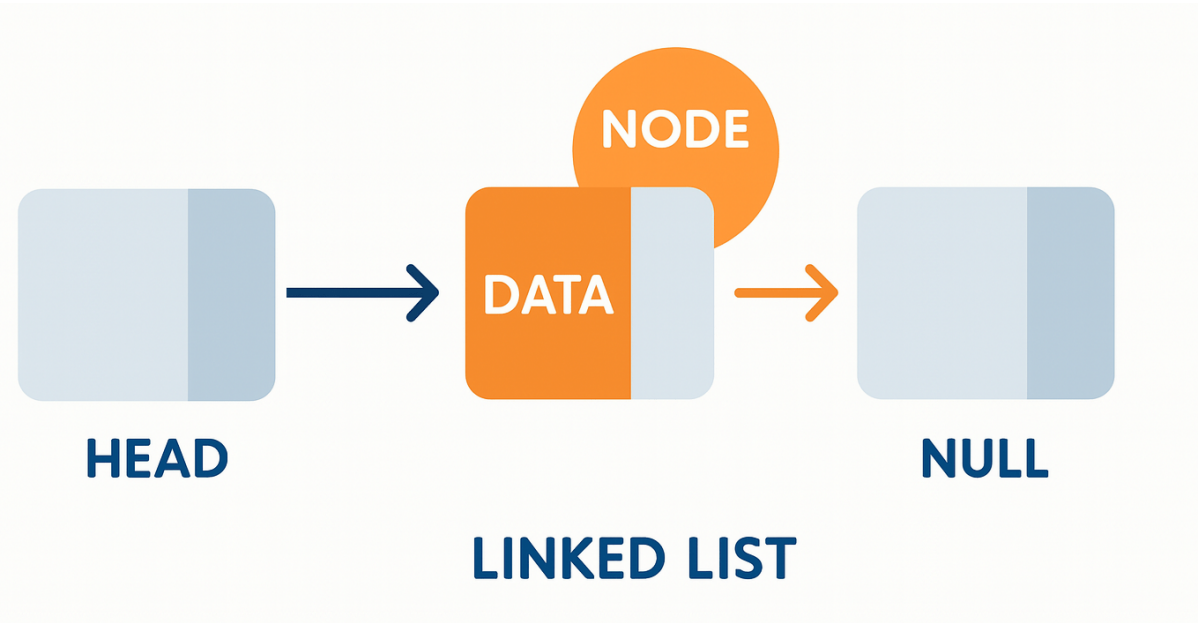
- ~~How memory works~~
- ~~Data structures~~
- Build your linked list!
- And a stack...

Singly linked list

- A linked list **starts** with **head-node**
- **Each element is separate object = node**
- A node consists out of the **data** and **reference to next Node**
- We need to be able to add nodes, delete nodes, search nodes, print list
- We need **2 classes**
  - Node & LinkedList



Class Node & LinkedList

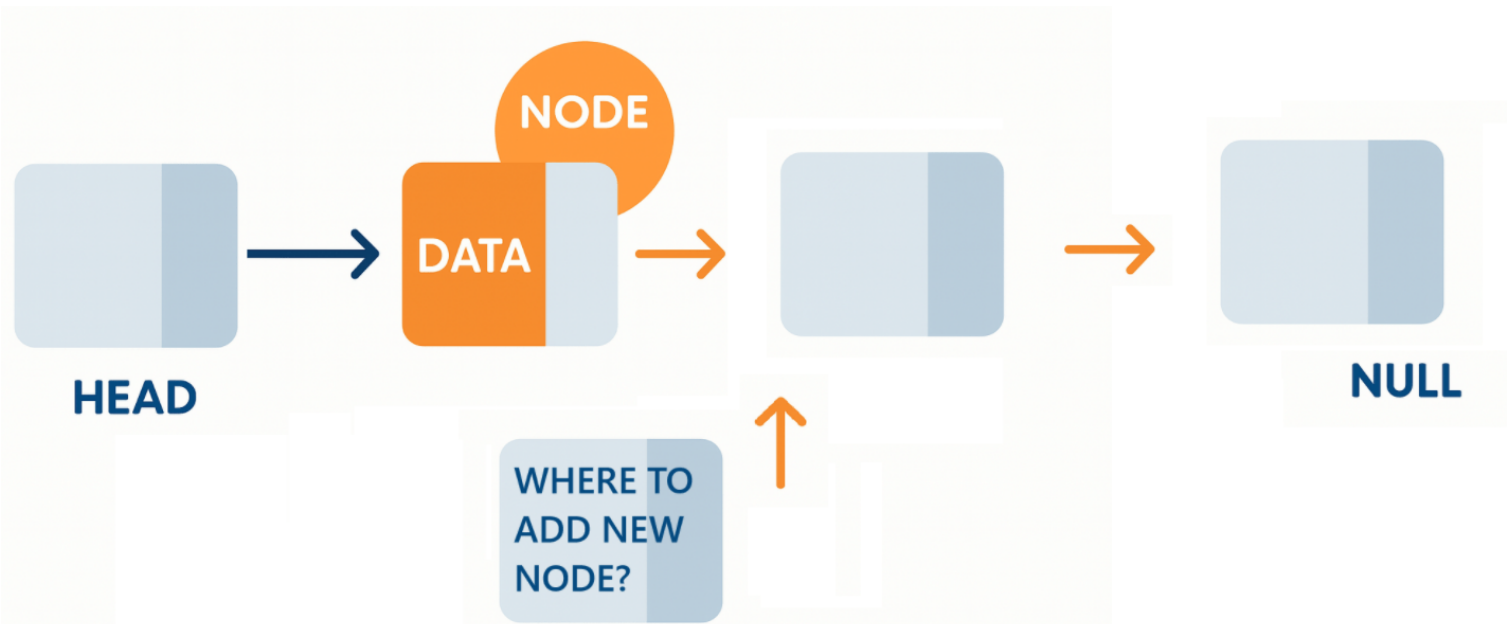


NODE
+ DATA
+ NEXT

LINKED LIST
+ HEAD (=NODE)
+ INSERTFRONT
+ INSERTLAST
+ DELETE
+ SEARCH
+ TOSTRING

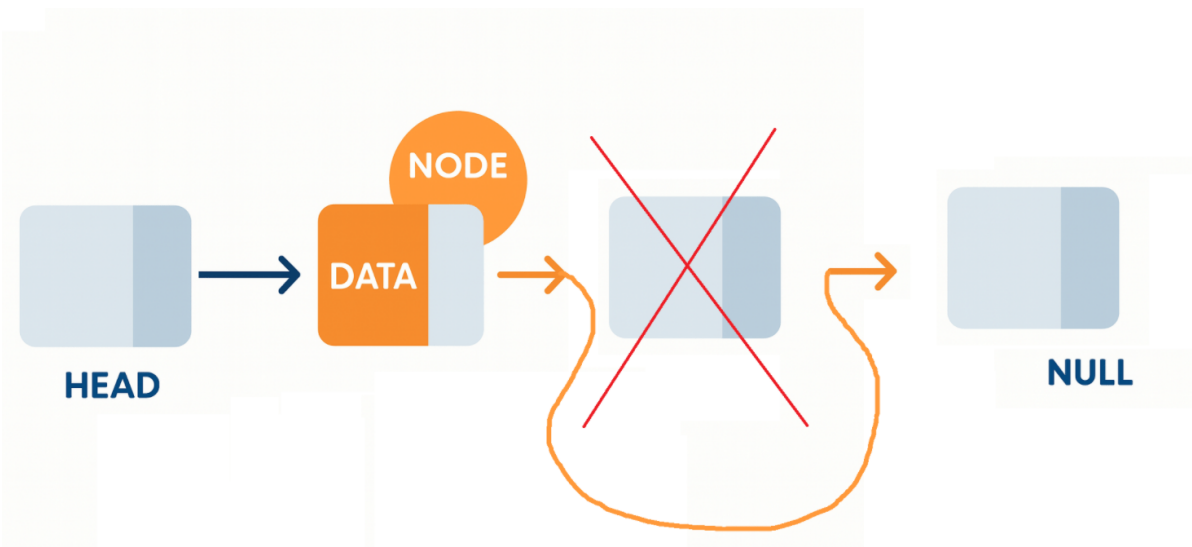
Insert node

- What to do with insert? Head or tail?
- Note: if list is still empty!



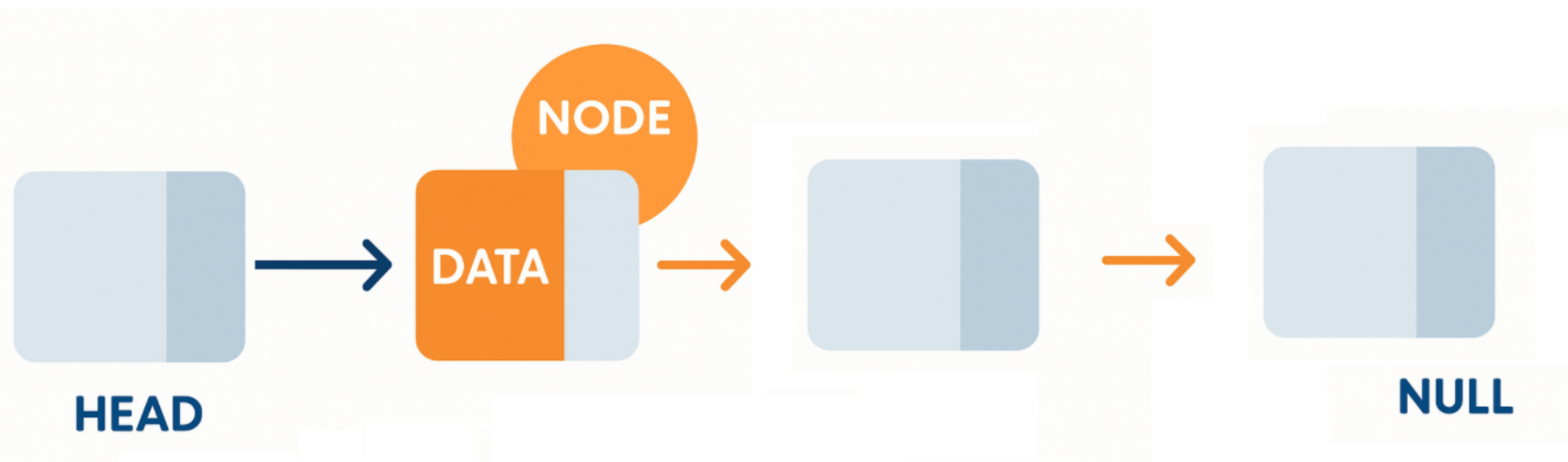
Delete node

- We look for an element to delete
  - what if element is head-element? Second element becomes head
  - what if element is tail-element? Next of previous element = null
  - what if element is in the middle? We bypass it!



Search node

- Search is a simple (sequential search)!
- Why not binary?



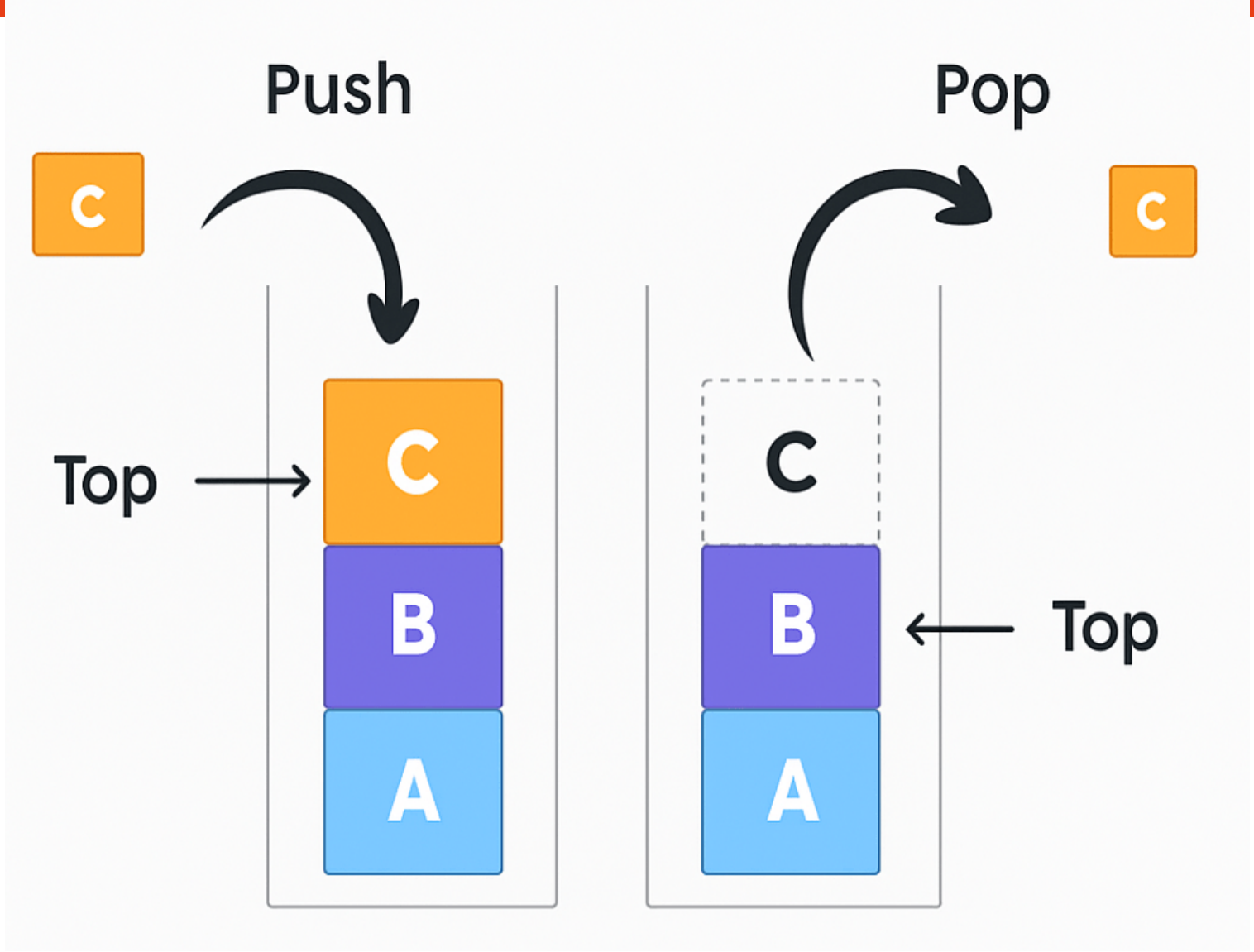
Let's try!

```
1  LinkedList list = new LinkedList();
2  list.InsertFront(5);
3  Console.WriteLine(list.ToString());
4  list.InsertFront(15);
5  Console.WriteLine(list.ToString());
6  list.InsertFront(55);
7  Console.WriteLine(list.ToString());
8  list.InsertLast(20);
9  Console.WriteLine(list.ToString());
10 list.Delete(15);
11 Console.WriteLine(list.ToString());
12
13 Console.WriteLine("Found 15? " + list.Search(15));
14 Console.WriteLine("Found 20? " + list.Search(20));
15
16 list.Delete(55);
17 Console.WriteLine(list.ToString());
18
19 list.Delete(20);
20 Console.WriteLine(list.ToString());
```

```
5 ->
15 -> 5 ->
55 -> 15 -> 5 ->
55 -> 15 -> 5 -> 20 -
>
55 -> 5 -> 20 ->
Found 15? False
Found 20? True
5 -> 20 ->
5 ->
```

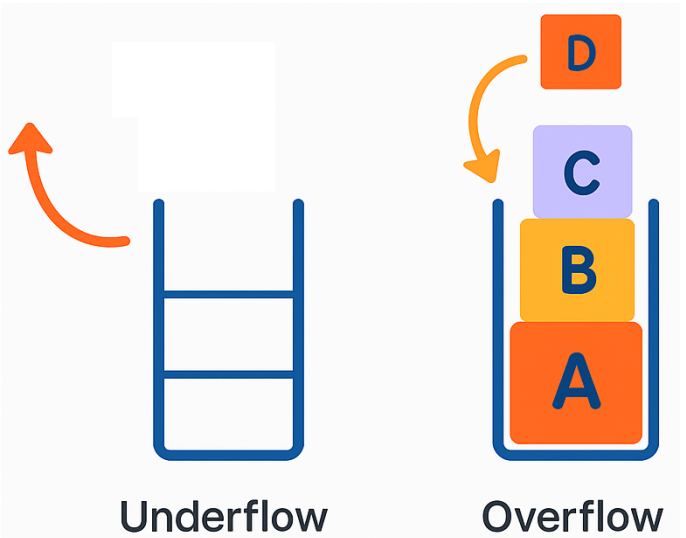
Data structures

- ~~How memory works~~
- ~~Data structures~~
- ~~Build your linked list!~~
- And a stack...


Push
Pop
C
C
Top
C
B
A
C
B
Top
A

Stack

- **LIFO** (last in, first out)
  - implement with array or list
- Two prinipal operations at top of stack
  - **push:** adds element to the collection
  - **pop**, removes last element that was added
- **Always remember the top!**
  - Stack **overflow:** push element when stack is full
  - Stack **underflow:** pop element when stack is empty



Let's try!

```
1  Stack stack = new Stack(5);
2  Console.WriteLine(stack.ToString());
3  stack.Push("this");
4  Console.WriteLine(stack.ToString());
5  stack.Push("is");
6  Console.WriteLine(stack.ToString());
7  stack.Push("a");
8  Console.WriteLine(stack.ToString());
9  stack.Push("stack");
10 Console.WriteLine(stack.ToString());
11 stack.Push("implementation");
12 Console.WriteLine(stack.ToString());
13 stack.Push("test");
14 Console.WriteLine(stack.ToString());
15 stack.Pop();
16 Console.WriteLine(stack.ToString());
17 stack.Pop();
18 Console.WriteLine(stack.ToString());
19 stack.Pop();
20 Console.WriteLine(stack.ToString());
21 stack.Pop();
22 Console.WriteLine(stack.ToString());
23 stack.Pop();
24 Console.WriteLine(stack.ToString());
25 stack.Pop();
```

```
Stack underflow
 | this
 | is | this
 | a | is | this
 | stack | a | is | this
 | implementation | stack | a | is |
this
Stack overflow
 | implementation | stack | a | is |
this
 | stack | a | is | this
 | a | is | this
 | is | this
 | this
Stack underflow
Stack underflow
```

- Linked List: chain of nodes where each node points to the next one (elements aren't stored in consecutive memory locations)
- Stack (LIFO - Last In, First Out)
- Queue (FIFO - First In, First Out)
- Choosing the right structure based on needs!

Recursion

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

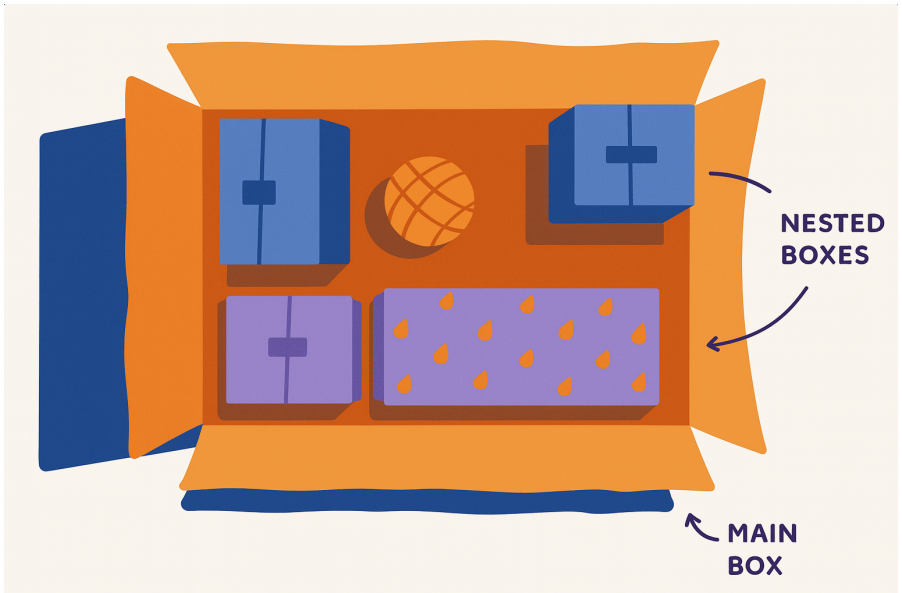
Recursion

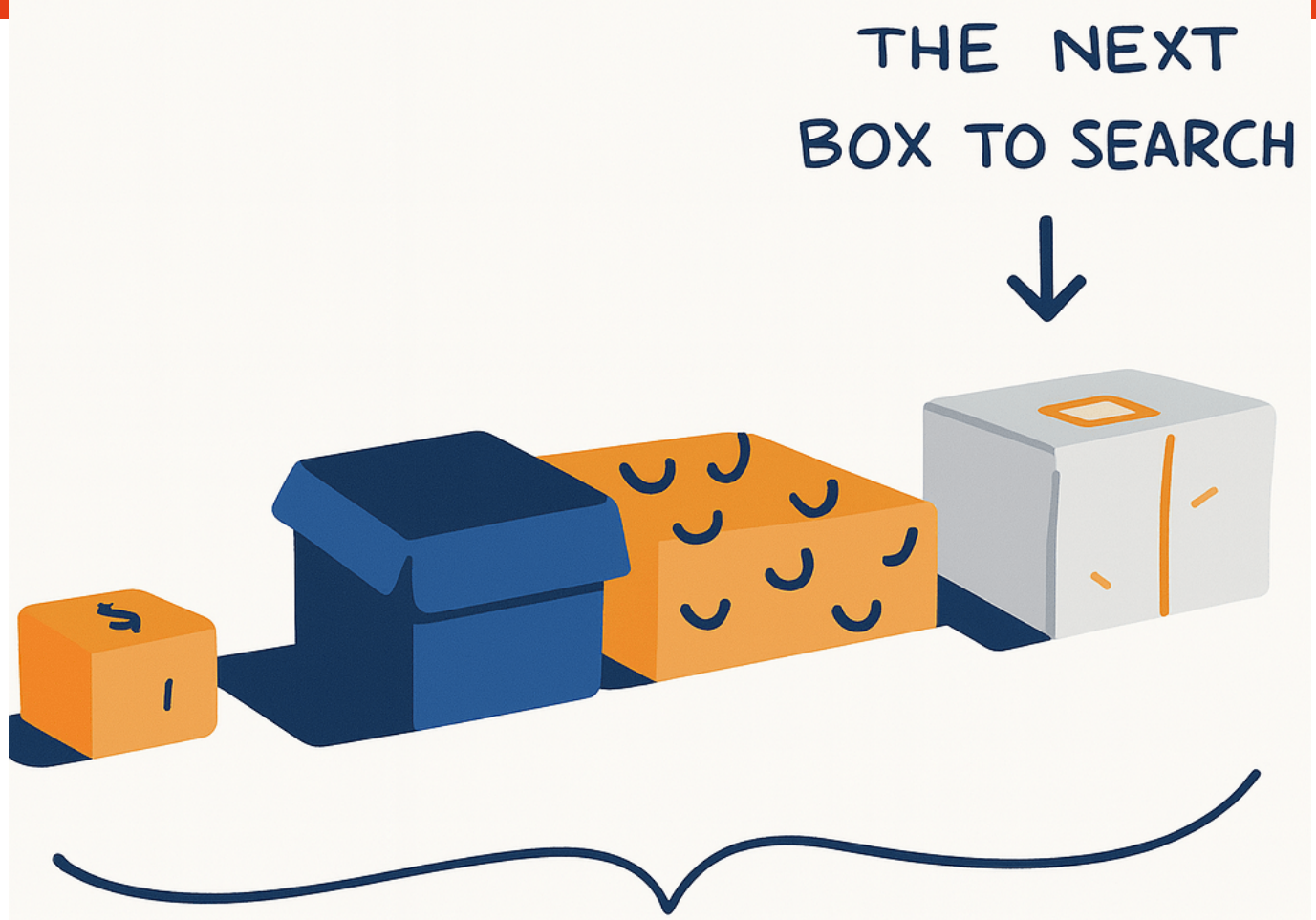
- Find the key!
- What is recursion?
- Factorial: a typical recursive algorithm
- Extra exercises

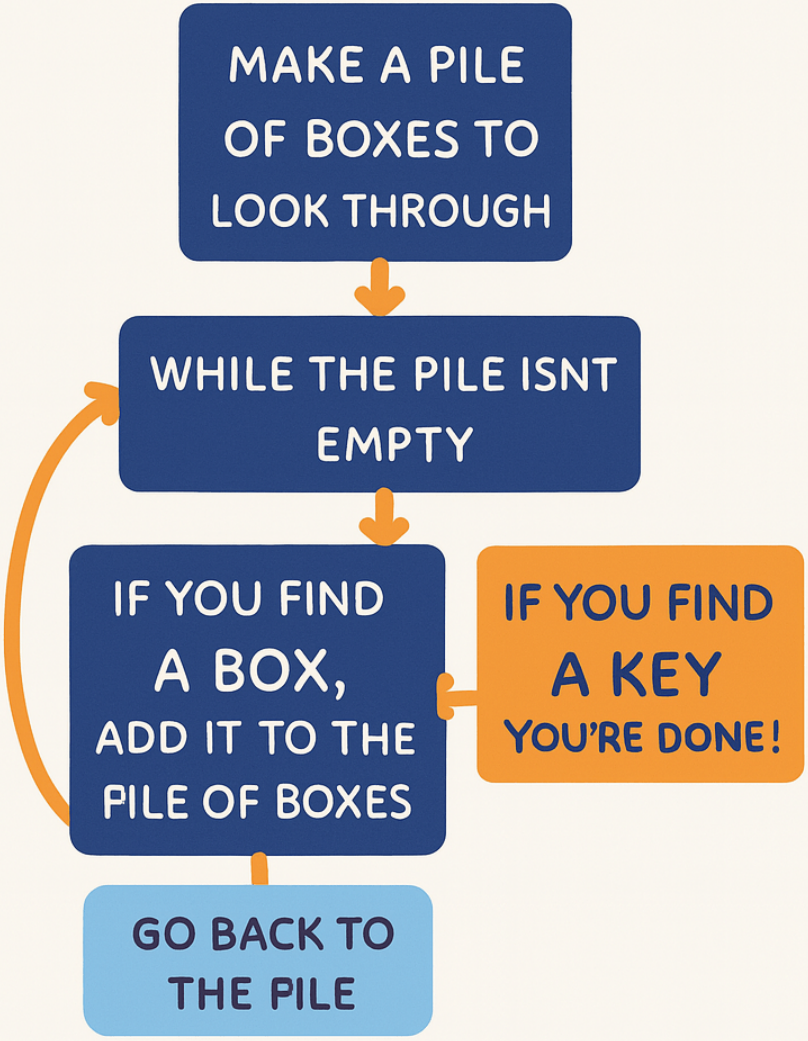
Looking for a key in a box

“ Suppose we are looking for a key in a box, but, the box contains more boxes, with more boxes inside those boxes and our hidden key is in a box somewhere.

What’s your algorithm to search?




THE NEXT
BOX TO SEARCH
Pile of boxes


MAKE A PILE OF BOXES TO LOOK THROUGH
WHILE THE PILE ISNT EMPTY
IF YOU FIND A BOX, ADD IT TO THE PILE OF BOXES
IF YOU FIND A KEY YOU'RE DONE!
GO BACK TO THE PILE

GO THROUGH EVERY ITEM IN THE BOX
IF YOU FIND A BOX...
IF YOU FIND A KEY, YOU ARE DONE!

Looking for a file in a folder

- We can easily transform this problem into a problem 'closer to home'
- Let's think of our box as a folder and our key as a file.
- Import the NuGet package **TM.ProgrammingAdvanced**
  - Use the Folders method to create some random folders (in a desired location), it will return the base folder (box) where you start searching.
  - In only one folder you will find key.txt Use PS to check!

```
string basefolder = Data.Folders(@"C:\");
```

```
Get-ChildItem -Recurse *.txt
```

Looking for a file in a folder

- Take a look at the structure of the randomized folder
  - *note: we all have different structures & names!*

```
PS C:\3hiy4okq5w0> tree /f
```

- Somewhere key.txt is hidden
- How are we going to find the correct file?
- Think of the previous algorithms!

```
C:\3hiy4okq5w0
|   0fvzhbxm2bo
|   0rnulzga14w
|   12bel4xmam1
|   └───5m3ru523nzr
|   1ukv51islsa
|   └───cy24pyqt3po
|       └───podfjjd1aut
|           └───1y2gcoyrvsv
|               └───x3hkpqclosj
|   4m2dlxrev5i
|   └───2yhlnm4xmaz
|       └───tf44usn1n1a
|           └───j1qlo0hrpkh
|   4z4jc1kn03c
|   abuuuwnhxid
|   └───pjxz1qwtk5i
|       └───boah42torz3
|   anxutdaoepv
|       key.txt
|
|   cfurkmpnha4
|   └───fkokcj3dxh3
|       └───4qzoozjcxa0
|           └───54lvqfmxwus
|               └───wsyl4khwooo
|   cxmudvc44sj
|   cxvkzko5yzg
```

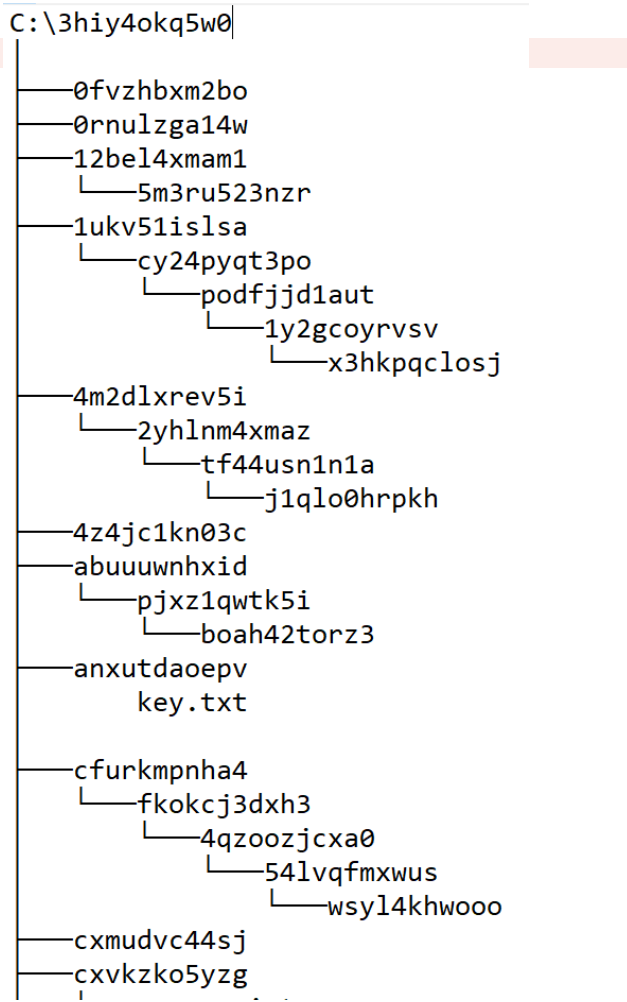
Algorithm 1

Boxes are folders, key is a file

1. Open the folder and make a list of folders inside
2. Take a folder and look trough it
3. If you find a file iside: you are done!
4. Otherwise add - the folders inside - to the list
5. Repeat as long as list isn't empty!

   or stop when file is found!

Method returns path to file!

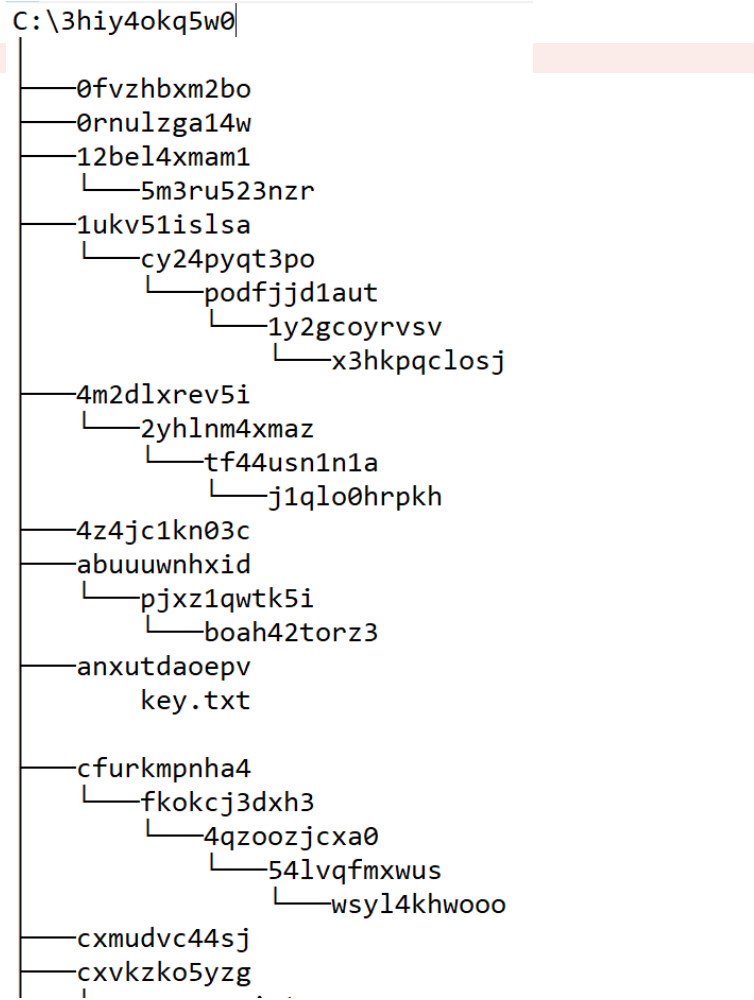


Algorithm 2

Boxes are folders, key is a file

1. Check every item in folder
2. if it is a box --> return to step 1
3. if it is a file: you are done!

Method returns path to file!



- *Which one is fastest?*
- *Test with counters and print out all paths!*
- *What do you notice? Is there a big difference between the two strategies?*

Let's compare

- How many folders does each algorithm go through.
- What do you see happening?

Algorithm1	Algorithm2
C:\3hiy4okq5w0\0fvzhbxm2bo C:\3hiy4okq5w0\0rnulzga14w C:\3hiy4okq5w0\12bel4xmam1 C:\3hiy4okq5w0\1ukv51islsa C:\3hiy4okq5w0\4m2dlxrev5i C:\3hiy4okq5w0\4z4jc1kn03c C:\3hiy4okq5w0\abuuuwnhxid C:\3hiy4okq5w0\anxutdaoepv Found at C:\3hiy4okq5w0\anxutdaoepv 8 folders	C:\3hiy4okq5w0\0fvzhbxm2bo C:\3hiy4okq5w0\0rnulzga14w C:\3hiy4okq5w0\12bel4xmam1 C:\3hiy4okq5w0\12bel4xmam1\5m3ru523nzr C:\3hiy4okq5w0\1ukv51islsa C:\3hiy4okq5w0\1ukv51islsa\cy24pyqt3po C:\3hiy4okq5w0\1ukv51islsa\cy24pyqt3po\podfjjd1aut C:\3hiy4okq5w0\1ukv51islsa\cy24pyqt3po\podfjjd1aut\1y2gcoyrvsv C:\3hiy4okq5w0\1ukv51islsa\cy24pyqt3po\podfjjd1aut\1y2gcoyrvsv\x3hkpqclosj C:\3hiy4okq5w0\4m2dlxrev5i C:\3hiy4okq5w0\4m2dlxrev5i\2yhlnm4xmaz C:\3hiy4okq5w0\4m2dlxrev5i\2yhlnm4xmaz\tf44usn1n1a C:\3hiy4okq5w0\4m2dlxrev5i\2yhlnm4xmaz\tf44usn1n1a\j1qlo0hrpkh C:\3hiy4okq5w0\4z4jc1kn03c C:\3hiy4okq5w0\abuuuwnhxid C:\3hiy4okq5w0\abuuuwnhxid\pjxz1qwtk5i C:\3hiy4okq5w0\abuuuwnhxid\pjxz1qwtk5i\boah42torz3 C:\3hiy4okq5w0\anxutdaoepv C:\3hiy4okq5w0\anxutdaoepv\key.txt Found at C:\3hiy4okq5w0\anxutdaoepv\key.txt 18 folders

```
C:\3hiy4okq5w0
├──0fvzhbxm2bo
├──0rnulzga14w
├──12bel4xmam1
│   └──5m3ru523nzr
├──1ukv51islsa
│   └──cy24pyqt3po
│       └──podfjjd1aut
│           └──1y2gcoyrvsv
│               └──x3hkpqclosj
├──4m2dlxrev5i
│   └──2yhlnm4xmaz
│       └──tf44usn1n1a
│           └──j1qlo0hrpkh
├──4z4jc1kn03c
├──abuuuwnhxid
│   └──pjxz1qwtk5i
│       └──boah42torz3
├──anxutdaoepv
│       key.txt
├──cfurkmpnha4
│   └──fkokcj3dxh3
│       └──4qzoozjcxa0
│           └──54lvqfmxwus
│               └──wsyl4khwooo
├──cxmudvc44sj
├──cxvkzko5yzg
```

Let's compare

Algorithm1

77 folders

```
C:\5ylzzwumytl\0ixsn2f4ct2
C:\5ylzzwumytl\0wg2bych0z2
C:\5ylzzwumytl\1cb0t3ppib0
C:\5ylzzwumytl\2ansukvj3gf
C:\5ylzzwumytl\2f1q0jemltq
...
C:\5ylzzwumytl\yp5eeb3lfbs\amophd2fasg\hmdeufy2ebn\22dbqo5jhwr
C:\5ylzzwumytl\0ixsn2f4ct2\mknijbjfswh\gihydoln35t\4zogk25ygmo\epazcdiys2d
C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj\uypkcew2q2z\5h2gkki4kst
Found at C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj\uypkcew2q2z\5h2gkki4kst
```

Algorithm2

12 folders

```
C:\5ylzzwumytl\0ixsn2f4ct2
C:\5ylzzwumytl\0ixsn2f4ct2\mknijbjfswh
C:\5ylzzwumytl\0ixsn2f4ct2\mknijbjfswh\gihydoln35t
C:\5ylzzwumytl\0ixsn2f4ct2\mknijbjfswh\gihydoln35t\4zogk25ygmo
C:\5ylzzwumytl\0ixsn2f4ct2\mknijbjfswh\gihydoln35t\4zogk25ygmo\epazcdiys2d
C:\5ylzzwumytl\0wg2bych0z2
C:\5ylzzwumytl\1cb0t3ppib0
C:\5ylzzwumytl\2ansukvj3gf
C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l
C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj
C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj\uypkcew2q2z
C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj\uypkcew2q2z\5h2gkki4kst
C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj\uypkcew2q2z\5h2gkki4kst\key.txt
Found at C:\5ylzzwumytl\2ansukvj3gf\edwulyk3l4l\rx33ilfcmkj\uypkcew2q2z\5h2gkki4kst\key.txt
```

```
C:\5ylzzwumytl
├──0ixsn2f4ct2
│  └──mknijbjfswh
│     └──gihydoln35t
│        └──4zogk25ygmo
│           └──epazcdiys2d
├──0wg2bych0z2
├──1cb0t3ppib0
├──2ansukvj3gf
│  └──edwulyk3l4l
│     └──rx33ilfcmkj
│        └──uypkcew2q2z
│           └──5h2gkki4kst
│                 key.txt
├──2f1q0jemltq
│  └──ymp4jp14g3z
│     └──dsxcn0lc4xp
│        └──teuds01v3qd
│           └──maaonhwihvu
├──2p5zbgh5o1l
├──3dmdd4epvop
│  └──5jhyfwosbl3
│     └──ym0xmvrrvy4
├──3g5dnb5nw0q
```

So what about the runtime?

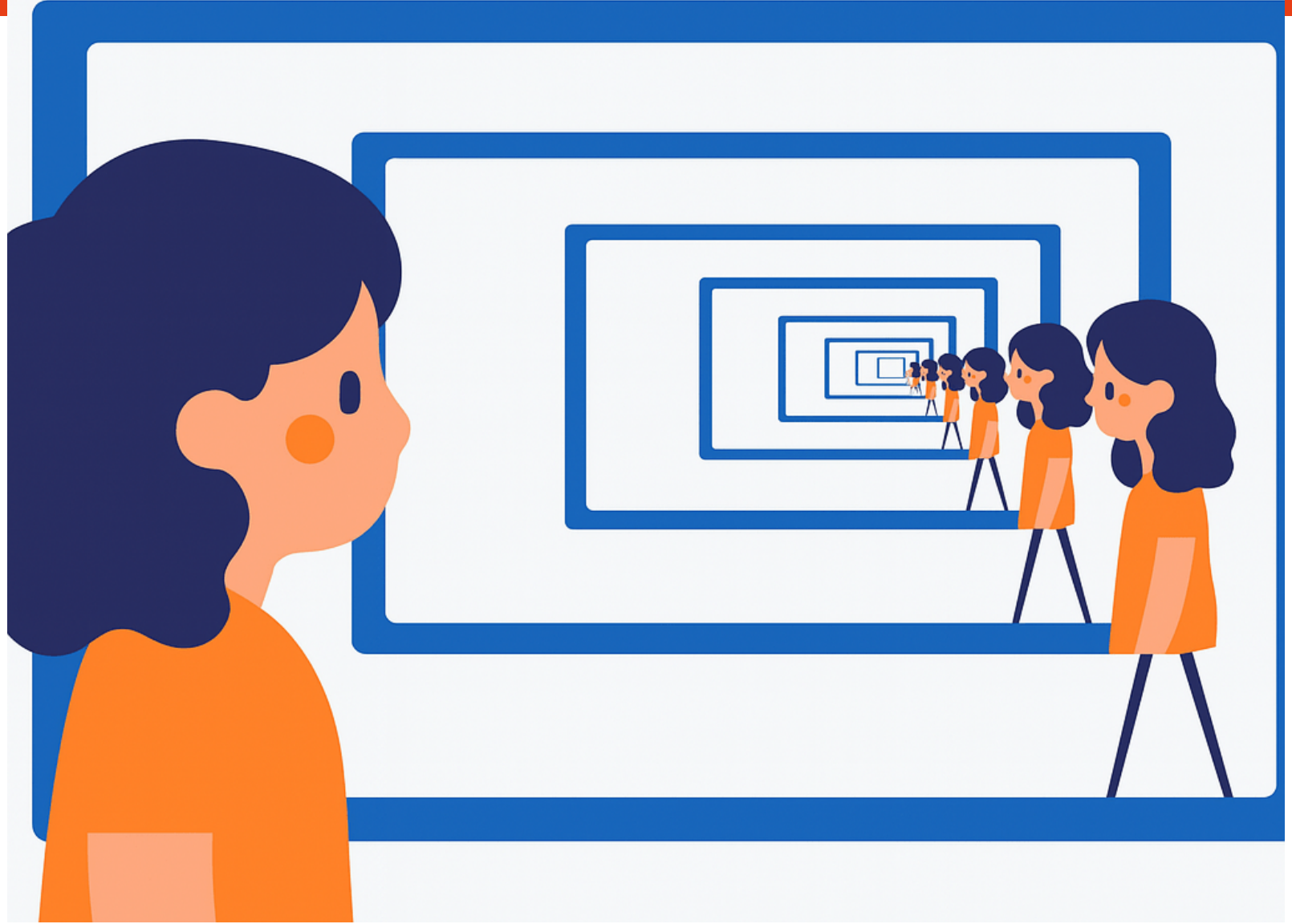
- It all **depends on the folder structure**! When a file is deep within the structure, algorithm 2 is better, otherwise algorithm 1!
- **Both** solutions are **correct** and **interchangeable**, it's a matter of taste ;)
  - Which one do you think is more clear
- The first algorithm is an **iterative function** (it repeats with a **loop**), the second algorithm is a **recursive function** (it **calls itself** to repeat)

```
1 while (list.Count > 0)
2 {
3   //code
4 }
```

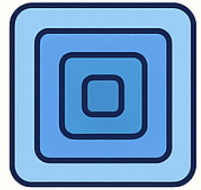
```
1 string result = Algorithm2(element);
2 if (result != null) return result;
```

Recursion

- ~~Find the key!~~
- What is recursion?
- Factorial: a typical recursive algorithm
- Extra exercises



What is recursion?

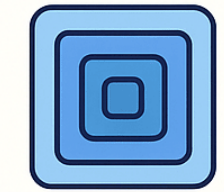


- **Recursion** is a **coding technique**
  - *a way of writing code*
  - *a programming technique*
- It's solving a problem by **solving smaller instances** of the **same problem**
- A recursive function **calls itself**
- **A recursive function consists of**
  - **Base case** (or cases) = endpoint that does not use recursion
  - A **recursive case** (general case) = reduces all other cases towards the base case
- Either you love recursion or you hate it ;)

Why do we need recursion?

- Some **problems** have a **recursive structure**
  - Tree operations, file system navigation, mathematical definitions...
- Some **key algorithmic techniques depend** on **recursion**
  - Backtracking: solving mazes, Sudoku puzzles, N-Queens problem...
  - Divide & Conquer: merge sort, quicksort...
- Recursion provides natural elegance because code mirrors the problem structure
- **Alternative**: You could force loops instead, but...
  - much more complex code
  - harder to understand and debug
  - loses the intuitive problem-solving approach

Recursion in real life?



- **Russian nesting dolls**: Open one doll, find a smaller doll inside, repeat until you find the tiny solid one
- **Family trees**: To understand your ancestry,
  look at your parents' ancestry, then their parents', etc.
- **Fractals in nature**: Tree branches split into smaller branches,
  which split into even smaller branches
- **Learning to read**: Learn letters → combine into words → combine into sentences → combine into paragraphs
- ....

Let's take a step back!

- Recursion is solving a problem by **solving smaller instances** of **same problem**
  - *we use the same technique for each folder, we just go deeper and deeper into the folder until nothing is underneath*
- A recursive function solves the problem **by calling itself**

```
 1 string Algorithm2(string path)
 2 {
 3   foreach (string element in Directory.GetFileSystemEntries(path))
 4   {
 5     if (Directory.Exists(element))
 6     {
 7       string result = Algorithm2(element);
 8       if (result != null) return result;
 9     }
10     else return element;
11   }
12   return null;
13 }
```

Base & recursive case

recursive case *base case*

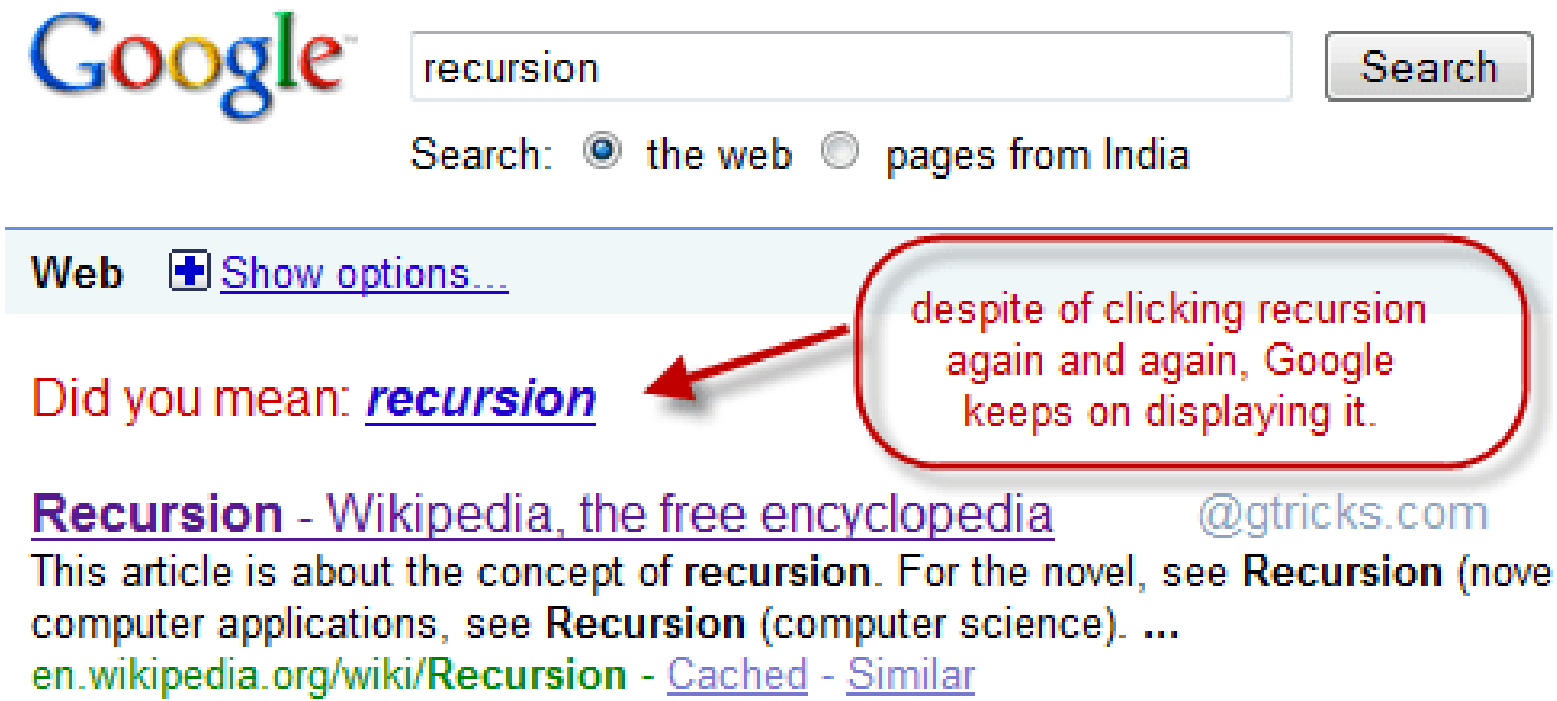
- How does the function know?

- **Where te stop by calling itself?**
  = base case
  - if the folder is empty (null) of file is found
- **Where to call itself?**
  = recursive case
  - if the folder is not empty

Base & recursive case

Without one or more base case(s) = ***infinite loop***

No base case = infinite loop!



Wanna see more easter eggs? https://cognitiveseo.com/blog/13616/google-easter-eggs/

Recursion

- ~~Find the key!~~
- ~~What is recursion?~~
- Factorial: a typical recursive algorithm
- Extra exercises

Factorial with iteration

- What is factorial exactly?

$$n! = n * (n-1) * (n-2) * (n-3) * ... * 1$$

- Let's first solve it in a good old fashion way with loops

```
1   static int Factorial(int number)
2   {
3     int fact = 1;
4     for (int i = 1; i <= number; i++)
5     {
6       fact = fact * i;
7     }
8     return fact;
9   }
10
11  //in main
12  Console.WriteLine("Factorial of 5: " + Factorial(5));
```

n	n!
0	1
1	1
2	2
3	6 = 3 * 2 * 1 * 1
4	24 = 4 * 3 * 2 * 1 * 1
5	120
6	720
7	5040
8	40320
...	...

Factorial with iteration

- Print result in every loop
  - fact gets larger after every iteration!

```
1: - fact = 1
2: - fact = 2
3: - fact = 6
4: - fact = 24
5: - fact = 120
Factorial of 5 :120
```

```
1   static int Factorial(int number)
2   {
3     int fact = 1;
4     for (int i = 1; i <= number; i++)
5     {
6       fact = fact * i;
7       Console.WriteLine(i + ": - fact = " +fact);
8
9     }
10    return fact;
11  }
12
13  //in main
14  Console.WriteLine("Factorial of 5: " + Factorial(5));
```

Factorial with recursion

- Now let's fix this with recursion
  - It's solving a problem by solving smaller instances of the same problem
  - **recursive** case **= return with call to smaller instance**
    smaller instance = number-1
  - **base** case **= 1** (return 1)
- So beautiful, isn't it?

```
1 int Factorial(int number)
2 {
3   if (number > 1) //recursive case
4     return (number * Factorial(number - 1));
5   else return 1; //base case
6 }
```

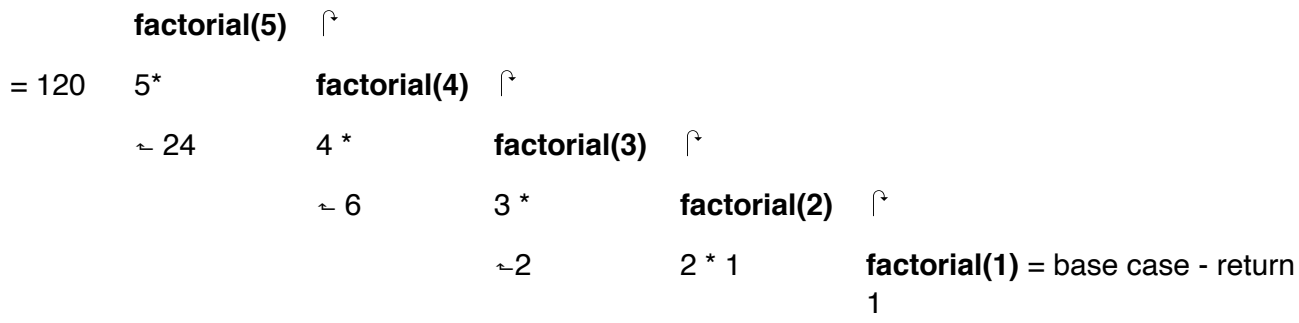
n	**n!**
0	1
1	**1 BASE CASE**
2	2
3	6 = 3 * 2 * 1 * 1
4	24 = 4 * 3 * 2 * 1 * 1
5	120
6	720
7	5040
8	40320
...	...

Factorial with recursion

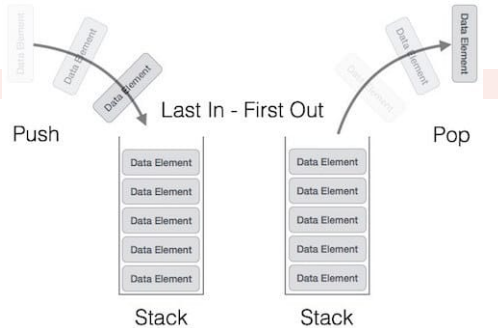
- Now print every call

```
1  int Factorial(int number)
2  {
3    Console.WriteLine("call with " + number);
4    if (number > 1)
5      return (number * Factorial(number - 1));
6    else return 1;
7  }
```

```
call with 5
call with 4
call with 3
call with 2
call with 1
Factorial of 5 :120
```



The call stack



- Every call to factorial is placed internally on the **call stack**
- Stack is datastructure (remember?) (LIFO)
  - every time we call ourself : push on stack
  - when we encounter base case : pop of stack
- What will happen if there are no base cases?

push	**push**	**push**	**push**	**push**	**pop**	**pop**	**pop**	**pop**	**pop**
				↱ factorial(1)	*BASE CASE* *factorial(1)* *↽ return 1*				
			↱ factorial(2)	factorial(2)	factorial(2)	factorial(2) ↽ return 2			
		↱ factorial(3)	factorial(3)	factorial(3)	factorial(3)	factorial(3)	factorial(3) ↽ return 6		
	↱ factorial(4)	factorial(4)	factorial(4)	factorial(4)	factorial(4)	factorial(4)	factorial(4)	factorial(4) ↽ return 24	
↱ factorial(5)	factorial(5)	factorial(5)	factorial(5)	factorial(5)	factorial(5)	factorial(5)	factorial(5)	factorial(5)	factorial(5) r↽return 120

Stack overflow!

- Remove the base cases

```
1  int Factorial(int number)
2  {
3    //if (number > 1)
4     Console.Write(number + " ");
5     return (number * Factorial(number - 1));
6    //else return 1;
7  }
```

```
...

call with -13646
call with -13647
call with -13648
call with -13649
call with -13650
call with -13651
call with -13652
call with -13653
call with -13654
call with -13655
Stack overflow.
```

- What happens?
  - An exception is thrown
  - **A stack overflow** occurs when **exceeding stack boundaries.**
  - **The size of the call stack** depends on programming language, machine architecture, available memory, etc...

Recursion

- ~~Find the key!~~
- ~~What is recursion?~~
- ~~Factorial: a typical recursive algorithm~~
- Extra exercises

1. Sum of Natural Numbers: compute the sum of the first n natural numbers recursively.
2. Count Digits: count the number of digits in a number using a recursive approach.
3. Reverse String: recursively reverse the characters in a string.

Extra coding exercises

Sum of natural numbers

Iterative

```
1 public static int SumNaturalIterative(int n)
2 {
3     int sum = 0;
4     for (int i = 1; i <= n; i++)
5     {
6         sum += i;
7     }
8     return sum;
9 }
```

Recursive

```
1  public static int SumNaturalRecursive(int n)
2  {
3      if (n == 1)
4      {
5          return 1;
6      }
7      else
8      {
9          return n + SumNaturalRecursive(n - 1);
10     }
11 }
```

Count Digits

Iterative

```
1  public static int CountDigitsIterative(int n)
2  {
3      int count = 0;
4      while (n > 0)
5      {
6          count++;
7          n = n / 10;
8      }
9      return count;
10 }
```

Recursive

```
1  public static int CountDigitsRecursive(int n)
2  {
3      if (n == 0)
4      {
5          return 0;
6      }
7      else
8      {
9          return 1 + CountDigitsRecursive(n / 10);
10     }
11 }
```

Reverse String

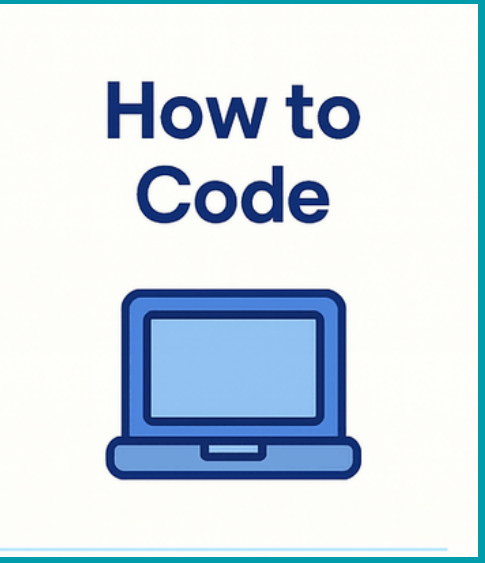
```
1  public static string ReverseString(string s)
2  {
3      char[] arr = s.ToCharArray();
4      int start = 0, end = s.Length - 1;
5      while (start < end)
6      {
7          char temp = arr[start];
8          arr[start] = arr[end];
9          arr[end] = temp;
10         start++;
11         end--;
12     }
13     return new string(arr);
14 }
```

Iterative

```
1  public static string ReverseStringR(string s)
2  {
3      if (s.Length <= 1)
4          return s;
5      else
6          return s[s.Length - 1] + ReverseStringR(s.Substring(0, s.Length - 1));
7  }
```

Recursive

- *Recursion is when a function calls itself.*
- *Every recursive function has two cases:*
  - *Base case*
  - *Recursive case*
- *All function calls go onto the call stack.*
- *The call stack can get very large, which takes up a lot of memory.*



- **Purpose**: Implementation techniques and coding patterns
- **Why it matters**: Having the right coding tools lets you turn your problem-solving ideas into working programs efficiently.
- **Real-world impact**: Writing clean, maintainable code that other developers can understand and extend.

Divide & Conquer

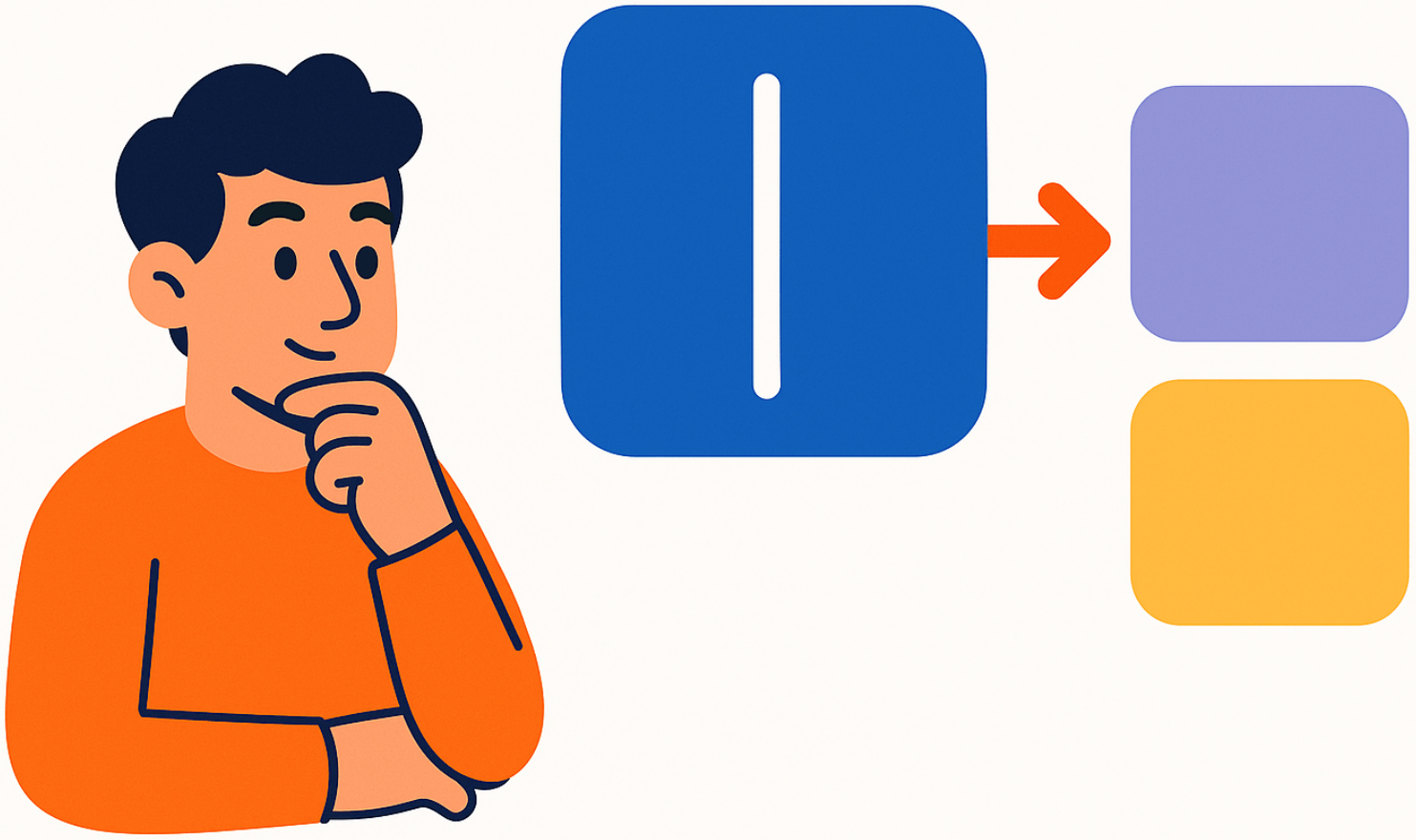
Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Divide & Conquer

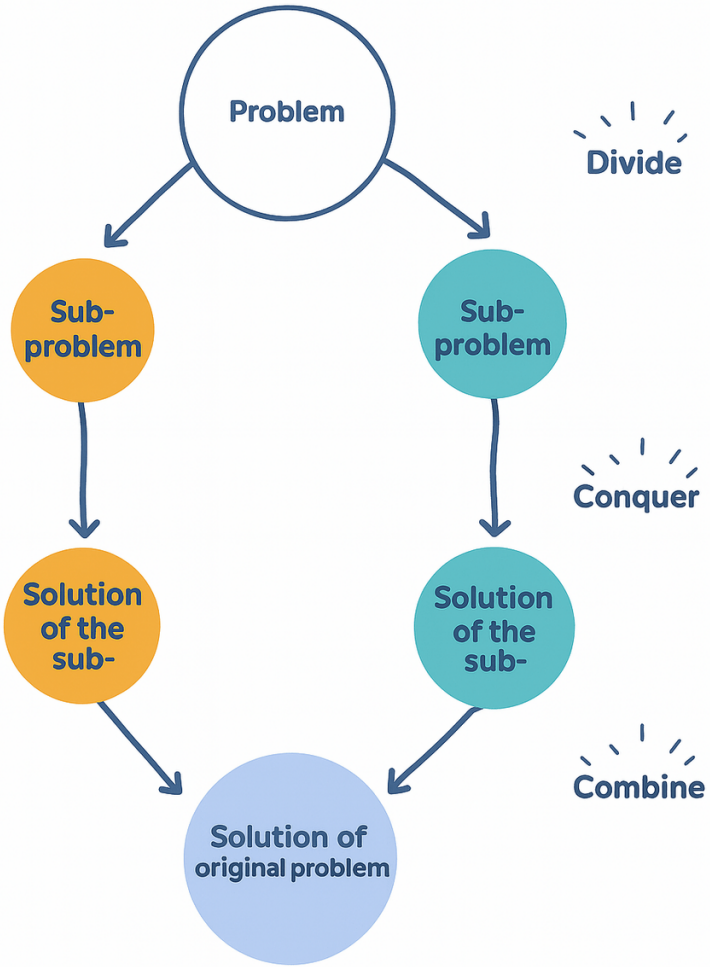
- What is divide & conquer?
- QuickSort
- Towers of Hanoi
- Extra exercises



What is divide & conquer?

- **Divide and Conquer: break big problems into smaller, similar subproblems**

- **Divide**: Split problem into smaller parts
- **Conquer**: Solve each part separately (often recursively)
- **Combine**: Merge solutions back together



Why do we need divide & conquer?

- **Big problems are overwhelming** - smaller pieces are manageable
- **Efficiency** - often much faster than tackling the whole problem at once ($O(n \log n)$ vs $O(n^2)$)
- **Parallelization** - subproblems can be solved simultaneously on different processors

Divide & conquer in real life?

- **Organizing a large event** - divide into teams (catering, music, decorations), each works independently
- **Software development** - break large project into modules, teams work on each module
- **Phone book search** - open to middle, decide which half, repeat (binary search)
- ...

Why is recursion essential?

- **Why use recursion for divide and conquer?**
  - **Natural fit** - each subproblem uses the same algorithm on smaller data
  - **Automatic handling** - recursion manages the "divide" and "combine" steps elegantly
  - **Clean code** - recursive solution mirrors the problem structure perfectly
  - **Multiple recursive calls** - unlike regular recursion (1 call), divide & conquer makes multiple calls (left half, right half)
- **Difference from regular recursion:**
  - **Regular recursion:** one path, gets smaller
  - **Divide & Conquer:** merge multiple paths

Divide & Conquer

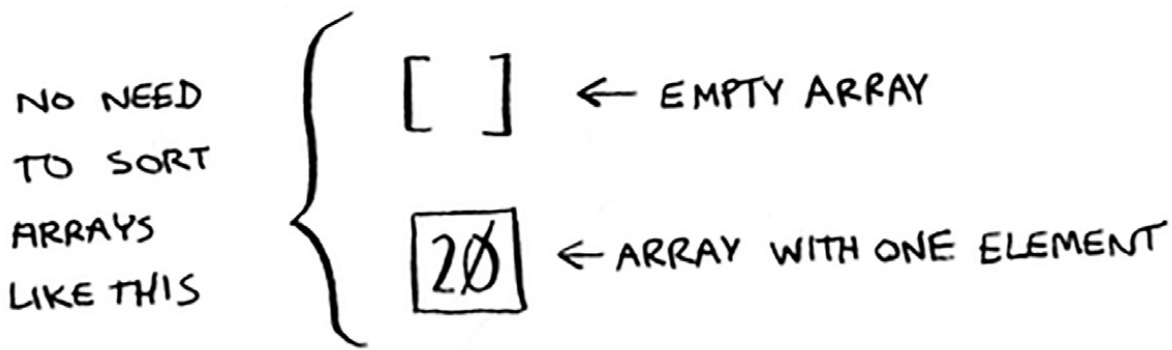
- ~~What is divide & conquer?~~
- QuickSort
- Towers of Hanoi
- Extra exercises

Quicksort

- **Sorting** algorithm, much faster than selection sort
- Frequently used in **real life**
  - The C standard library has a function called qsort (implementation of quicksort)
- Quicksort also uses D&C!
- **D&C strategy**:
  - Figure out a simple case as the base case.
  - Figure out how to reduce your problem and get to the base case.

Quicksort: the base case

- Remember what to do with arrays?

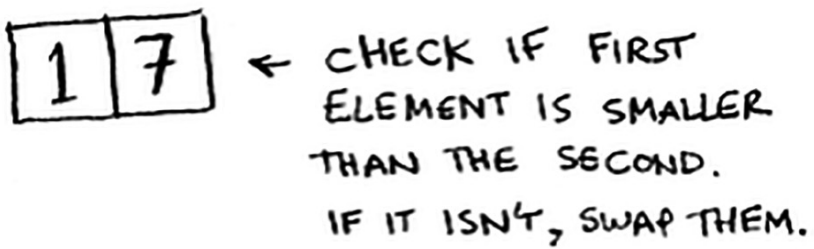


- Yes! Empty arrays and arrays with just one element will be the base case.

  You can just return those arrays because there's nothing to sort!

Quicksort: the recursive case

- Take a look at bigger arrays!
- Arrays with two elements: also easy!



- Take a look at an array of 3 elements!
  - how to break down array towards basecase?
  - use a number as a pivot!

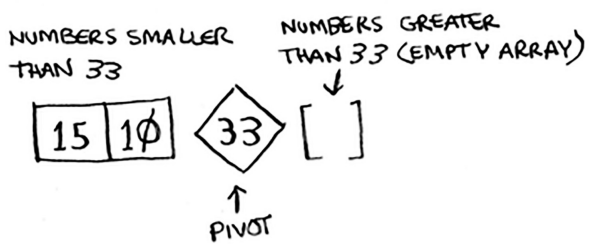
Quicksort: the algorithm

- Pick an element from the array = **the pivot**

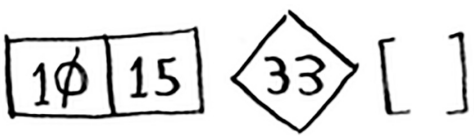
  - you can even choose randomly, it will work!
- Find elements smaller and greater than the pivot = **partitioning**!

  we end up with:

  - A sub-array of all the numbers less than the pivot
  - the pivot
  - A sub-array of all the numbers greater than the pivot



- Sort the subarrays with quicksort and merge everything together!



Quicksort: the algorithm

- Blue = pivot
- After partitioning you end up with two sub-arrays: sort them as well!

Quicksort: 'the code'

```
1 //main
2 List<int> list = new List<int> { 12, 56, 34, 78, 23, 89, 45, 67, 90, 21 };
3
4 QuickSort qs = new QuickSort();
5 list = qs.DCLists(list);
6 qs.Print(list);
```

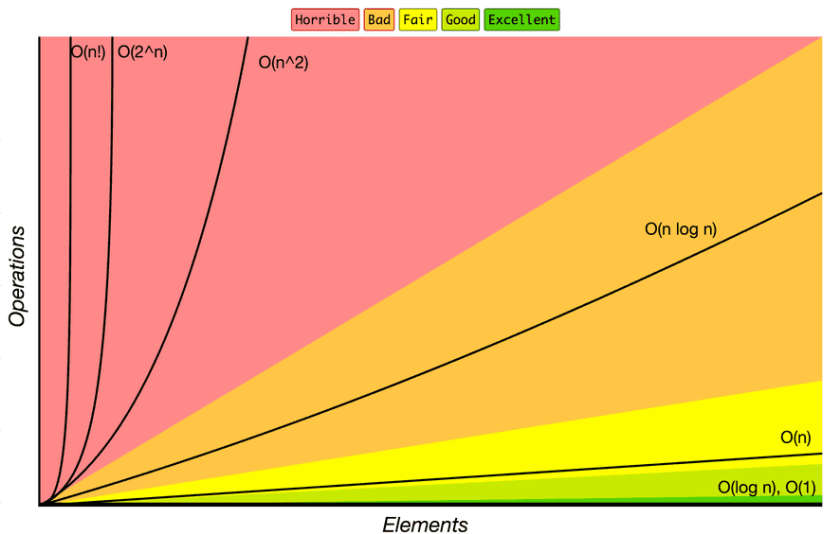
- The `DCLists` function receives a list.
- If the list has 1 or fewer elements, it returns the list.
- The first element is the pivot.
- The list is split into:
  - **less** (elements smaller than the pivot)
  - **greater** (elements larger than the pivot)
- Recursively sorts both parts and combines them with the pivot.

What about Big O?

- Count the steps... 1999 operations needed
- And remember the other operations needed?
  - Bubble: 19900
  - Selection: 19900
  - Insertion: 11037
- It depends on the sequence and pivot how well it performs, but in general it's

$$O(n * log\ n)$$

O	Complexity	
O(1)	constant	
O(log n)	logarithmic	
O(n)	linear time	
O(n * log n)	log linear	
O(n^2)	quadratic	
O(n^3)	cubic	
O(2^n)	exponential	
O(n!)	factorial	slow



Divide & Conquer

- ~~What is divide & conquer?~~
- ~~QuickSort~~
- Towers of Hanoi
- Extra exercises

Tower of Hanoi: the problem

- Tower of Hanoi consists of three towers with n disks placed
- The goal of the puzzle is to move the stack to another tower following these rules:
  - Only one disk can be moved at a time.
  - No disk can be placed on top of the smaller disk.

Tower of hanoi: the strategy

- Find base case
  - what is the simplest case?
  - only one disk seems very easy?
- Find recursive case
  - You need to move closer to 1 disk with every recursive call.
  - How do you reduce your problem size?

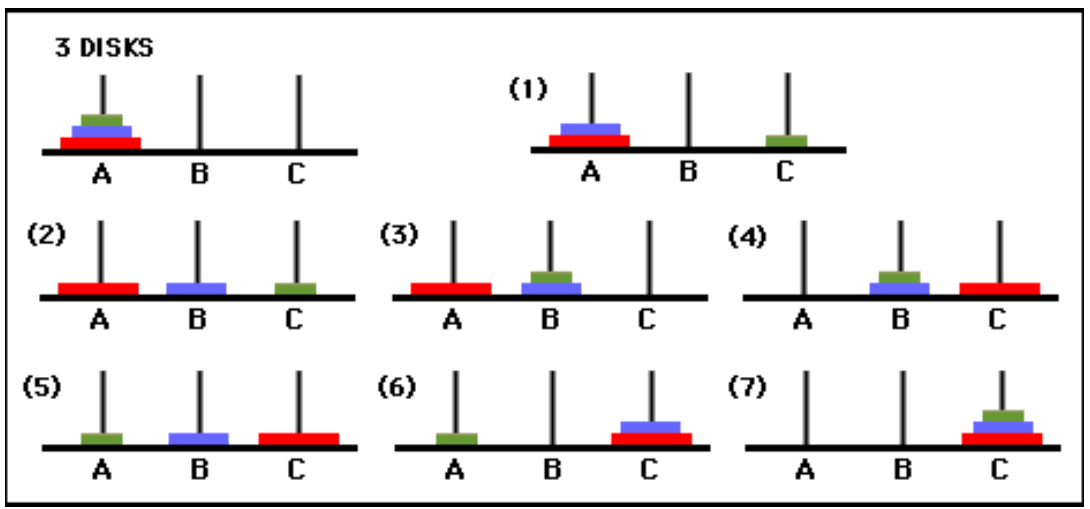
> *Start writing & drawing!!*

draw movements of 1 disk, 2 disks...

Tower of Hanoi: the strategy

- How to solve this? 😨
  - Write down all the movements
- What do you notice?

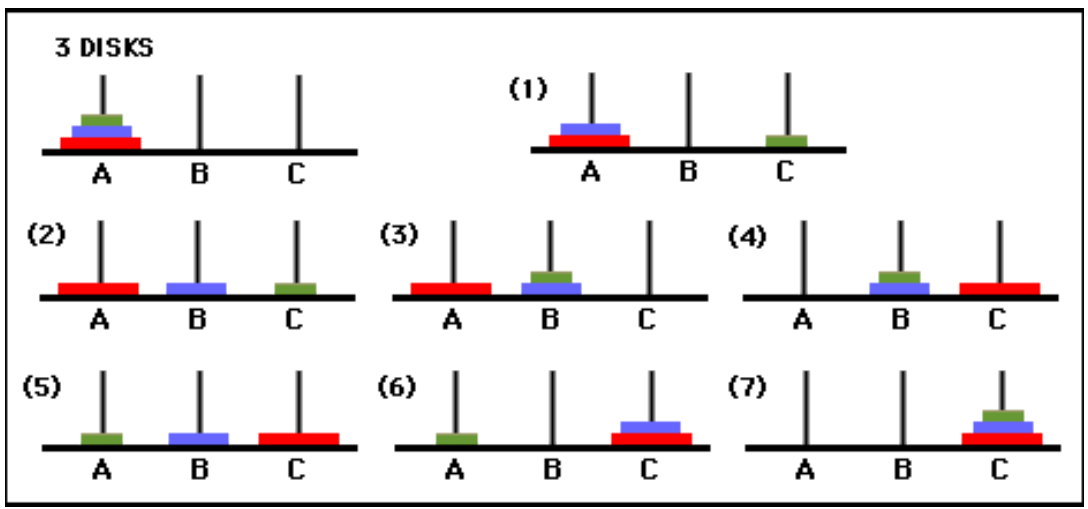
1 disc	2 discs	3 discs
1: A to C	1: A to B 2: A to C 1: B to C	1: A to C 2: A to B 1: C to B 3: A to C 1: B to A 2: B to C 1: A to C



Tower of Hanoi: the strategy

- Really that simple? Yes!
- Sometimes a problem can be overwhelming but thanks to recursion we can solve it by **solving smaller instances of the same problem**!

1 disc	**2 discs**	**3 discs**
1: A to C	*1: A to B* **2: A to C** *1: B to C*	1: A *to* C *2: A to B* 1: C *to* B **3: A** *to* **C** 1: B *to* A *2: B to C* 1: A *to* C
BASE	RECURSIVE	



Tower of Hanoi: the code!

- So pretty: clear & simple!
- How to check if it is right? Just check by doing yourself!

```
1 Hanoi hanoi = new Hanoi();
2 hanoi.DivideAndConquer(3, 'A', 'C', 'B');
```

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

BASE	**RECURSIVE**
disk 1 from A to C	*disk 1 from A to B* **disk 2 *from* A** *to* **C** *disk 1 from B to C*

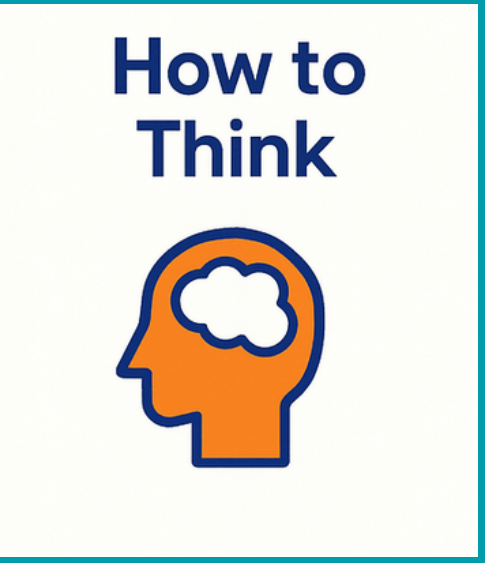
Divide & Conquer

- ~~What is divide & conquer?~~
- ~~QuickSort~~
- ~~Towers of Hanoi~~
- Extra exercises

1. **Merge sort** - Split array in half, sort each half, merge back together *(Quicksort does work before recursion (partitioning), Merge sort does work after recursion (merging))*
2. **Array Multiplication** - Split array, multiply each half, multiply results
3. **Binary search** - Split sorted array in half, search relevant half

Extra coding exercises

- *Divide and Conquer splits big problems into smaller, similar subproblems*
- *Divide → Solve each part → Combine results back together*
- *Uses recursion with multiple recursive calls (unlike regular recursion with one call)*
- *Often much more efficient - turns $O(n^2)$ problems into $O(n \log n)$ solutions*



- **Purpose**: Core problem-solving approaches and mental frameworks for computational challenges
- **Why it matters**: These are the fundamental thinking patterns that solve 90% of computational problems. Master these strategies, and you can tackle almost any coding challenge.
- **Real-world impact**: How Netflix recommends movies, how GPS finds optimal routes, how compression algorithms work, how game AI makes decisions.

Backtracking

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

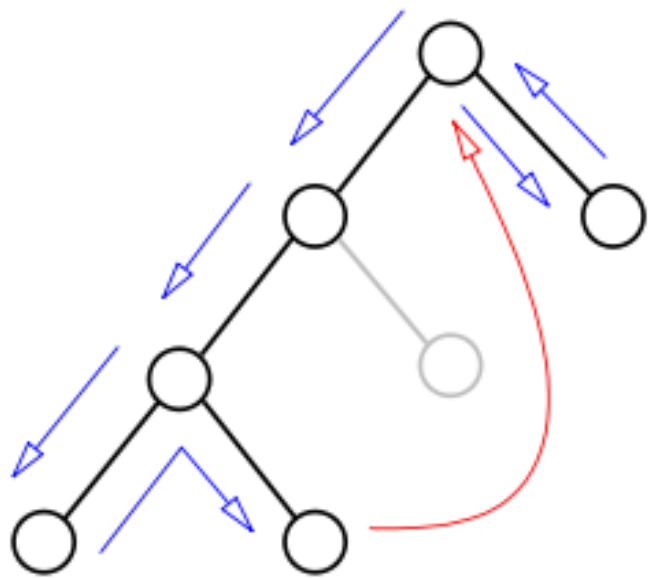
Backtracking

- What is backtracking?
- NQueens problem
- Subset problem

What is Backtracking?

- **Trying out different sequences** of decisions until we find one that "works."

- **The key insight: systematic exploration**
  - **Try a choice** → if it works, continue deeper
  - **If it fails** → **backtrack** and try a different choice
  - Explore **all possibilities** until a solution is found (or none exists)



Why do we need Backtracking?

- **Some problems have no direct path to the solution**
  - You must explore **multiple possibilities** to find the answer
  - **Trial and error** is the only way forward
- **Classic problems** that require backtracking:
  - **Maze** solving - try a path, hit dead end, go back and try another
  - **Sudoku** puzzles - place a number, if it leads to contradiction, undo and try different number
  - Finding all **permutations** - generate all possible arrangements
  - ...
- Backtracking is essential to **find solutions that follow specific rules**

Why is recursion essential?

Backtracking Algorithm

- **Automatic "undo" mechanism**
  - When function returns → automatically back to previous decision point
  - No manual tracking needed
- **Each recursive call = one choice**
  - Try choice → recurse → if fails, return = automatic backtrack
- **Without recursion:**
  - You'd manually track every choice made
  - Manually undo each choice when stuck
  - Much more complex code

Backtracking in real life

- **Navigating a maze:** Walk forward until you hit a wall, then go back to the last intersection and try a different path
- **Planning a vacation:** Book flights → hotels → activities, but if no hotels available, go back and try different flights
- **Job hunting:** Apply to Company A → if rejected, try Company B → if also rejected, maybe change your resume and start over
- ...

Types of Backtracking

Backtracking Algorithm

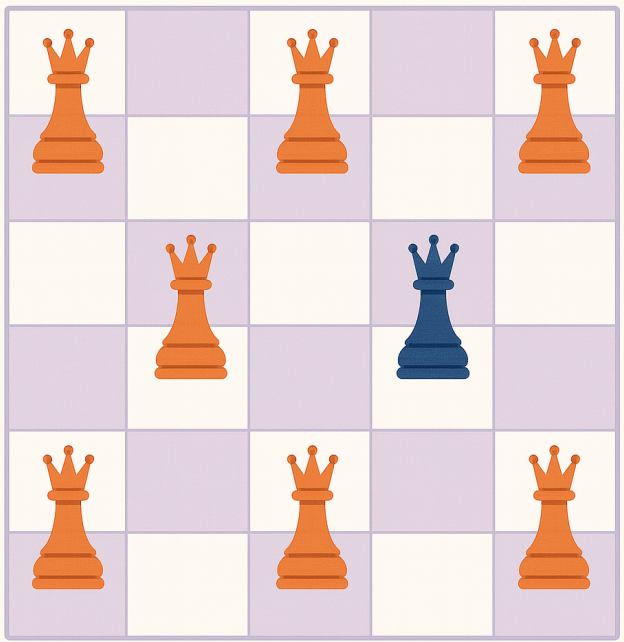
- **Decision** Problem: search for a feasible solution.
  - *Finding whether there exists a path from A to B in a maze.*
- **Optimization** Problem: search for the best solution.
  - *The Traveling Salesman Problem - finding the shortest route to visit all cities.*
- **Enumeration** Problem: find all feasible solutions.
  - *Generating all permutations of a given set of elements.*

Backtracking

- ~~What is backtracking?~~
- NQueens problem
- Subset problem

Hello eight queens!

- **Hello world** of backtracking
- *The eight queens puzzle is the problem of placing eight chess queens on an 8×8 chessboard so that no two queens threaten each other: a solution requires that no two queens share the same row, column, or diagonal.*

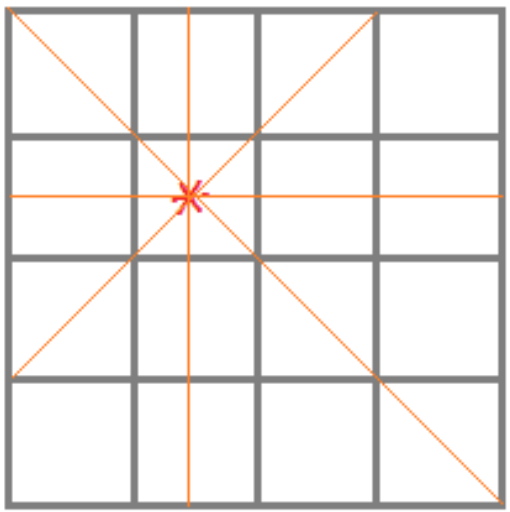


Place the queens!

1. Start in the leftmost column
2. If all queens are placed. return true
3. Try all rows in the current column.

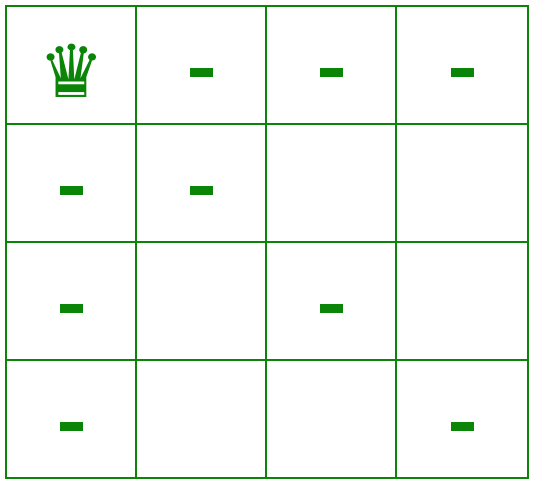
   Do the following for every row:

   1. if the queen can be placed safely: mark this and go on
   2. if placing a queen leads to a solution: return true
   3. if placing a queen doesn't lead to a solution: unmark



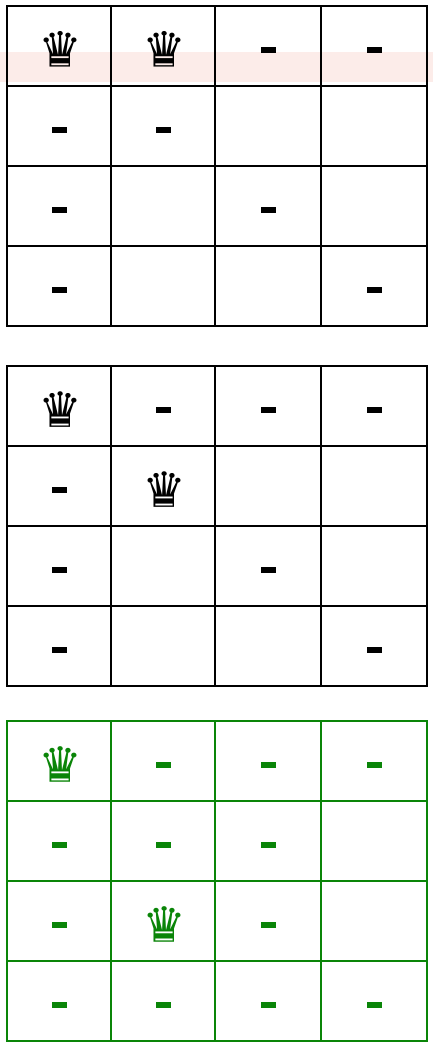
Let's go for four (queens ;))

- Start in column 1
- Find the best row (= best fit)
- In this scenario: the queen is just happy where she is: 1,1



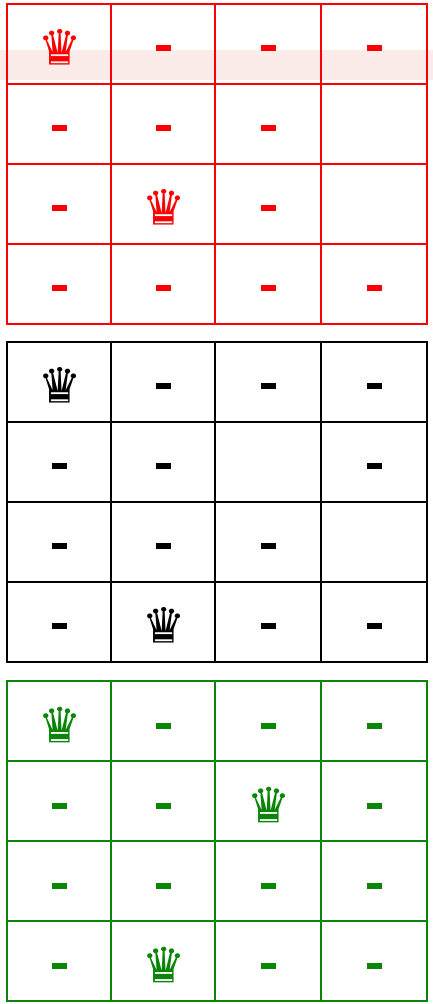
The second queen

- Place in column 2: find the best row (= best fit)
  - row 1: no!
  - row 2: no!
  - row 3: hooray!



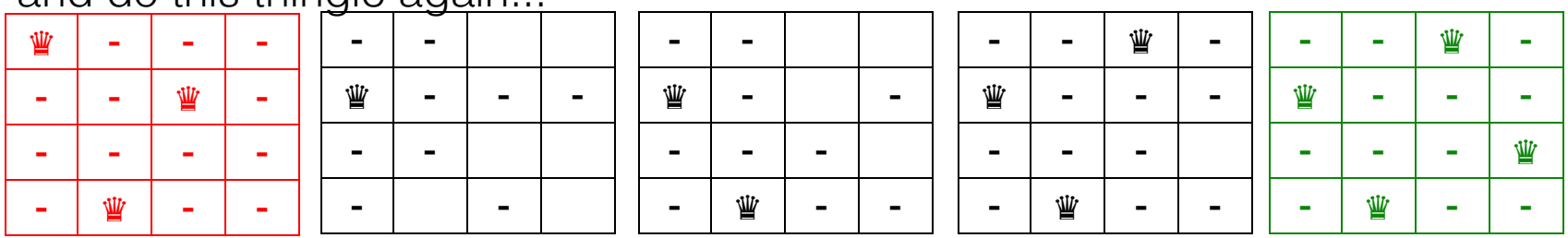
The third queen

- Find a good spot in column 3, this going to be a problem, because everything is blocked ...
- Backtracking (this is just working for us because we used recursion and we will pick up where we left off - hooray for the call stack!)
  - place the second queen in 4th row (next row)
- Found a good spot in column 3!



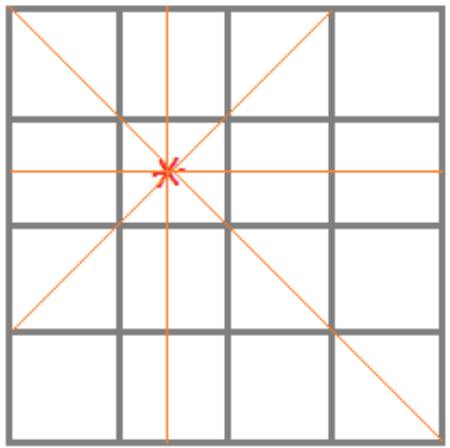
The fourth queen

- Find a good spot in column 4... cr!$ç&!!p, another problem!
- Backtracking!
  - go all the way back to first column
  - and do this thingie again...



Now code!

- Board = 2D array of ints/chars/...
- Solve the queens problem by starting in column 0
  - If all queens are placed. return true
  - Try all rows in the current column (for-loop)
    1. if the queen can be placed safely: mark this and go on
       Note: we should only check things on the left side! We traverse the columns, so there can never be a queen on the right side!
    2. if placing a queen leads to a solution: return true
    3. if placing a queen doesn't lead to a solution: unmark



Backtracking

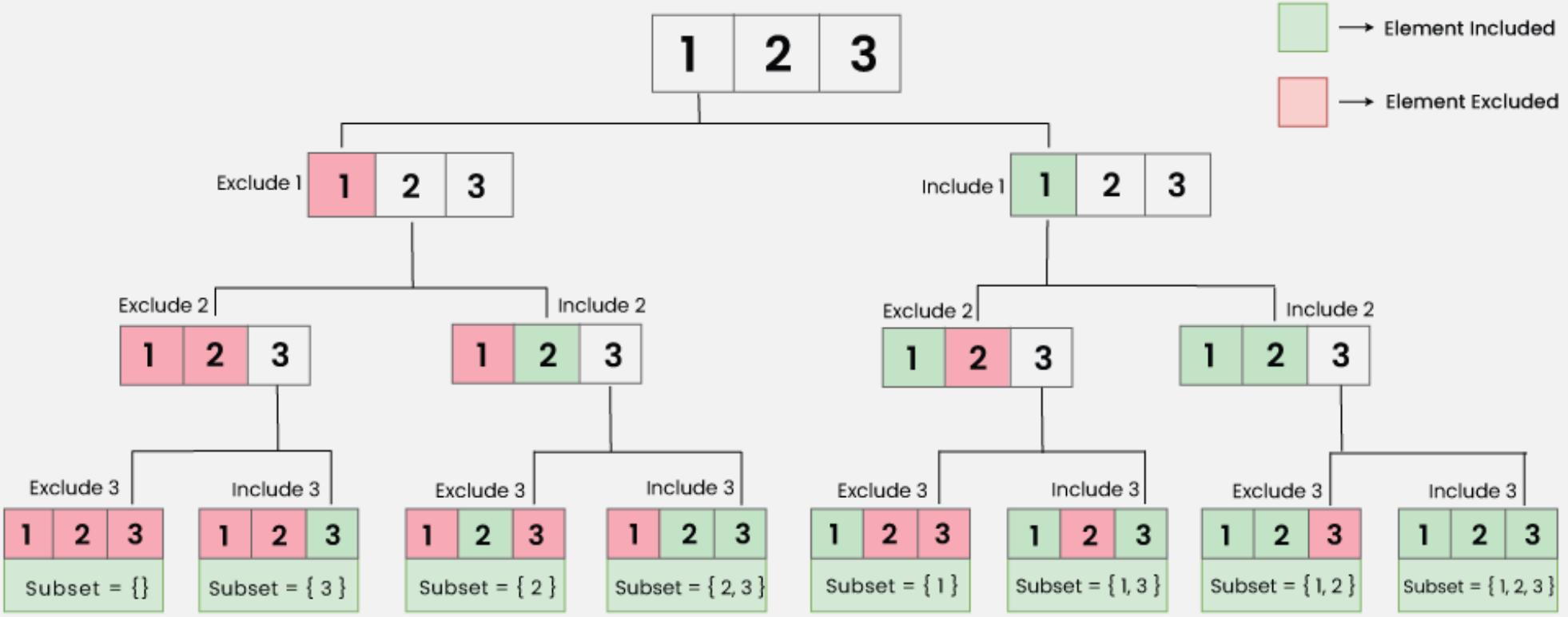
- ~~What is backtracking?~~
- ~~NQueens problem~~
- Subset problem

Subset problem

- Given an array Arr[] of size N, we want to print all the subsets of the array.
- A subset of an array is a tuple that can be obtained from the array by removing some (possibly all) elements of it.
- Example
  - Input: N = 3 Arr = [1, 2, 3]
  - Output: {} {1} {1, 2} {1, 2, 3} {1, 3} {2} {2, 3} {3}
  - Explanation: These are all the subsets that can be formed from the given array.
    It can be proven that no other subset exists other than the given output

https://www.geeksforgeeks.org/backtracking-algorithms/

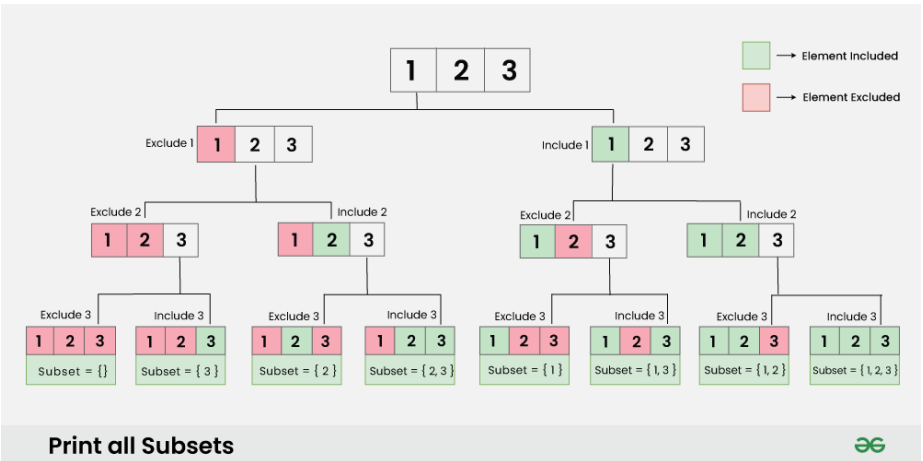
Subset problem



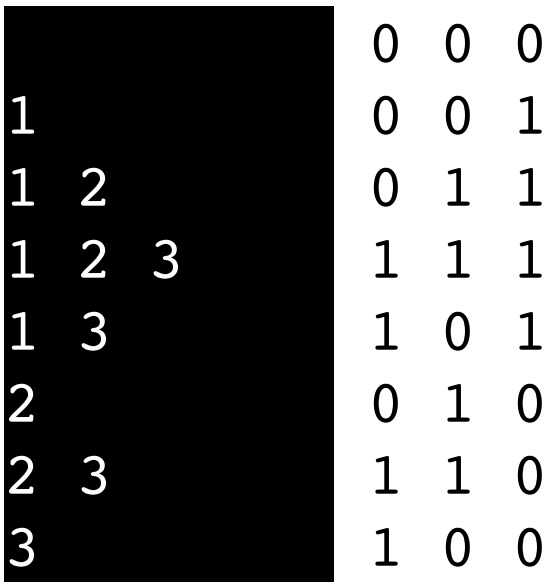
Print all Subsets

Subset problem

1. Start with empty subset and add to result list.
2. iterate through the elements of input:
    1. Include current element in the subset.
    2. Recursively call yourself with the updated subset and the next index.
    3. Exclude the current element from the subset (backtrack)



Another (binary) approach?



- This looks very binary.
- Can you come up with a different approach?
- How many subsets are there?
- 3 element = 8 subsets (2^3)

Another (binary) approach?

0 0 0
0 0 1
0 1 1
1 1 1
1 0 1
0 1 0
1 1 0
1 0 0

- Calculate how many subsets you have
- Now iterate over the possible subsets (*0 - 000, 1 - 001, 2 - 010...)*
  - We take an element or we don't
  - Combine elements in subset
  - Add subset to results list

- *Backtracking is trial-and-error problem solving*
  - *Try a choice → if it works, continue*
  - *If it fails → go back and try different choice*
- *Backtracking uses recursion:*
  - *Make a choice and recurse*
  - *If recursion fails, undo choice and try next option*

How to Think

- **Purpose**: Core problem-solving approaches and mental frameworks for computational challenges
- **Why it matters**: These are the fundamental thinking patterns that solve 90% of computational problems. Master these strategies, and you can tackle almost any coding challenge.
- **Real-world impact**: How Netflix recommends movies, how GPS finds optimal routes, how compression algorithms work, how game AI makes decisions.

Dynamic programming

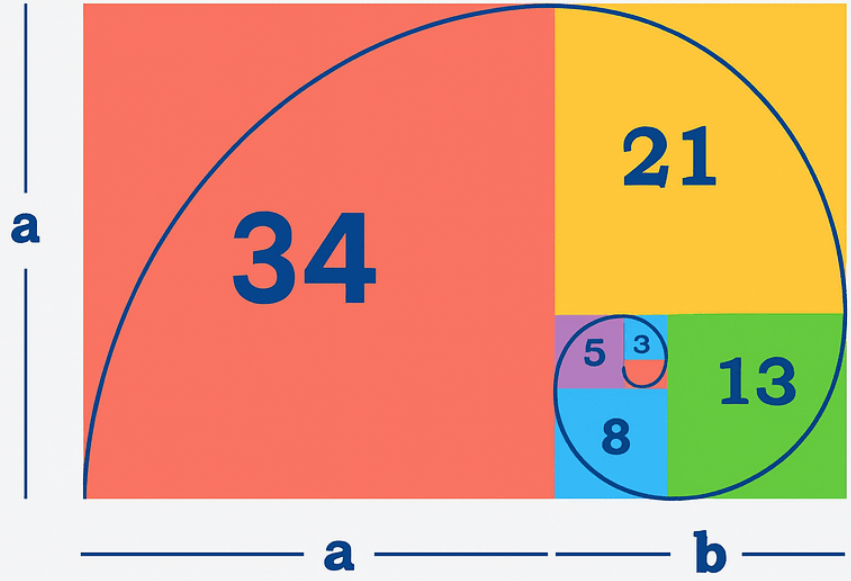
Programming advanced

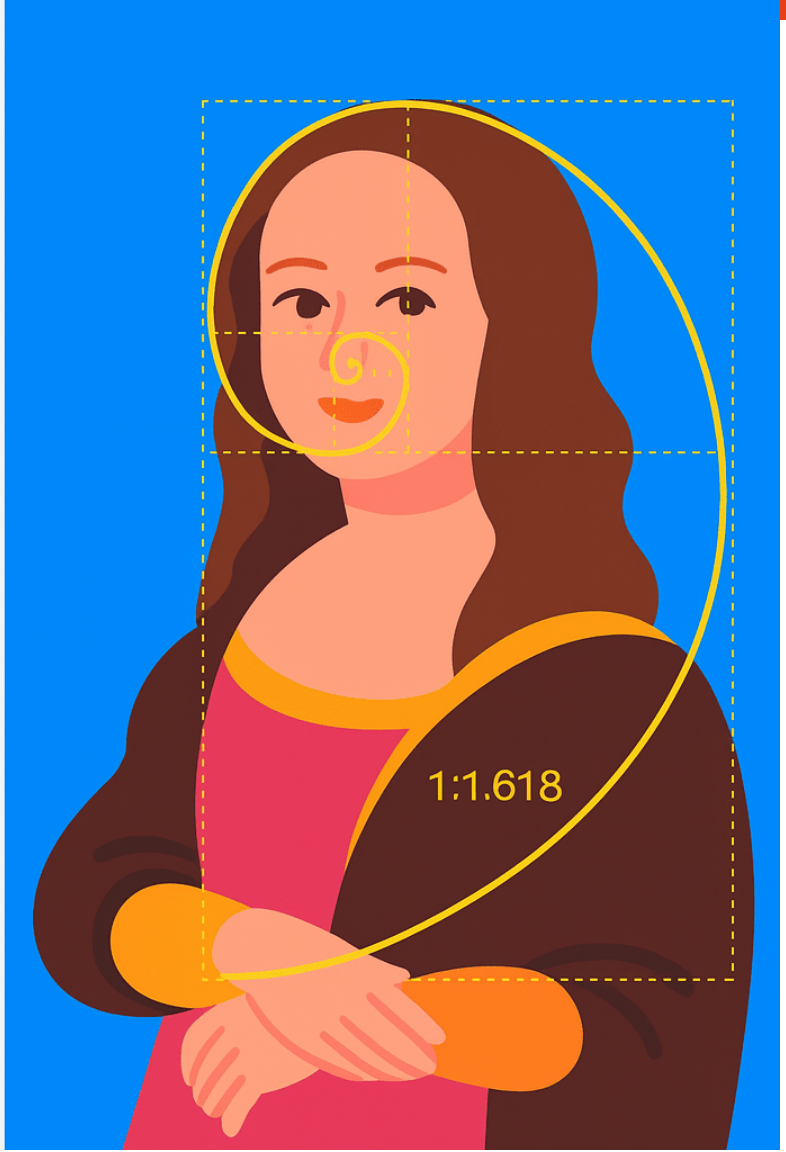
YP0822 - YP0847

Elke Boonen & Rik Tonnelier

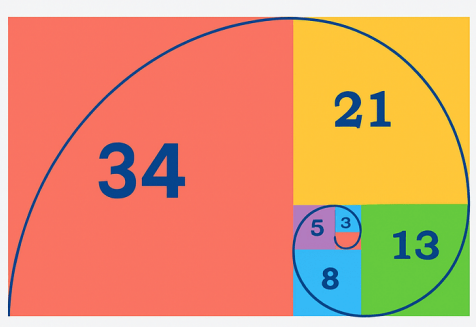
Dynamic programming

- Let's take a look at fibonacci
- What is dynamic programming?
  - Top-down: memoization
  - Bottom-up: tabulation
- Rod cutting problem
- Recursive techniques


34
21
5
3
13
8
a
a
b


1:1.618

Fibonacci

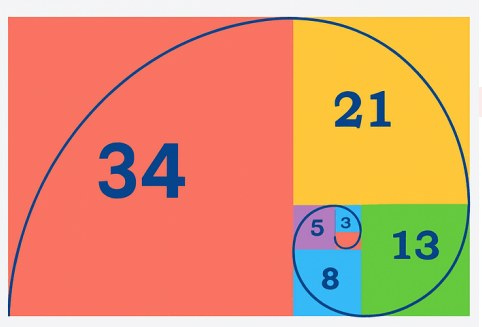


- Let's take a look to the algorithm of last year
  - return if number is 0,1
  - otherwise iterate

```
1  int Fibonacci(int number)
2  {
3    if (number <= 1) return number;
4
5    int fib1 = 0, fib2 = 1, fib3 = 1;
6
7    for (int i = 2; i < number; ++i)
8    {
9      fib1 = fib2;
10     fib2 = fib3;
11     fib3 = fib1 + fib2;
12   }
13   return fib3;
14 }
15
16 //in main
17 Console.WriteLine(Fibonacci(10));
```

```
2: fib: 1
3: fib: 2
4: fib: 3
5: fib: 5
6: fib: 8
13
```

Fibonacci



- Fibonacci is similar to Factorial, right?

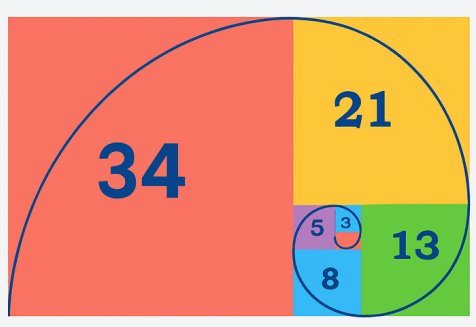
  ... and actually it is a beautiful recursive algorithm ❤

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2}, n > 1$$

Fibonacci with recursion



- Base cases: return if number is 0 or 1
- Recursive case: return with n-1 + n-2

- So beautiful in all it's simplicity ❤

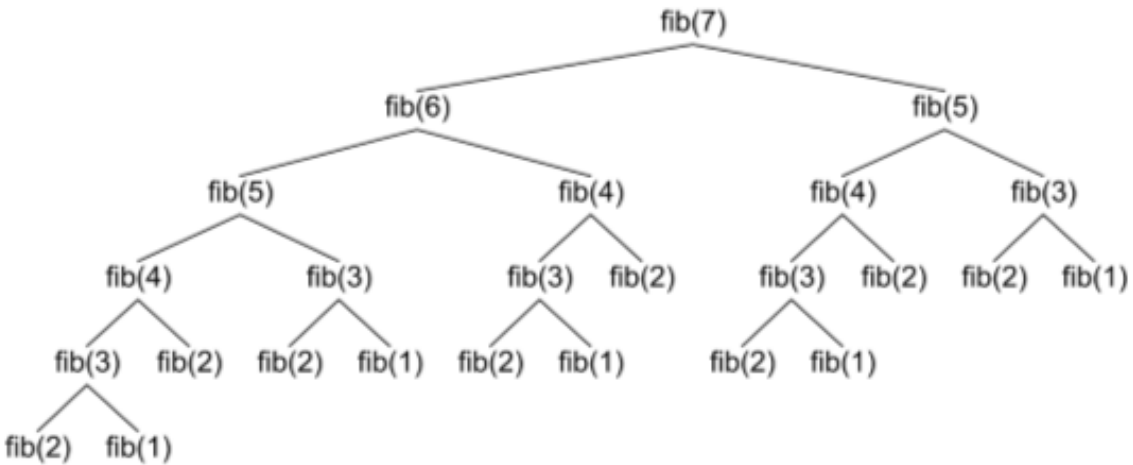
```
1 int F(int number)
2 {
3     if (number <= 1) return number;
4
5     return F(number - 1) + F(number - 2);
6 }
```

$$f_0 = 0$$
$$f_1 = 1$$
$$f_n = f_{n-1} + f_{n-2}, n > 1$$

- It will also give you the right result back, but what if we output every call... what will we see?

Fibonacci with recursion

- Output all calls
  - *Holy moly... what's that?*
- Because we make two calls every recursive case, we end up with a tree structure of calls




```
call with 7 call with 6
call with 5 call with 4
call with 3 call with 2
call with 1 call with 0
call with 1 call with 2
call with 1 call with 0
call with 3 call with 2
call with 1 call with 0
call with 1 call with 4
call with 3 call with 2
call with 1 call with 0
call with 1 call with 2
call with 1 call with 0
call with 5 call with 4
call with 3 call with 2
call with 1 call with 0
call with 1 call with 2
call with 1 call with 0
call with 3 call with 2
call with 1 call with 0
call with 1 13
```

Run time

- *Remember every call is pushed on the stack, so let's count the operations!*

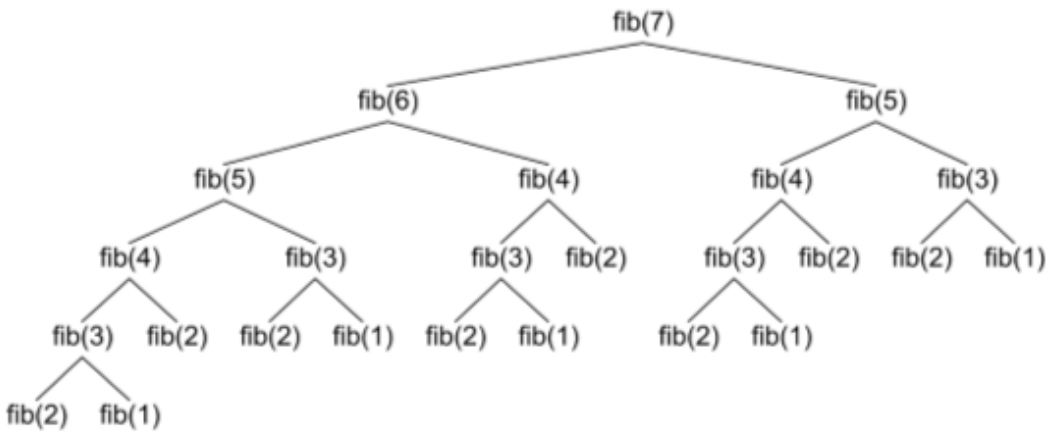
- The **iterative solution** runs in linear time $O(n)$
- The **recursive** solution in **exponential** $O(2^n)$

- So the recursive function is much slower than the iterative function!

n	iterative	recursive
0	0	1
1	0	0
2	0	3
3	1	5
4	2	9
5	3	15
6	4	25
7	5	41
8	6	67
9	7	189
10	8	177
11	9	287
12	10	465
...		

Shall I repeat myself again?

- Look at the graph, this implementation does a lot of repeated work
- So this is a bad solution for finding a Fibonacci number!
  - Boohoo, it was so beautiful!
- How can we solve this issue?
  - because yes, I like recursion ;)
  - through storing some of our work during execution!
    = dynamic programming!



Dynamic programming

- ~~Let's take a look at fibonacci~~
- What is dynamic programming?
  - Top-down: memoization
  - Bottom-up: tabulation
- Rod cutting problem

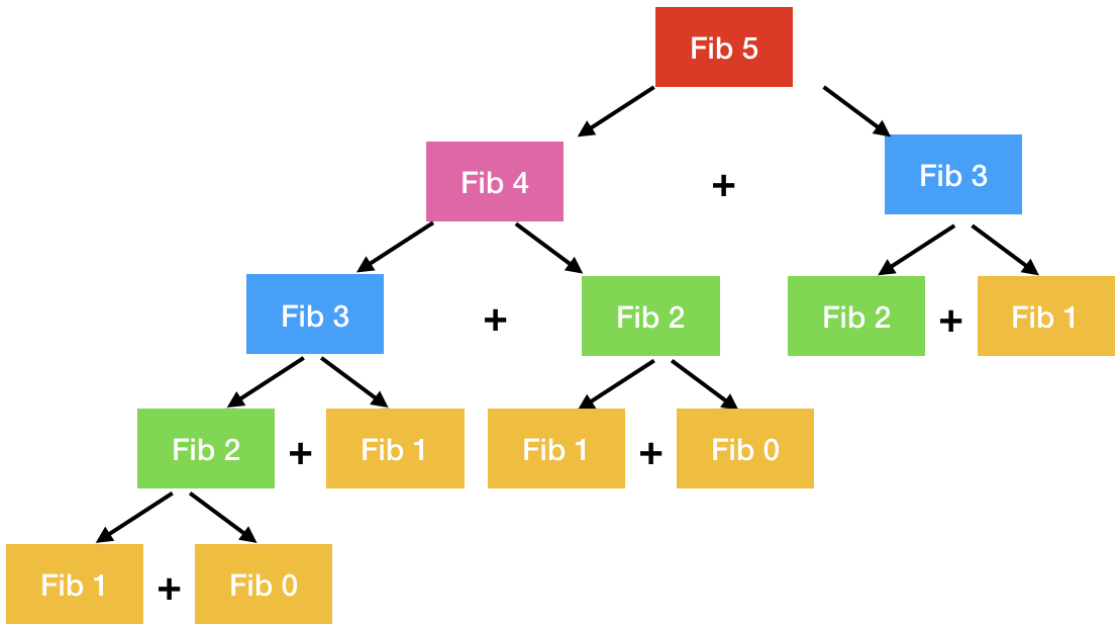
What is dynamic programming?

- **An optimization technique for plain recursion**
- Main idea = **store intermediate results!**
- Need solution again = no need in solving it again!

- Imagine bag of coins: count the total number of coins
  - Count the number of coins in first bag
  - Go to next bag
  - Are you going to count the number of coins in the first bag again? No!

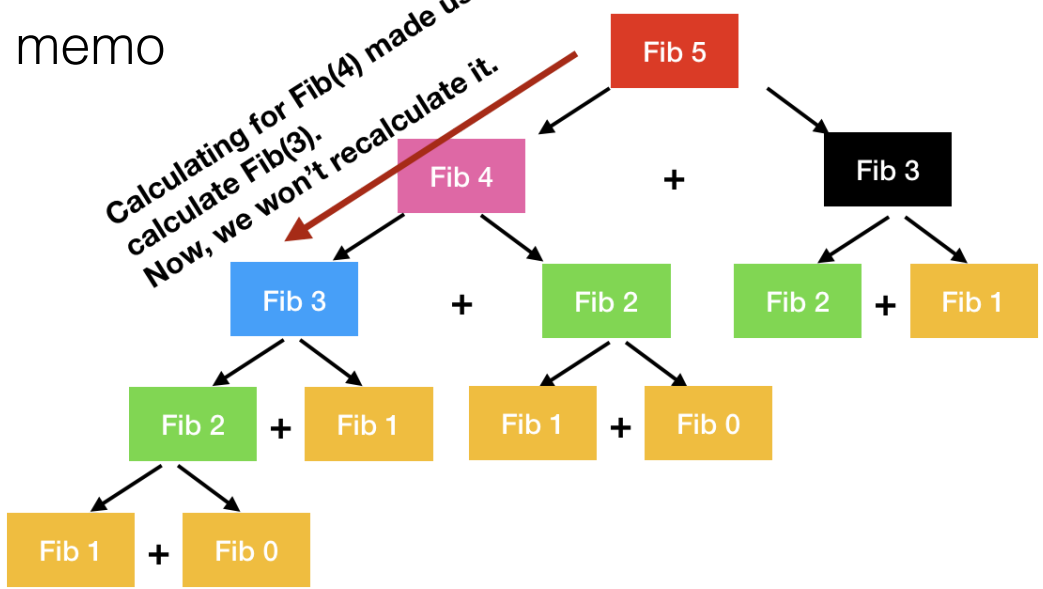
What is dynamic programming?

- Take a look at Fibonacci
- The same problem appears more than once!
  - F(1) = 4 times
  - F(2) = 3 times
  - F(3) = 2 times
  - ...
- *Test with F(50), what happens?*



Top-down: memoization

- Store solutions along the way
- Store solutions in a lookup table (array)
- Memoization comes from taking a memo



Fibonacci with memoization

- Start from the recursive solution
- Use an array to store the intermediate results!

```
1  //declare array
2  int[] mem;
3
4  int FibonacciDPMem(int number)
5  {
6    if (number <= 1) return number;
7
8    if (mem[number] > 0) return memo[number];
9
10   mem[number] = FibonacciDP(number - 1) + FibonacciDP(number -
   2);
11   return mem[number];
12 }
13
14 //in main initialise array
15 int number = 7;
16 mem = new int[number + 1]; //why + 1?
17 Console.WriteLine(FibonacciDP(number));
```

Run time

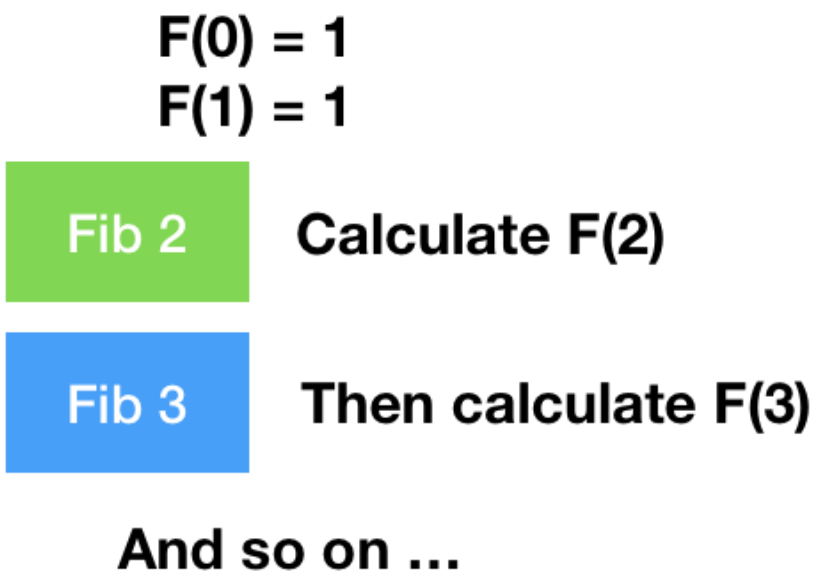
- Let's take a look at the operations count
  - It became a lot faster
  - It's now up to $O(2n)$
- Theoretically $O(n) = O(2n)$ *(Big O leaves constants out)*
- Practically O(n) will be faster, but not significant when n is large enough

n	iterative	recursive	dynamic
0	0	1	0
1	0	1	0
2	0	3	1
3	1	5	2
4	2	9	4
5	3	15	6
6	4	25	8
7	5	41	10
8	6	67	12
9	7	109	14
10	8	177	16

www.desmos.com/calculator/hr5qhhqklg?embed

Bottom-up: tabulation

- Start at the bottom
- Fill a lookup table from the start
- Start with the smallest item
  (sorting elements first)



Fibonacci with tabulation

- Throw recursion overboard ;)
- Run time is same as iterative solution!

```
1  int FibonacciDPTab(int number)
2  {
3    int[] tab = new int[number + 2]; //why + 2?
4    tab[0] = 0;
5    tab[1] = 1;
6    for (int i = 2; i < number; i++)
7    {
8      tab[i] = tab[i - 1] + tab[i - 2];
9    }
10   return tab[number];
11 }
```

Memoization <> tabulation

- **Memoization**
  - Fill table on demand
  - **A natural way of solving a problem (coding is easier with a complex problem)**
  - When *you don't need all the solutions to subproblems*
- **Tabulation**
  - The table is filled from the first entry to the last
  - **You need to know the order of elements (harder to code)**

> *Generally, memoization is slower than tabulation because of recursive calls.*

So, which one is the best?

- So… which solution is the best?
  - Look at run time
    - Iterative, memoization, and tabulation $O(n)$
    - Recursive $O(2^n)$
  - Look at space needed
    - Iterative: always 3 variables $O(1) - constant$
    - Memoization: array n+1 places $O(n)$
    - Tabulation: array n+1 place $O(n)$

So, when to use what?

- Use **recursion** when it solves the **problem more clearly**!

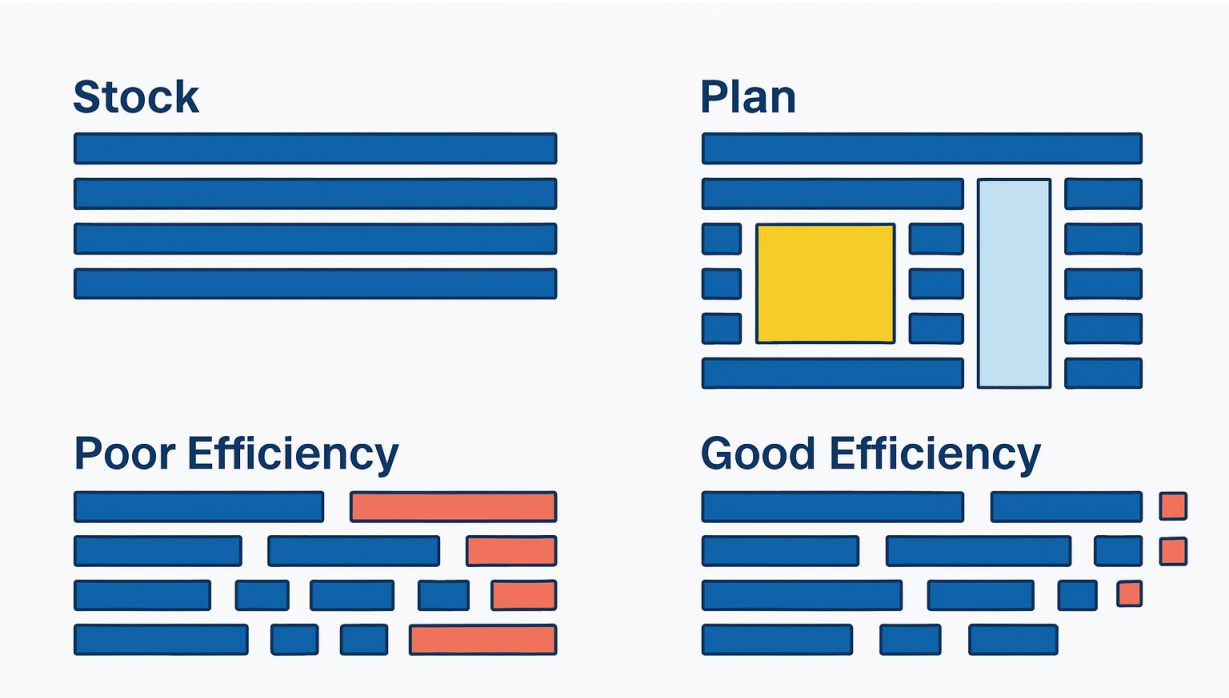
- You can use **dynamic programming** when
  - there are **overlapping subproblems**
  - the problem has an **optimal substructure**:
    the optimal solution can be obtained by optimal solutions of its subproblems.

Dynamic programming

- ~~Let's take a look at fibonacci~~
- ~~What is dynamic programming?~~
  - ~~Top-down: memoization~~
  - ~~Bottom-up: tabulation~~
- Rod cutting problem
- Recursive techniques

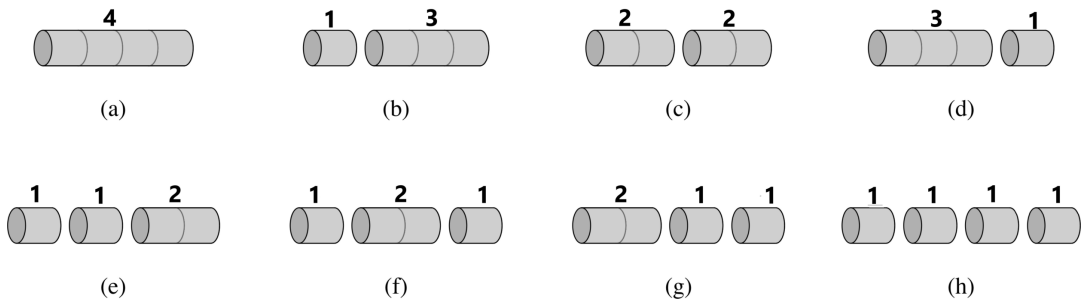
Rod cutting problem

- Another typical problem!
- 1D stock cutting problem
- A rod of length n needs to be cut in different sizes
  - Each size = different price
  - How to maximize our profit?
- Real-life problem!



Rod cutting problem

- n = 4 units, how to maximize profit?



- 8 possible options:
  - a = 9 (no cuts)
  - b = 1 + 8 = 9 (= d) (1 cut)
  - **c = 5 + 5 =10 (1 cut)**
  - e = 1 + 1 + 5 = 7 (= f,g) (2 cuts)
  - h = 1 + 1 + 1 + 1 = 4 (3 cuts)

unit size	price
1	1
2	5
3	8
4	9
5	10
6	17
7	17
8	20
9	24
10	30

Rod cutting problem

- How to solve this? Divide & conquer!
- Solve this by creating smaller instances of same problem
- r = max amount of money, p = piece

$$r_n = max(p_i + r_{n-i}) \; with \; 1 <= i <= n$$

Divide & conquer!

$$r_n = max(p_i + r_{n-i}) \; with \; 1 <= i <= n$$

- We can solve this problem by 'cutting' it into smaller problems!

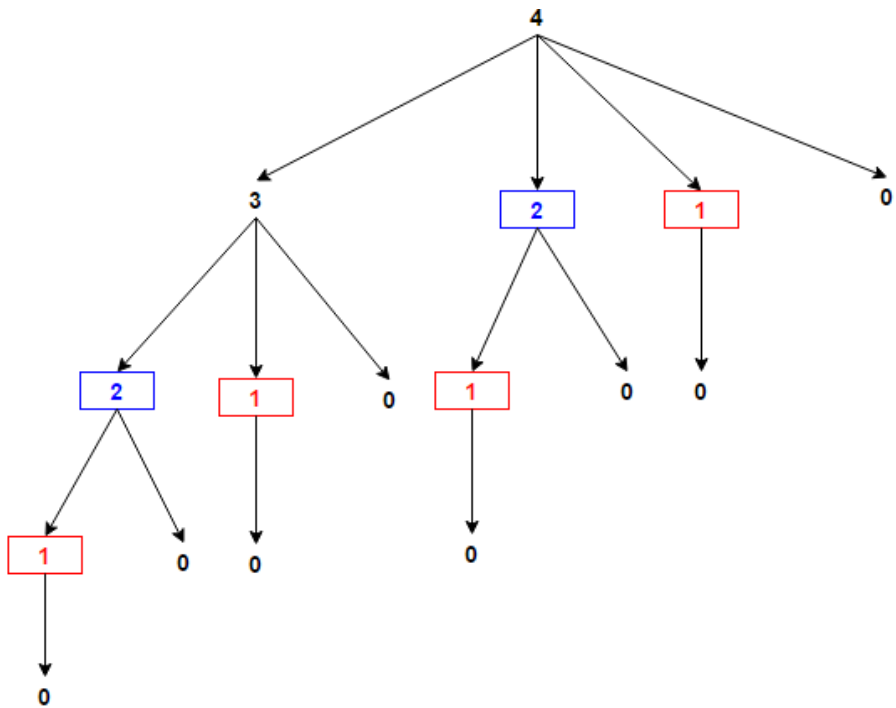
```
1  public int Recursion(int n)
2  {
3    if (n == 0) return 0; //base case
4    int max = -1;
5    for (int i = 1; i <= n ; i++)
6    {
7      //prices = property in class
8      max = Math.Max(max, Prices[i - 1] + Recursion(n - i));
9    }
10   return max;
11 }
```

```
Highest profit: 10
```

```
1 Cutting c = new Cutting(new int[] { 1,5,8,9 });
2 Console.WriteLine("Highest profit: " + c.Recursion(4));
```

Overlapping subproblems!

- 8 possible solutions
- Same subproblems
  - 4 times 1
  - 2 times 2
- Dynamic programming!

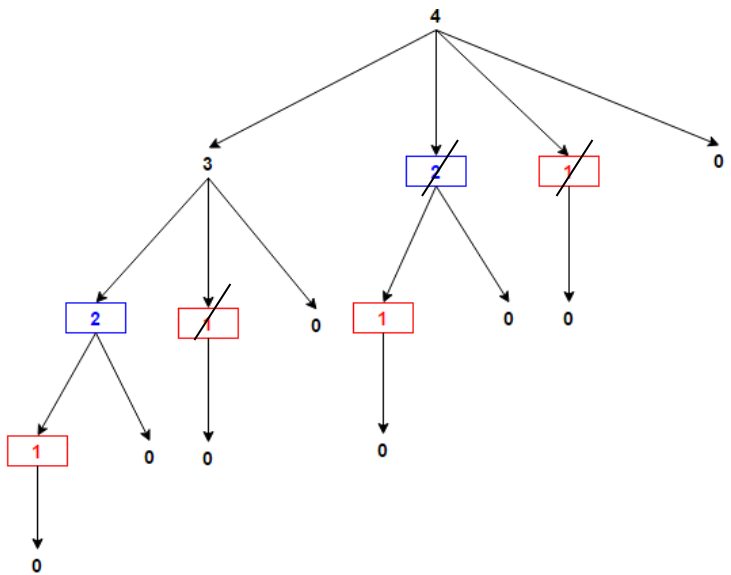


Memoization

- Store intermediate results along the way!
- Easy to transform!
  - create memorization array - initialize array

```
1  public int Memoization(int n)
2  {
3    if (_memoization[n] >= 0) return _memoization[n];
4    if (n == 0) return 0;
5    int max = -1;
6    for (int i = 1; i <= n; i++)
7    {
8      max = Math.Max(max, Prices[i - 1] + Memoization(n -
9      _memoization[n] = max;
10   }
11   return max;
12 }
```

```
1 Cutting c = new Cutting(new int[] { 1,5,8,9 });
2 Console.WriteLine("Highest profit: " + c.Memoization(4));
```



Tabulation

- Bottom-up approach
- How are you going to fill the table?
  - start with the smallest piece (rod length =1)
  - use the smallest piece build up!
- Look at the formula again

$$r_n = max(p_i + r_{n-i}) \; with \; 1 <= i <= n$$

unit size	price
1	1
2	5
3	8
4	9
5	10
6	17
7	17
8	20
9	24
10	30

Tabulation

- Implement $r_n = max(p_i + r_{n-i})\ with\ 1 <= i <= n$
  - $n = 1 \Rightarrow r_1 = 1$
  - $n = 2 \Rightarrow r_2 = 5$
  - $n = 2 \Rightarrow r_3 = 8$
  - $n = 3 \Rightarrow r_4 = 10$

$$max\ of$$
$$i = 1 \rightarrow p_1 + r_1 = 1 + 1 = 2$$
$$i = 2 \rightarrow p_2 + r_0 = 5 + 0 = 5$$

$$max\ of$$
$$i = 1 \rightarrow p_1 + r_2 = 1 + 5 = 6$$
$$i = 2 \rightarrow p_2 + r_1 = 5 + 1 = 6$$
$$i = 3 \rightarrow p_3 = 8$$

$$max\ of$$
$$i = 1 \rightarrow p_1 + r_3 = 1 + 8 = 9$$
$$i = 2 \rightarrow p_2 + r_2 = 5 + 5 = 10$$
$$i = 3 \rightarrow p_3 + r_1 = 8 + 1 = 9$$
$$i = 4 \rightarrow p_4 = 9$$

unit size	price
1	1
2	5
3	8
4	9
5	10
6	17
7	17
8	20
9	24
10	30

Tabulation

- Now really implement the formula! $r_n = max(p_i + r_{n-i})\ with\ 1 <= i <= n$
  - j <= i

```
1  public int Tabulation(int n)
2  {
3    int[] tabulation = new int[_length + 1];
4    int[] s = new int[_length + 1];
5    tabulation[0] = 0;
6
7    for (int i = 1; i <= n; i++)
8    {
9      int max = -1;
10     for (int j = 1; j <= i; j++)
11     {
12       max = Math.Max(max, Prices[j - 1] + tabulation[i - j]);
13       tabulation[i] = max;
14     }
15    }
16   return tabulation[n];
17 }
```

Dynamic programming

- ~~Let's take a look at fibonacci~~
- ~~What is dynamic programming?~~
  - ~~Top-down: memoization~~
  - ~~Bottom-up: tabulation~~
- ~~Rod cutting problem~~
- Recursive techniques

RECAP

Algorithmic (recursive) techniques

1. **Brute-force** - Try all possibilities
2. **Divide and Conquer** - Break down the problem into smaller, solvable sub-problems, then combine results.
3. **Dynamic Programming** (DP) - Break down into sub-problems, storing solutions to avoid recomputation
4. **Backtracking** - Try solutions recursively, backtrack when constraints are violated.

Where to use recursion?

- **Brute-force** - **Sometimes** used to generate all combinations
  - Focus: Exhaustive search
- **Divide and Conquer** - **Essential** - multiple recursive calls (left half, right half)
  - Focus: Breaking down + combining
- **Dynamic Programming** - **Optional** - can use recursion + memoization OR iterative approach
  - Focus: Avoiding redundant work
- **Backtracking** - **Essential** - provides automatic "undo" mechanism
  - Focus: Trial-and-error with systematic backtracking

When to use which technique?

- **Brute-force**: Small datasets, simple problems, or when no better solution exists
- **Divide & Conquer**: Large problems that can be split into similar, independent parts
- **Dynamic Programming**: Problems with overlapping subproblems (same calculation repeated)
- **Backtracking**: Constraint satisfaction problems (puzzles, combinations with rules)

What about time & space efficiency?

- **Time Efficiency**
  - Brute-force: Usually slow (tries everything)
  - Divide & Conquer: Often efficient (O(n log n))
  - Dynamic Programming: Fast by avoiding recalculation
  - Backtracking: Can be slow (explores many possibilities) but often the only way
- **Memory usage**
  - Brute-force: Low memory
  - Divide & Conquer: Moderate (call stack)
  - Dynamic Programming: High (stores all solutions)
  - Backtracking: Moderate (call stack for current path)

Graphs

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

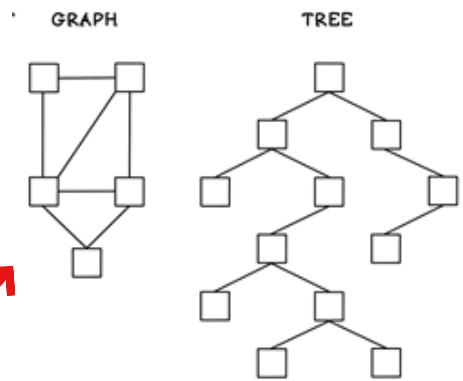
Graphs

- Graphs
- The maze!
- Searching in graphs
  - Depth-first search
  - Breadth-first search

What are graphs?

- **Non-linear data structure** consisting of vertices and edges
- Graphs are a simple and understandable way to **visualize relationships**
- **Derived from mathematics**

  G = (V, E) where G represents the graph
  - **Vertices (V)**: Set of vertices that store data
  - **Edges (E)**: Set of connections between vertex pairs
- **Trees are a special kind of graphs**



Why do we need graphs?

- **Relationship modeling**: Represent complex many-to-many relationships between data elements
- **Algorithm efficiency**: Enable specialized algorithms (BFS, DFS, shortest path, MST)
- **Traversal capabilities**: Systematic ways to visit and process connected data
- **Problem solving**: Natural fit for network, connectivity, and pathfinding problems
- **Pattern discovery**: Identify hidden structures and trends in interconnected data
- **Optimization**: Find best routes, most efficient connections and cost savings

Graphs in real life?

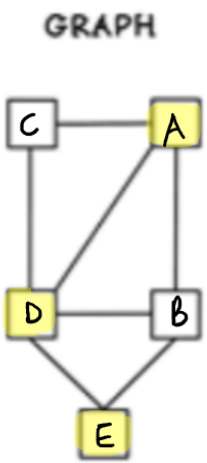
- **Social networks**: Facebook, LinkedIn - people as vertices, friendships as edges
- **Transportation systems**: Road networks, flight routes, metro locations and routes for GPS navigation
- **Internet & Web**: Servers connected by networks, web pages linked by hyperlinks
- **Biological networks**: Protein interactions, food chains in ecosystems
- **Supply chain**: Suppliers, distribution centers and retailers for logistics optimization
- **Computer networks**: Network topology design, routing protocols, cybersecurity
- **...**

A lot of different graphs

- **Undirected**: relationships are mutual (eg roads between cities)
- **Directed**: edges indicate a direction (like parents/children)
- **Disconnected**: one or more nodes are not connected (like continents)
- **Acyclic**: no cycles
- **Complete**: every node is connected to every other nod
- **Weighted:** edges between nodes have a weight
- ...

A path in a graph

Graphs



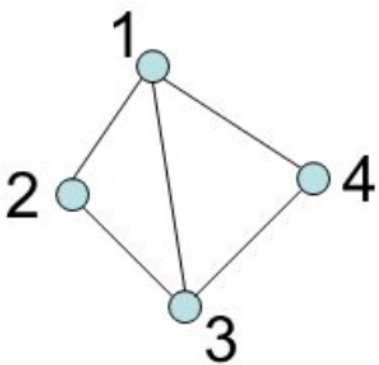
NODES = A, B, C, D, E
EDGES = AB, AC, AD
BD, BE
CD
DE

PATH FROM A TO E

- Graphs contain **nodes** (vertices) and **connections** (edges)
- **Move** from one **node** to **another via edges**
- **A path is a sequence of nodes connected through edges**
- Think of **nodes as destinations** and **edges** as the **routes** between them
- **Multiple paths may exist** between two nodes
- Goal is to find the **optimal route**!

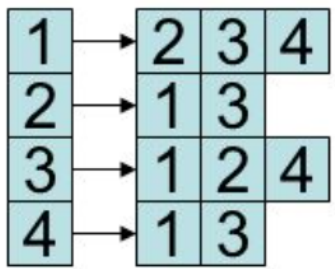
From graph to code!

- We can use an adjacency matrix or list!
  - adjacency matrix = 2d array
  - adjacency list = array of lists



	1	2	3	4
1	0	1	1	1
2	1	0	1	0
3	1	1	0	1
4	1	0	1	0

Adjacency matrix

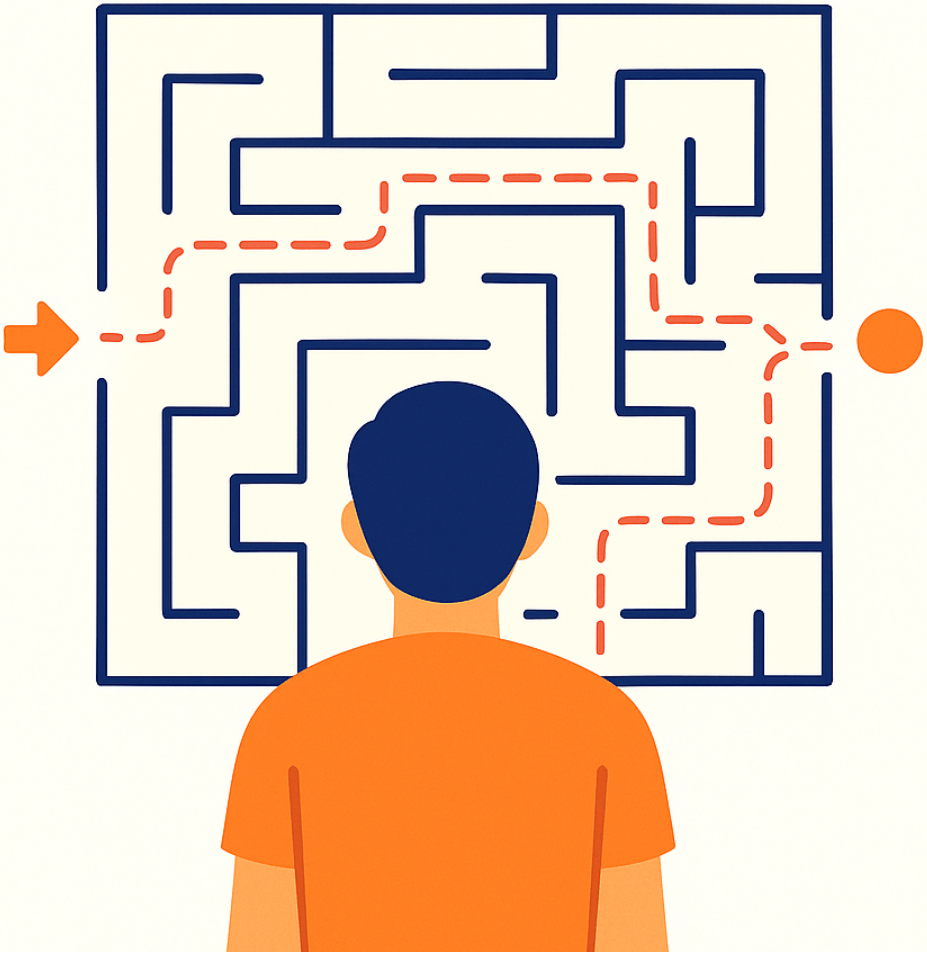


Adjacency list

Graphs

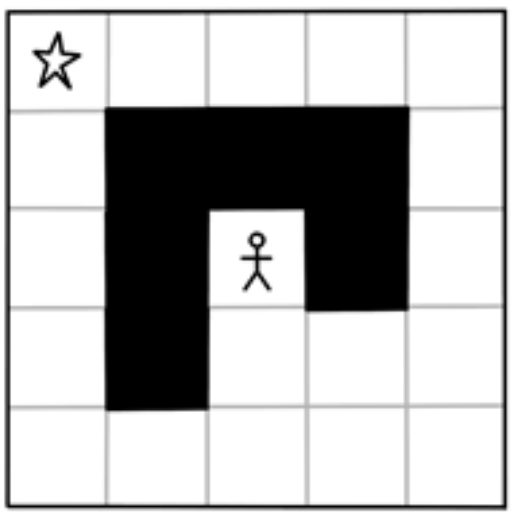
- ~~Graphs~~
- The maze!
- Searching in graphs
  - Depth-first search
  - Breadth-first search

Navigating a Maze



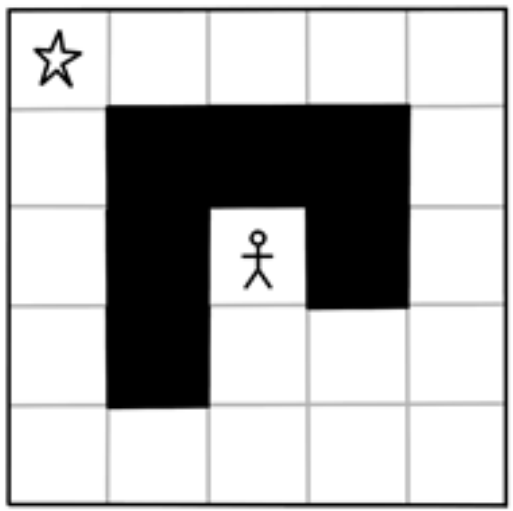
The maze!

“ *Find a path out of the maze!*

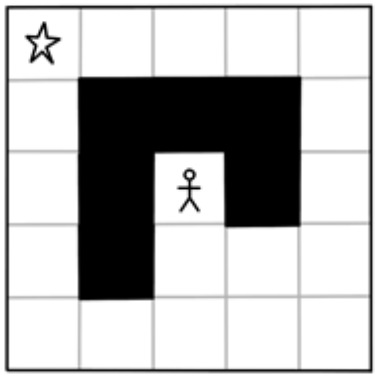


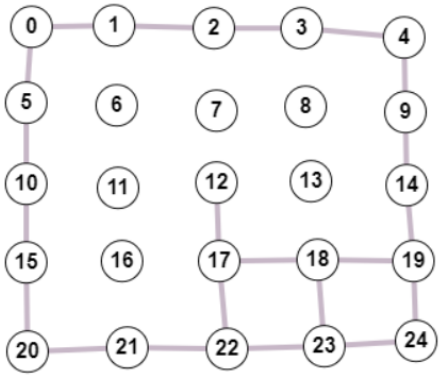
Translate maze to data structure!

- How on earth can we do that?
- Start by translating your maze to a graph!
- How many nodes are in this maze?
  - 5? 10? 25?
  - wait... see those boxes?
- Nodes have connections between them (= edges)
  - white boxes: we have a connection, we can move!
  - black boxes: no connection, we can't move!



Translate maze to data structure!





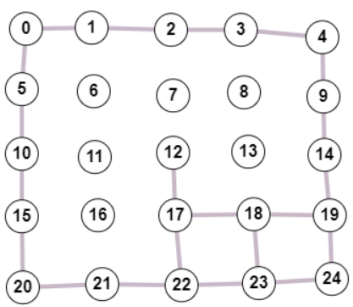
NODES = 0 ... 24

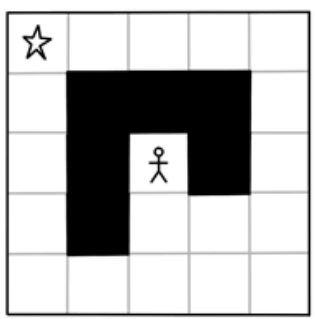
EDGES = 01, 05
10, 12
21, 23
32, 34
43, 49
94, 914
149, 1419
1918, 1924, 1914
...

- How to code our maze?
  - all boxes are nodes
  - all adjacent white boxes represent edges
- Choose between an adjacency list or matrix!
  - you can choose whatever you like, I chose a list (hash table last week)

Translate maze to data structure!

- An adjacency list = **array of lists**
  - array = fixed number of nodes
  - per node = list of 'connected neighbors'
  - **initialize all lists per node**
- Build up the adjacency list by adding edges!
- If one node is connected, it's also the other way around!

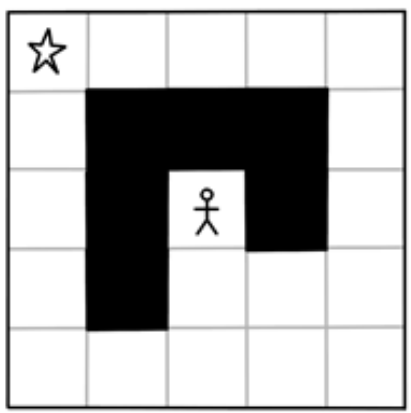


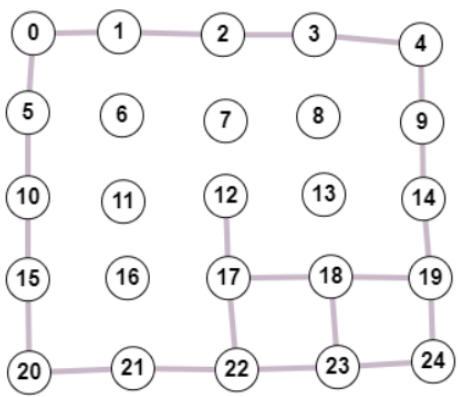


```
1  class Program
2  {
3    static void Main(string[] args)
4    {
5       Maze maze = new Maze(25);
6
7       maze.AddEdge(0, 1);
8       maze.AddEdge(1, 2);
9       maze.AddEdge(2, 3);
10      maze.AddEdge(3, 4);
11      maze.AddEdge(4, 9);
12      maze.AddEdge(9, 14);
13      maze.AddEdge(18, 17);
14      maze.AddEdge(18, 19);
15      maze.AddEdge(19, 14);
16      maze.AddEdge(19, 24);
17      maze.AddEdge(24, 23);
18      maze.AddEdge(18, 23);
19      maze.AddEdge(22, 23);
20      maze.AddEdge(17, 12);
21      maze.AddEdge(21, 22);
22      maze.AddEdge(21, 20);
23      maze.AddEdge(20, 15);
24      maze.AddEdge(15, 10);
25      maze.AddEdge(10, 5);
26      maze.AddEdge(5, 0);
27
28      Console.WriteLine(maze.ToString()
29 }
```

Print adjacency list

- Check the nodes & connections (=edges) by printing them out!
- Is everything ok?





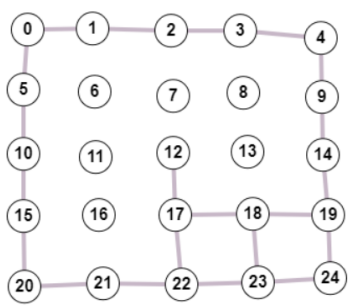

```
0 --> 1 5
1 --> 0 2
2 --> 1 3
3 --> 2 4
4 --> 3 9
5 --> 10 0
6 -->
7 -->
8 -->
9 --> 4 14
10 --> 15 5
11 -->
12 --> 17
13 -->
14 --> 9 19
15 --> 20 10
16 -->
17 --> 18 12
18 --> 17 19 23
19 --> 18 14 24
20 --> 21 15
21 --> 22 20
22 --> 23 21
23 --> 24 18 22
24 --> 19 23
```

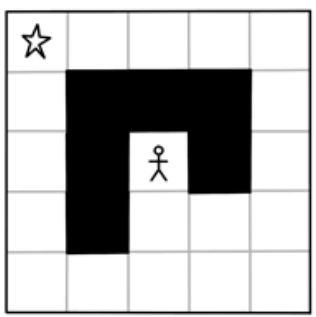
Graphs

- ~~Graphs~~
- ~~The maze!~~
- Searching in graphs
  - Depth-first search
  - Breadth-first search

The maze!

- How do I get out of the maze?
- How do we find the exit?
  - we have our coded maze!
  - we have nodes and connections (edges)
  - we know where we are (node 12)
  - we 'just' need to search for a route out!
- Can we come up with an algorithm?



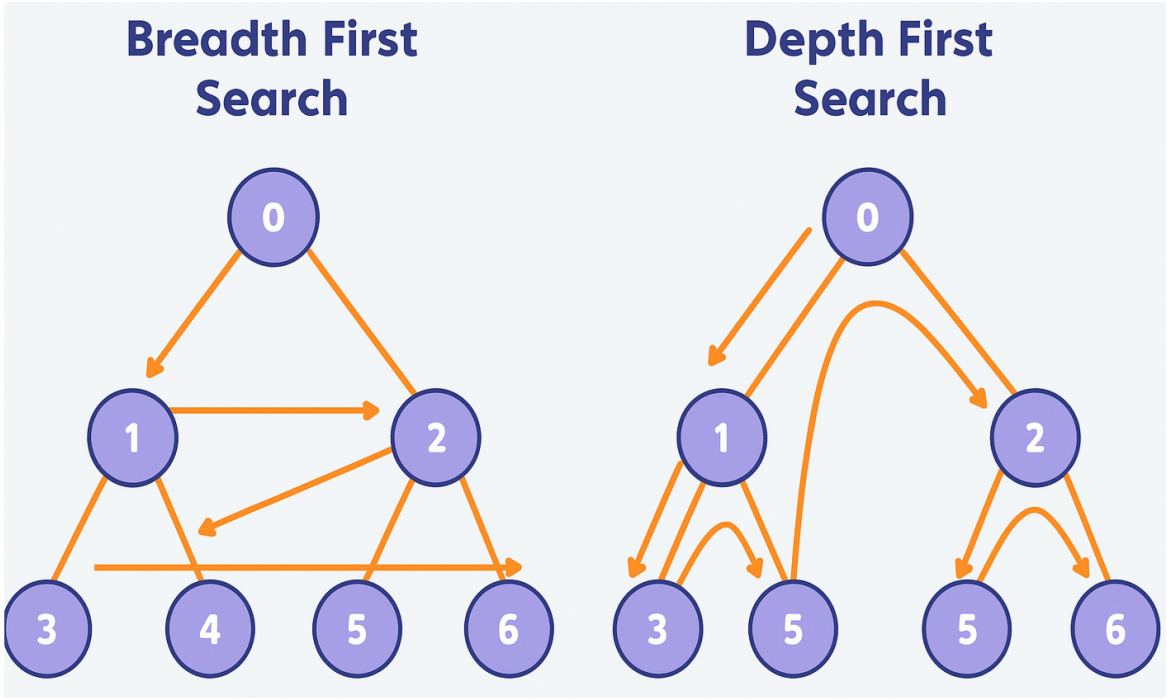


Searching in graphs/trees

- Suppose you want to learn something new, how do you start learning?
  - **Wide search:** looking for many things
    - In the maze: looking wide = looking at all the neighbors (children), see what it brings you
  - **Narrow search:** immerse yourself in one thing
    - In the maze: looking deep = take a node and start digging on one path
- Have we ever done this kind of search?
  - Yes, we did! Think about searching for the key in a box (the file in a folder!)
  - Think about the lesson about recursion...

Deep or wide search

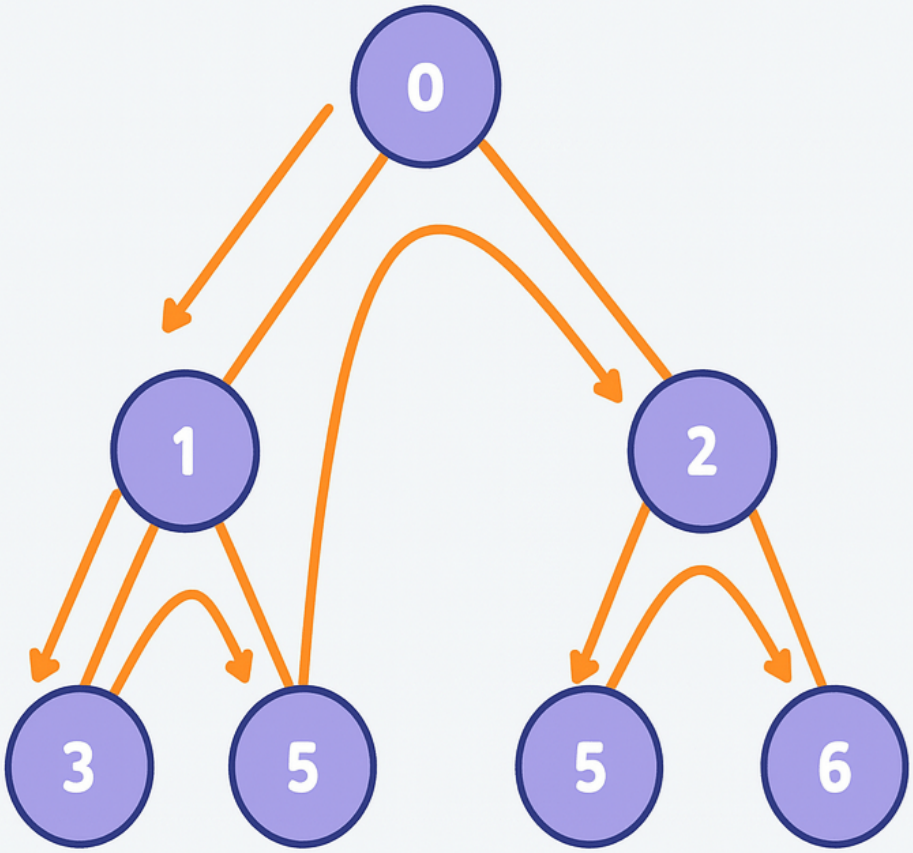
- Deep search = **Depth-first search**
- Wide search = **Breadth-first search**



Graphs

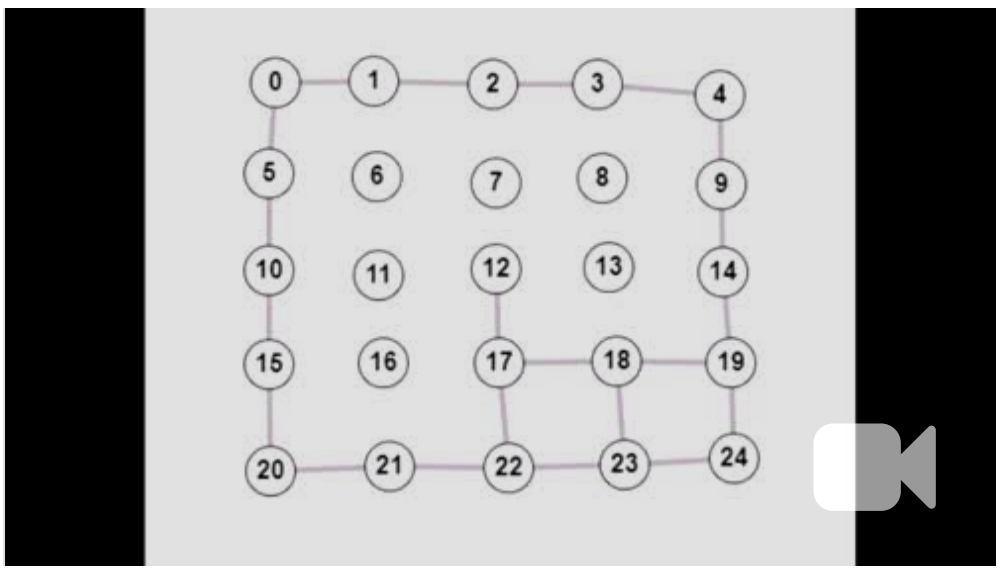
- ~~Graphs & trees~~
- ~~The maze!~~
- ~~Searching in graphs~~
  - Depth-first search
  - Breadth-first search

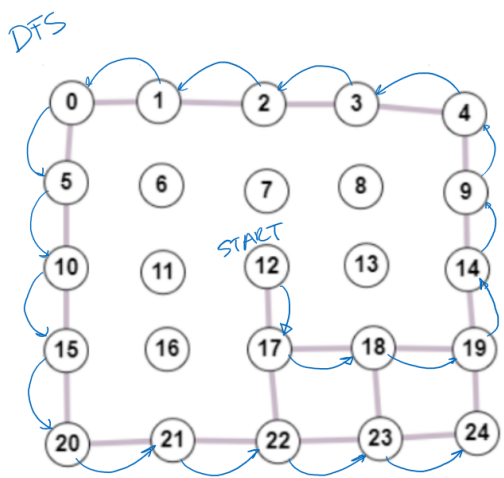
Depth First Search

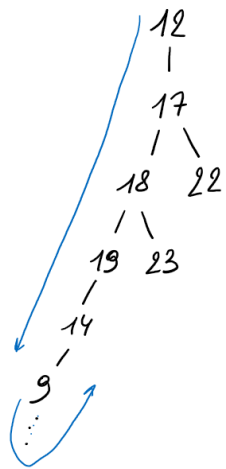


DFS in maze

- Deep search
- Visit all nodes

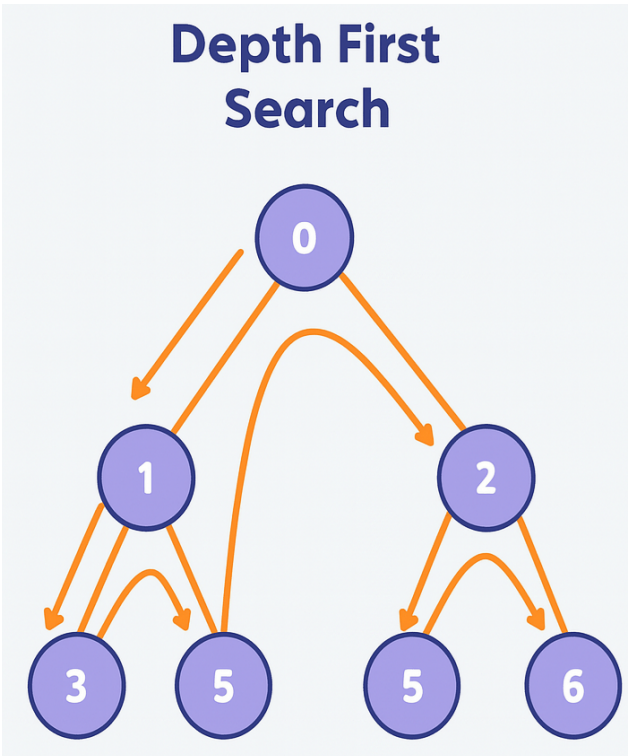






DFS pseudo code

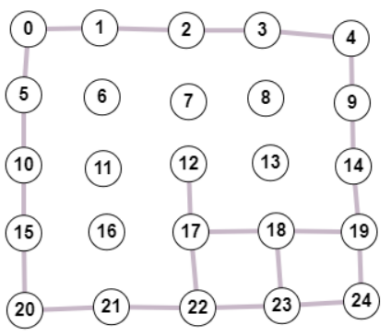
- **Start** with a node!
- **Store** the nodes you have **already visited**!
  - *you don't want to end up in an infinite loop!*
- **As long as there are (connected) nodes to visit**:
  - **check children** of node
    - child already visited? do nothing
    - child not visited yet? **visit**! (recursive call or push to stack)
- **Implement with stack or recursion**

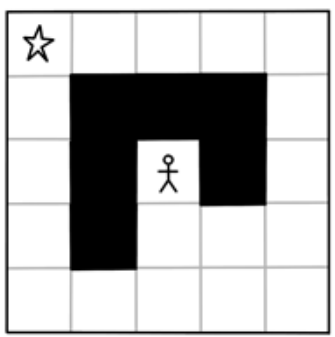


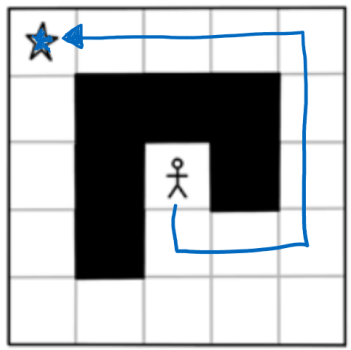
Recursive DFS

- An array of booleans for visited nodes: index in array = nr of node!
- Recursion: call ourselves with a not visited node!
- But now we are visiting all nodes... where do we need to stop?

  What is our base case?



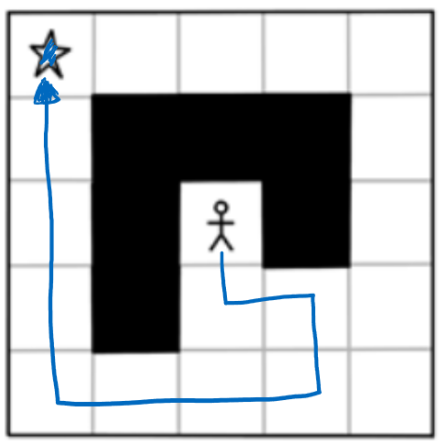




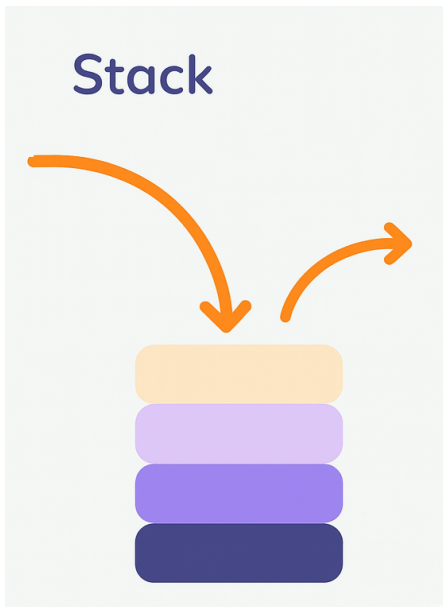
```
next->12
next->17
next->18
next->19
next->14
next->9
next->4
next->3
next->2
next->1
next->0
next->5
next->10
next->15
next->20
next->21
next->22
next->23
next->24
```

DFS with stack!

- Try it with a stack!
- As long as there are nodes on the stack: keep on popping!
- **Why do we end up with a different route?**



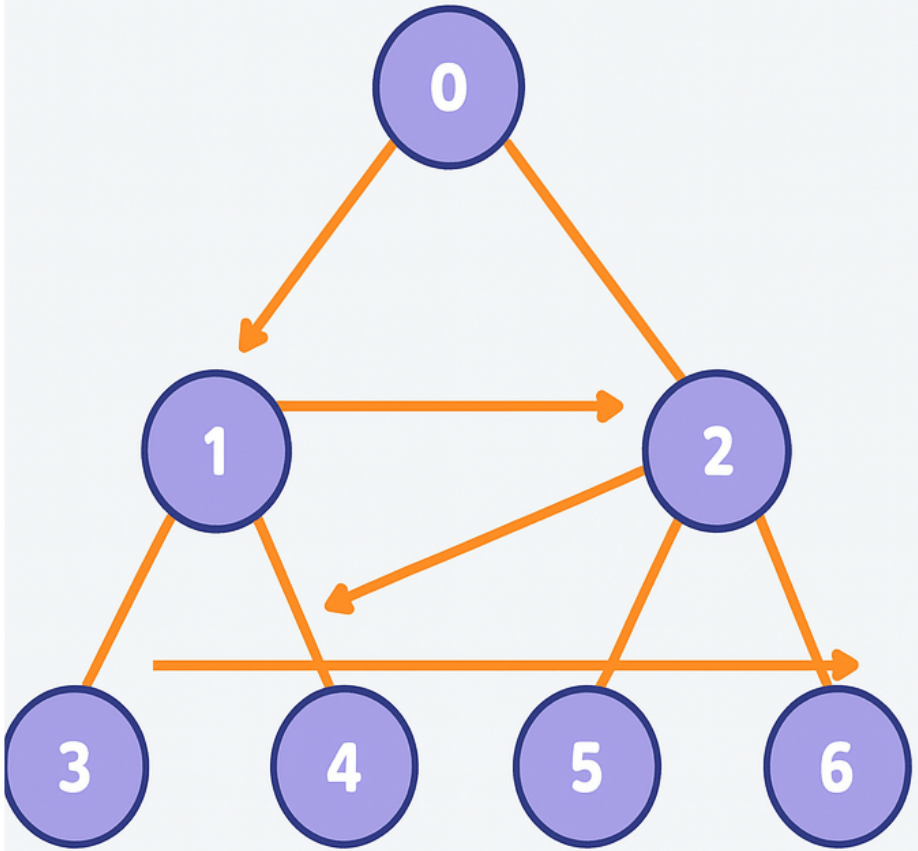
```
next->12
next->17
next->18
next->23
next->22
next->21
next->20
next->15
next->10
next->5
next->0
```



08 Graphs

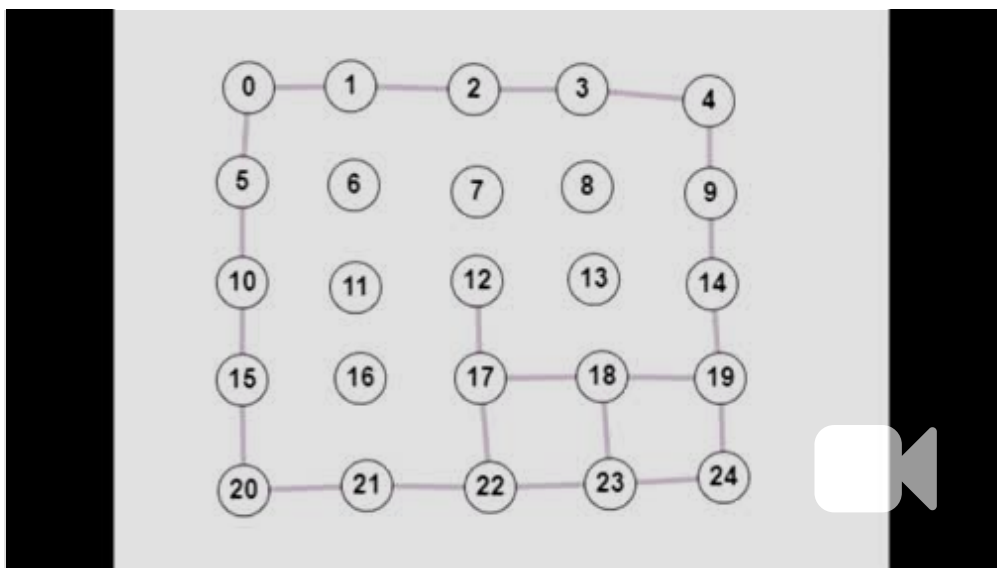
- ~~Graphs & trees~~
- ~~The maze!~~
- ~~Searching in graphs~~
  - ~~Depth-first search~~
  - Breadth-first search

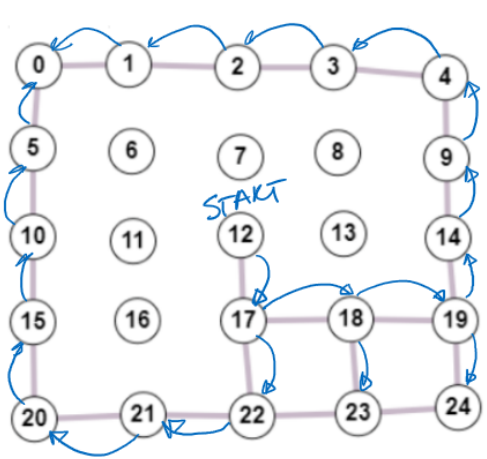
Breadth First Search

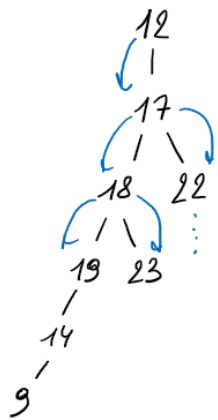


BFS in maze

- Wide search
- Search all nodes!

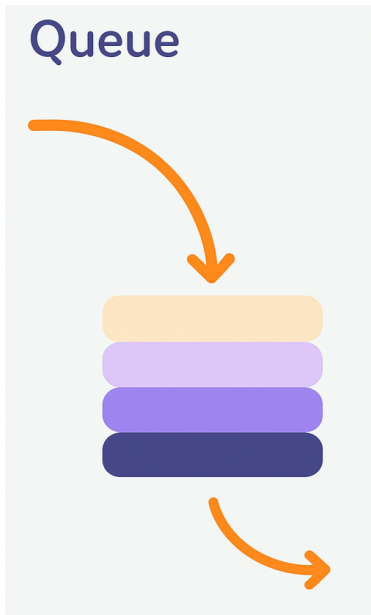






BFS pseudo code

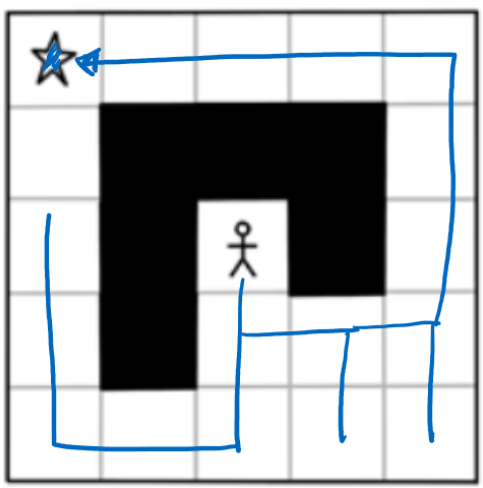
- Instead of always going deeper... we go wider, very similar to DFS! We don't use recursion or a stack, but we use a queue (FIFO: what comes in first, is viewed first)
- Start with a node: enqueue node!
  - Store the nodes you have already visited!
- As long as the queue is not empty: dequeue node
  - check children of node
    - child not visited yet? visit! (enqueue!)
    - child already visited? do nothing



BFS

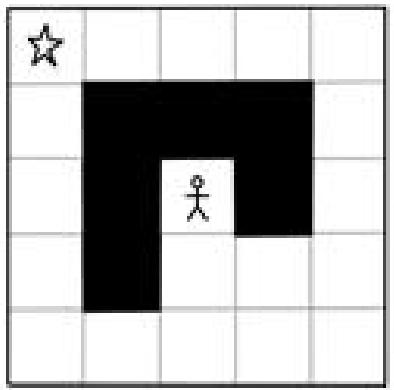
- Searching wide --> checking every path
- Stop if next child is 0!

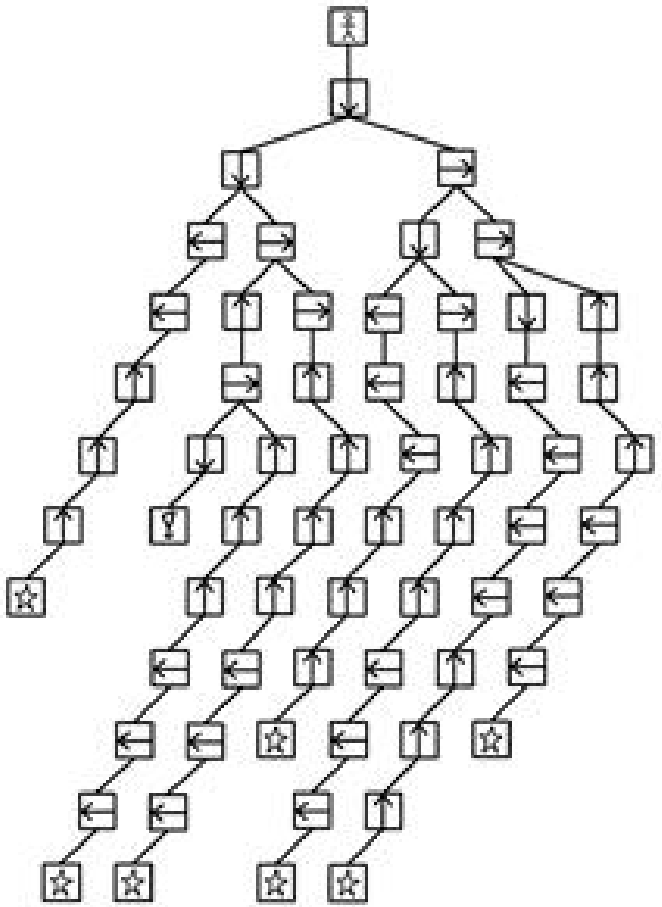
```
next->12
next->17
next->18
next->19
next->23
next->14
next->24
next->22
next->9
next->21
next->4
next->20
next->3
next->15
next->2
next->10
next->1
```



Find all paths

“ DFS & BFS are not only used to search for a path but also to find all possible paths!





- *DFS or BFS? It all depends on context! Depth, nr of connections, and your specific problem matter.*
- *BFS → Better when target is close to source (Finding neighbors, friends of friends)*
- *DFS → Better when target is far from source (Checking if graph is fully connected)*
- *Graph algorithms are among the most useful algorithms available.*

How to Organize Data

- **Purpose**: Choosing the right way to store and access information
- **Why it matters**: The way you organize data determines what you can do with it efficiently. Wrong choice = slow program.
- **Real-world impact:** How databases store millions of records, how social networks map connections, how file systems organize your computer.

Weighted graphs

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Weighted graphs

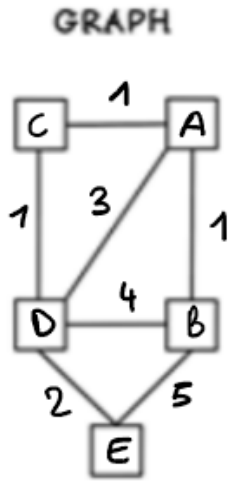
- Weighted graphs
- What is a heuristic?
- Finding the shortest path with Dijkstra!
- The maze again!

Weighted graph

- A graph where each **edge** has a numerical value = **weight**
- Each **connection** has a "**cost**" or "value"
- Shown as numbers on edges
- Stored in adjacency matrices/lists with weights
- **Types**:
  - **Positive weights**: Most common (distances, costs, time)
  - **Negative weights**: Less common but useful (profits, elevation changes)
  - **Zero weights**: Essentially free connections

Path in a weighted graph

- **Several ways** to get from point A to E, each with different cost!
- What is the **best path** from A to E?
  - The **shortest** or the **cheapest**?
  - Or a combination of those two?



BEST PATH FROM A TO E?

A B E = 1 + 5 = 6

A D E = 3 + 2 = 5

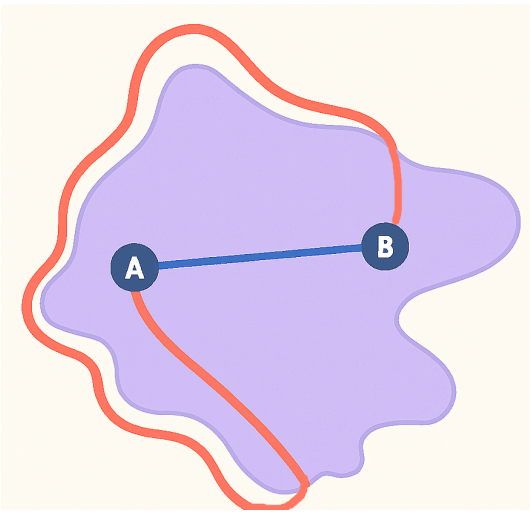
A C D E = 1 + 1 + 2 = 4

A D B E = 3 + 4 + 5 = 12

Why do we need weighted graphs?

Graphs

- **Problems** regular graphs
  - **All-or-nothing problem**: only tell us IF a connection exists, not **HOW GOOD** it is
  - **No optimization**: You can find A path, but not necessarily the **BEST path**
  - **Unrealistic modeling**: Real world has costs, distances, and preferences
  - **Poor decision making**: Without weights, you might choose a terrible path just because it exists



Why do we need weighted graphs?

- **Real-world accuracy**
  - **P**aths are longer, more expensive, or take more time
- **Optimization problems**
  - We want the "best" path, not just "any" path
- **Decision making**
  - Weights help us make informed choices
- **Resource management**:
  - Understanding costs helps allocate resources efficiently
- **Complex relationships**:
  - Some connections are stronger/weaker than others

Weighted graphs in real life

- **GPS Navigation**: edges = roads, weights = distance or travel time
  - Finding shortest route from A to B considering traffic
- **Social Networks**: edges = friendships, weights = interaction frequency
  - Recommending friends based on mutual connections strength
- **Internet Routing**: edges = network connections, weights = bandwidth or latency
  - Finding fastest data transmission paths
- **Flight Networks**:edges = flight routes, weights = cost, time, or distance
  - Finding cheapest or fastest flights with connections
- **...**

Weighted graphs

- ~~Weighted graphs~~
- What is a heuristic?
- Finding the shortest path with Dijkstra!
- The maze again!

Archimedes shouted "Eureka!" when he realized while taking a bath that he could measure irregular objects by how much water they displace. He could now test if the king's crown was pure gold or mixed with cheaper silver by comparing water displacement!

What is a heuristic?

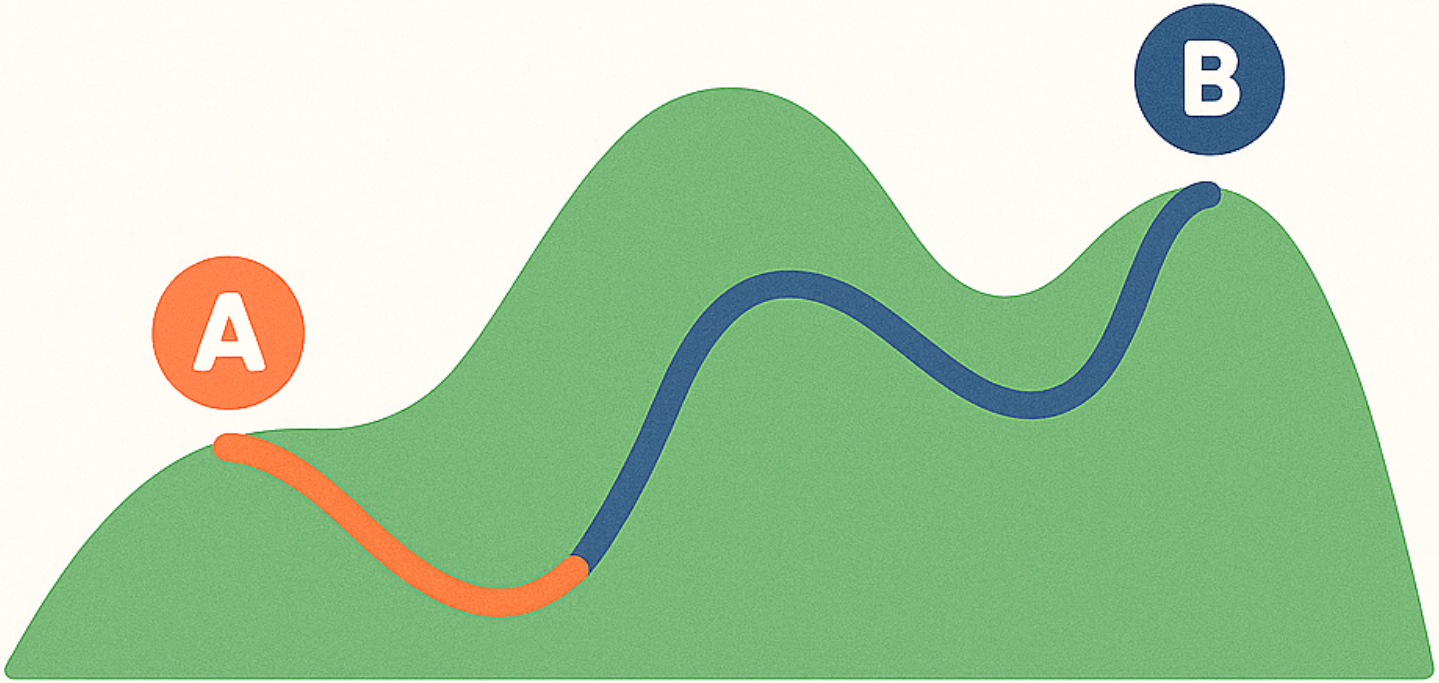
- **Heuristics = art of finding**
  - *Greek: heuriskein = find, heurèka = I found it!*
- A **heuristic** is "**rule of thumb**"
- **Educated guess** that helps solve problems faster
- Heuristics **don't guarantee the best solution**, but they're usually "good enough"
- **Speed vs Accuracy** trade-off
  - Fast approximate solutions vs slow perfect solutions
- **Human-like thinking**: how we make quick decisions with limited information

Why do we need heuristics in graphs?

- **Computational complexity**: Finding the absolute best path can be extremely slow for large graphs
- **Real-time constraints**: GPS navigation needs answers in seconds, not hours
- **Good enough solutions**: Sometimes 95% optimal is better than waiting for 100% optimal
- **Practical limitations**: Memory and processing power aren't infinite

Heuristics in graph problems

- **Shortest Queue** Heuristic: "Pick the Line That Looks Fastest"
  - **Real-life**: Network routing path least traffic, picking the shortest line at market
  - **Problem**: Doesn't account for different processing speeds of each "lane"
- **Frequency-Based** Heuristic: "Popular Routes Are Usually Good"
  - **Real-life**: Social media "People you may know", following most-used hiking trail
  - **Problem**: Popular doesn't always mean optimal for your specific situation
- **Avoid Congestion** Heuristic: "Stay Away from Busy Areas"
  - **Real-life**: Taking back roads to avoid highway traffic, shopping during off-peak hours, choosing less popular tourist destinations
  - **Problem**: "Busy" areas might be faster or better despite the crowds



Hill climbing: Heuristic finds a 'pretty good' solution but misses the 'best' solution

Problems with simple heuristics

- **The Local Optimum Problem**
  - **Hill climbing**: Heuristic finds a "pretty good" solution but misses the "best" solution
  - **No backtracking**: Once you make a choice, you can't undo it
  - **Tunnel vision**: Focusing on immediate gains instead of long-term optimal path
- **The Information Problem**
  - **Incomplete data**: Heuristics work with limited information
  - **Changing conditions**: Traffic, weather, or network conditions change constantly
  - **Context missing**: A route might be short but dangerous, crowded, or expensive

Simple heuristics aren't enough!

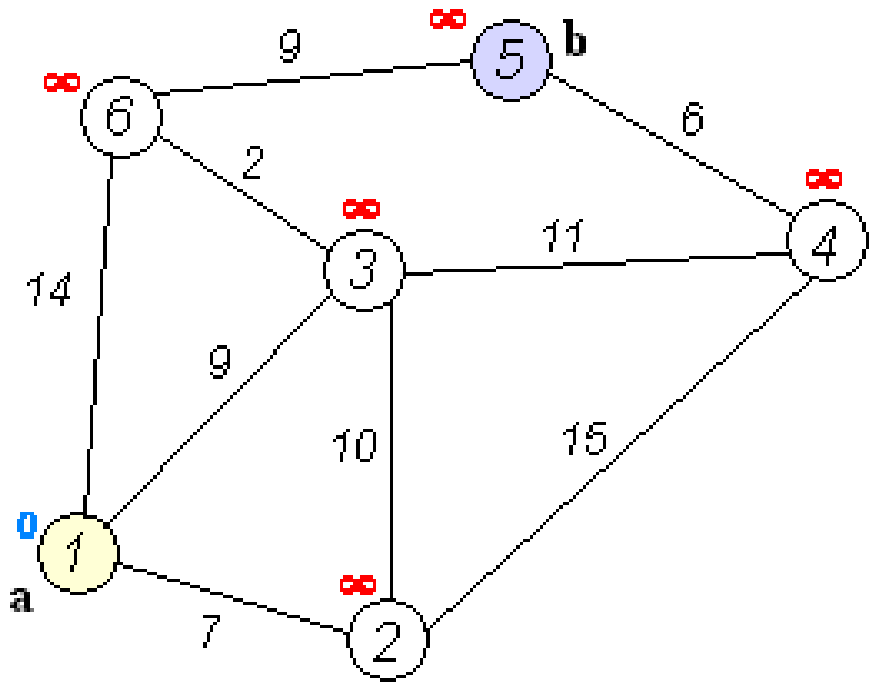
- What's is good about simple heuristics:
  - **Speed matters**: Solutions need to be fast enough for real-world use
  - **Good enough is often good enough**: Perfect isn't always necessary
  - **Local information is valuable**: Immediate neighbors matter most
- Why we need a better algorithm?
  - **Global perspective**: Considers the entire graph, not just local choices
  - **Systematic approach**: Methodical exploration prevents missing optimal paths
  - **Proof** of **correctness**: Mathematical guarantee of finding the best solution

Weighted graphs

- ~~Weighted graphs~~
- ~~What is a heuristic?~~
- Finding the shortest path with Dijkstra!
- The maze again!

Dijkstra

- Dijkstra's algorithm is an algorithm for finding the **shortest path between nodes in a weighted graph**
- It was created by computer scientist Edsger W. Dijkstra in 1956!
- It's all about finding the cheapest node to find a path!



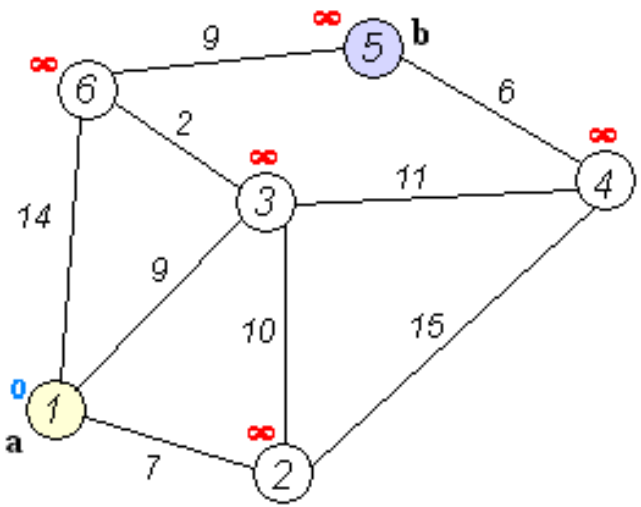
Dijkstra about Dijkstra's algorithm

Edsger Wybe Dijkstra, Rotterdam, 11 mei 1930 – Nuenen, 6 augustus 2002)

“ ***What’s the shortest way to travel from Rotterdam to Groningen?*** *It is the algorithm for the shortest path, which I designed in about 20 minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a 20-minute invention. In fact, it was published in 1959, three years later. The publication is still quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper.* ***Without pencil and paper you are almost forced to avoid all avoidable complexities****. Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame. — As quoted in the article Edsger W. Dijkstra from An interview with Edsger W. Dijkstra.*

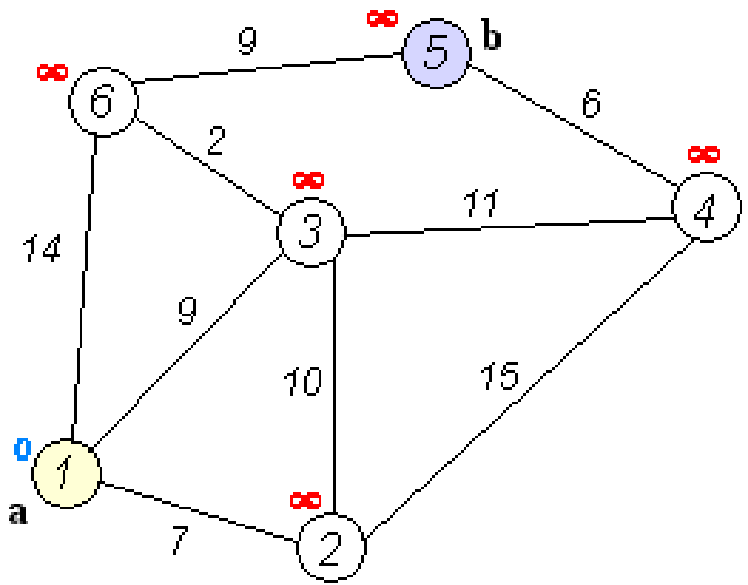
Basics of Dijkstra's algorithm

- **Start at a node (the start node)**
- Analyze the graph to **find** the **shortest** path between that **node** and all other **connected nodes**!
- **Store** the currently **known shortest distance** from each node to the source node and **update values** if you find a shorter path!
- Once the algorithm has **found** the **shortest path** between the source node and another node, that node is **marked** as "**visited**".
- The **process continues until all the nodes in the graph have been visited!** This way, we have a path that connects the source node to all other nodes following the shortest path possible to reach each node.



Basics of Dijkstra's algorithm

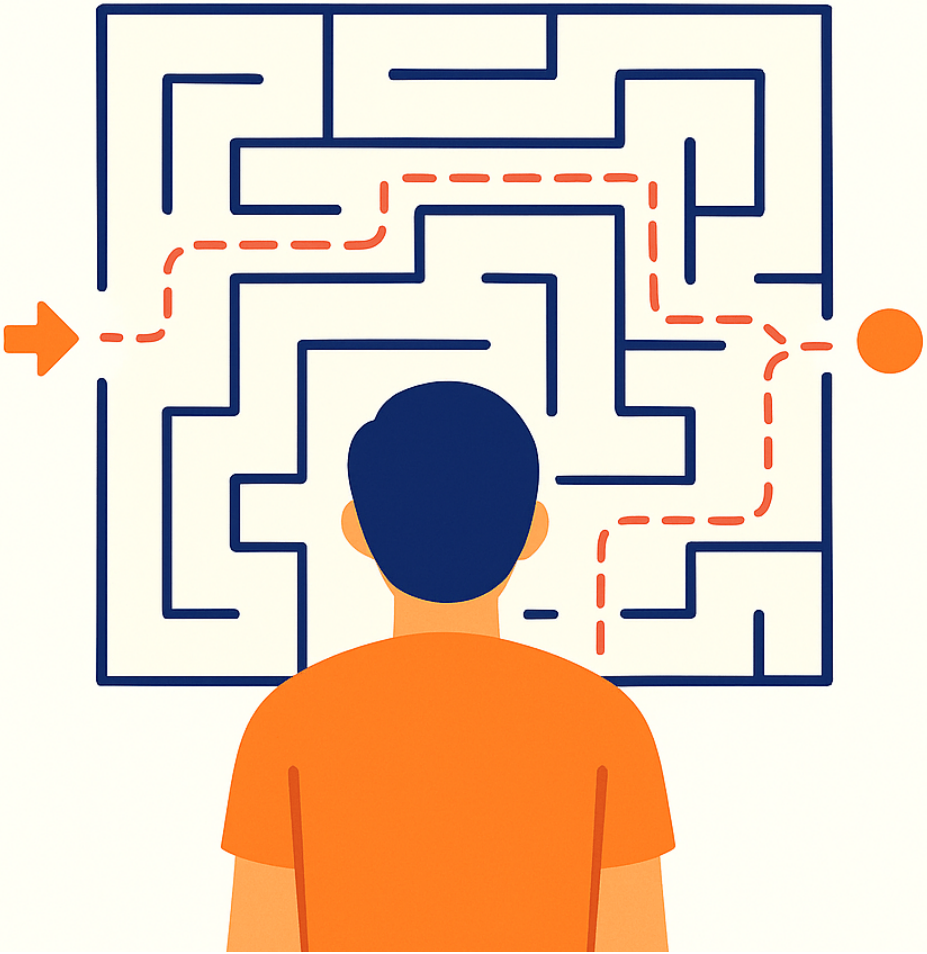
- Initialise distance array (on infinity = Int32.Max) and visited array (on false)
- Start at 1 (visited 1 = true)
  - distance to 2 = **7**: store in distance array (< infinity)
  - distance to 3 = **9**: store in distance array (< infinity)
  - distance to 6 = **14** - store in distance array (< infinity)
- Go to 2 (shortest path so far) (visited 2 = true)
  - only check for not visited neighbours!
  - distance to 3 = 7 + 10 = 17 > 9: 9 remains in distance array
  - distance to 4 = **22**: store in distance array (< infinity)
- Go to 3 (shortest path so far) (visited 3 = true)
  - only check for not visited neighbors!
  - distance to 6 = 9 + 2 = **11** : store in distance array (< infinity)
  - distance to 4 = 9 + 11 = **20:** store in distance array (< 22)
- ....



Weighted graphs

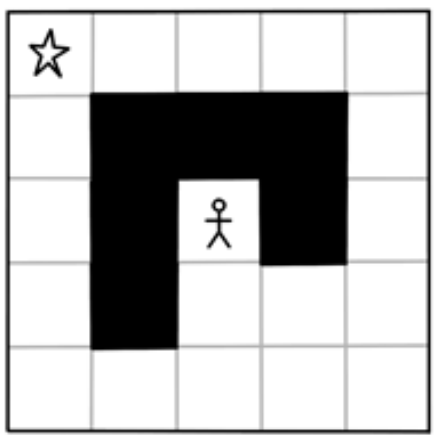
- ~~Weighted graphs~~
- ~~What is a heuristic?~~
- ~~Finding the shortest path with Dijkstra!~~
- The maze again!

Navigating a Maze

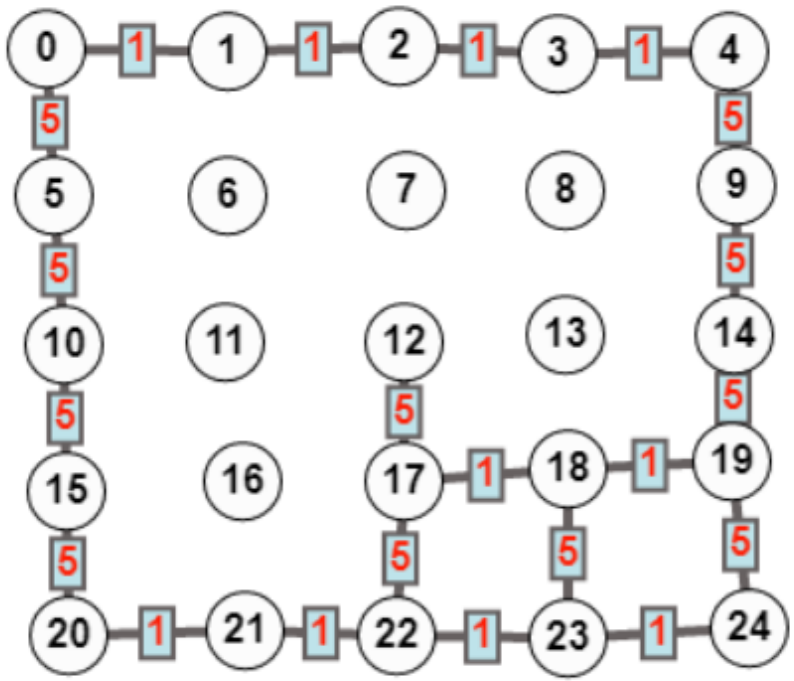


The maze!

- Suppose the cost in the maze:
  - movement top-bottom = 5
  - movement left-right = 1

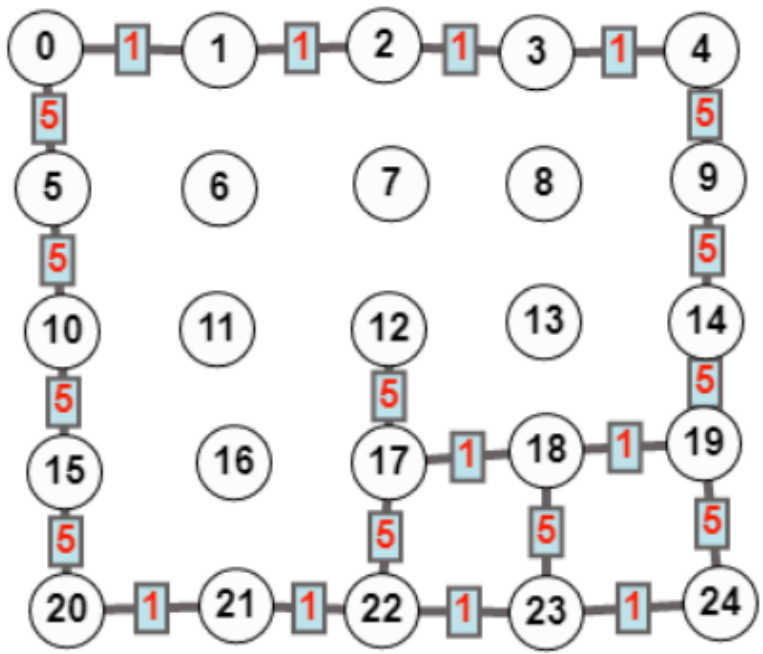


	COST
↑	5
↓	5
←	1
→	1

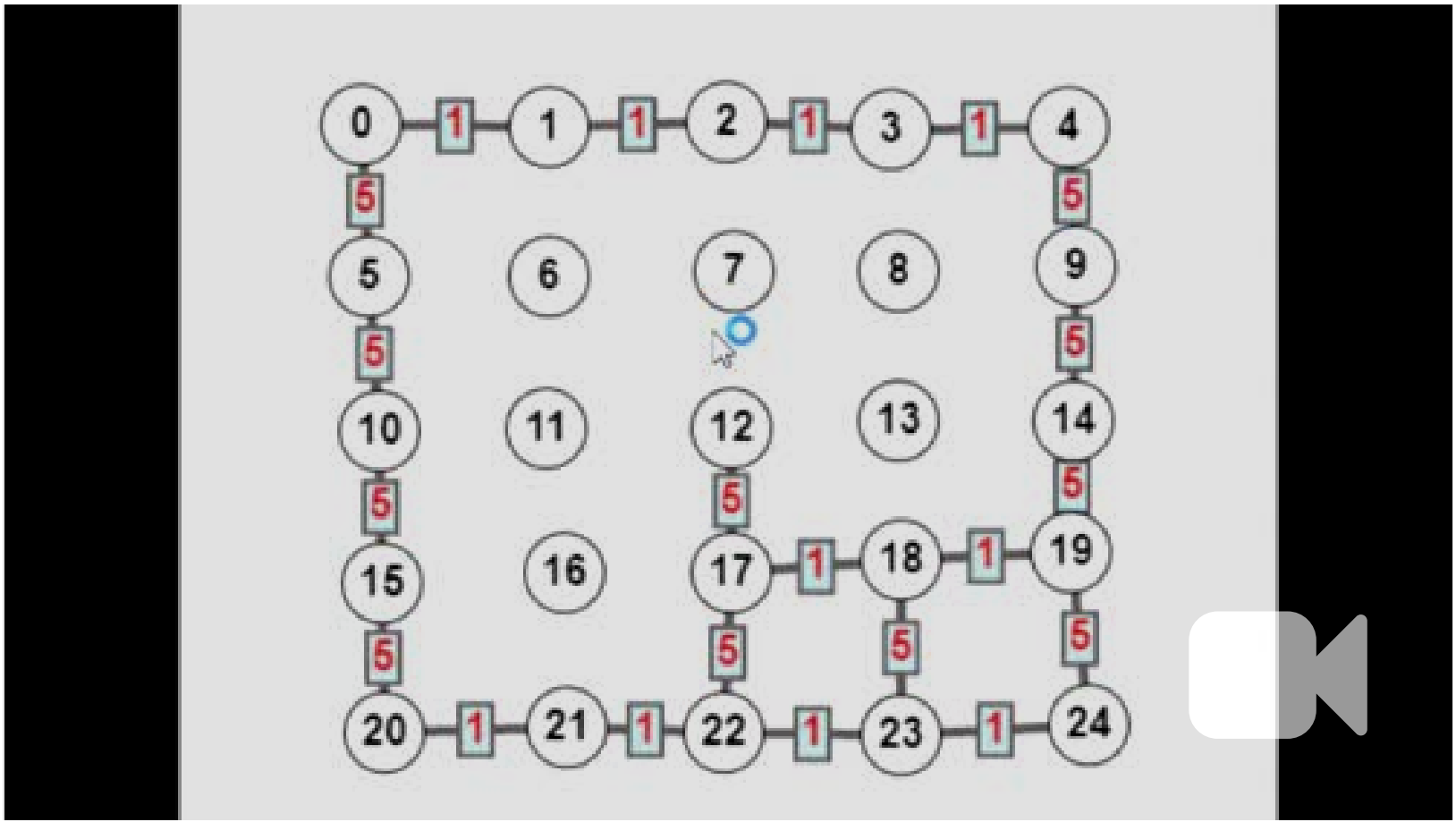


The shortest/cheapest path

- What is our cost from 12 to ...
  - 12
    - 17 -> **5**
  - 17
    - **18 -> 5+1 = 6 lowest distance**
    - 22 -> 5 + 5 = 10
  - 18
    - 17 -> is already visited, so ignore
    - **19 -> 6 + 1 = 7 lowest distance**
    - 23 -> 6 + 5 = 11
  - 19
    - etc...

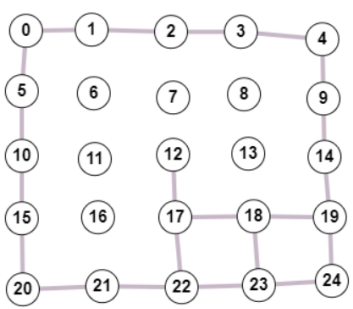


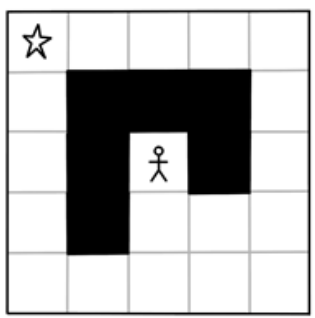
Dijkstra in the maze



Translate maze to data structure!

- Start from previous Maze (BFS & DFS)
- An adjacency list = **array of lists**
  - array = fixed number of nodes
  - per node = list of 'connected neighbors'
  - **initialize all lists per node**
- If one node is connected, it's also the other way around!

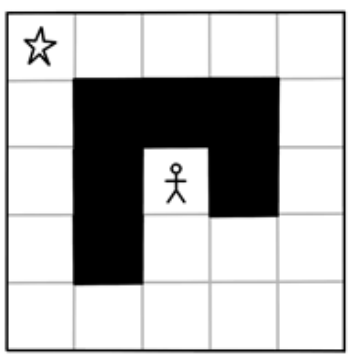




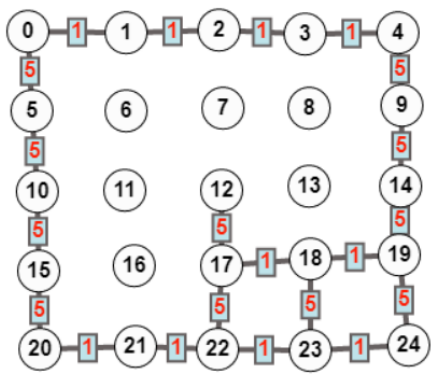
```
1  class Program
2  {
3    static void Main(string[] args)
4    {
5       Maze maze = new Maze(25);
6
7       maze.AddEdge(0, 1);
8       maze.AddEdge(1, 2);
9       maze.AddEdge(2, 3);
10      maze.AddEdge(3, 4);
11      maze.AddEdge(4, 9);
12      maze.AddEdge(9, 14);
13      maze.AddEdge(18, 17);
14      maze.AddEdge(18, 19);
15      maze.AddEdge(19, 14);
16      maze.AddEdge(19, 24);
17      maze.AddEdge(24, 23);
18      maze.AddEdge(18, 23);
19      maze.AddEdge(22, 23);
20      maze.AddEdge(17, 12);
21      maze.AddEdge(21, 22);
22      maze.AddEdge(21, 20);
23      maze.AddEdge(20, 15);
24      maze.AddEdge(15, 10);
25      maze.AddEdge(10, 5);
26      maze.AddEdge(5, 0);
27
28      Console.WriteLine(maze.ToString());
29 }
```

Translate maze to data structure!

- What about the costs on every edge?
  - cost between 17 & 18 = 1 --> absolute value of (17-18)
  - cost between 20 & 15 = 5 --> absolute value of (20-15)
  - cost between 9 & 14 = 5 --> absolute value of (9-14)
  - etc...
- Yes, it's that easy

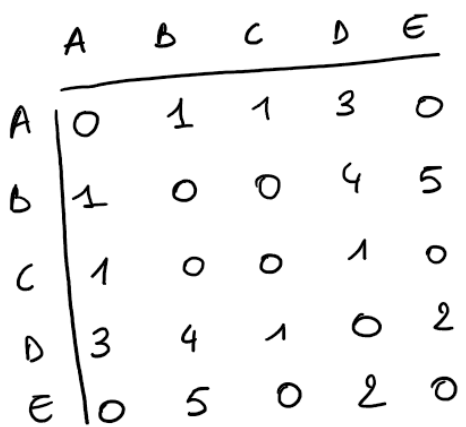


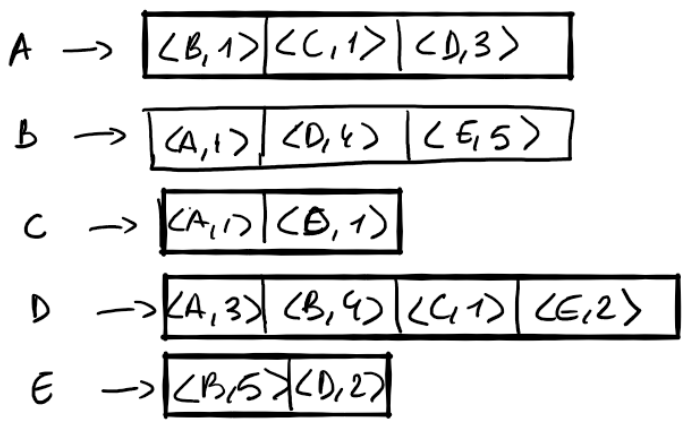
	COST
↑	5
↓	5
←	1
→	1



What if the cost isn't that easy?

- Again use an adjacency matrix or list!
  - adjacency matrix = 2d array --> easy to put weights in!
  - adjacency list = array of lists --> use <key,value>-pairs!





Ok, let's code!

- We will have to use a distances-array and a visited-array (same length as nodes)
  - initialize everything on Int32.Max & false
- Start with start-node!
- While true 😨 (relax, we can take a break!)
  - get next node (the shortest distance of a node that is not already visited)
    - if node is -1 break! so there is no next node!
  - for each neighbor
    - calculate the cost and update if lower
- Print out distance array

Ok, let's code!

- Ok, so the shortest path to 0 has a cost of 26...
- But uh-oh... where is our path?
  - How can you solve this?
  - The same way as with the shortest distance!
  - If you have your shortest distance, you also have the predecessor of that node, once your store your predecessor you get your path!

```
node: 0 --> cost: 26
node: 1 --> cost: 25
node: 2 --> cost: 24
node: 3 --> cost: 23
node: 4 --> cost: 22
node: 5 --> cost: 29
node: 6 --> cost: 2147483647
node: 7 --> cost: 2147483647
node: 8 --> cost: 18
node: 9 --> cost: 17
node: 10 --> cost: 24
node: 11 --> cost: 2147483647
node: 12 --> cost: 0
node: 13 --> cost: 2147483647
node: 14 --> cost: 12
node: 15 --> cost: 19
node: 16 --> cost: 2147483647
node: 17 --> cost: 5
node: 18 --> cost: 6
node: 19 --> cost: 7
node: 20 --> cost: 14
node: 21 --> cost: 13
node: 22 --> cost: 12
node: 23 --> cost: 11
node: 24 --> cost: 12
```

Ok, let's code!

- Follow the predecessors!
- From 0 to 1 to 2 to 3 ... all the way up to 12!

- *Try to code it yourself!*

```
node: 0 --> cost: 26 with predecessor 1
node: 1 --> cost: 25 with predecessor 2
node: 2 --> cost: 24 with predecessor 3
node: 3 --> cost: 23 with predecessor 4
node: 4 --> cost: 22 with predecessor 9
node: 5 --> cost: 29 with predecessor 10
node: 6 --> cost: 2147483647 with predecessor -1
node: 7 --> cost: 2147483647 with predecessor -1
node: 8 --> cost: 18 with predecessor 9
node: 9 --> cost: 17 with predecessor 14
node: 10 --> cost: 24 with predecessor 15
node: 11 --> cost: 2147483647 with predecessor -1
node: 12 --> cost: 0 with predecessor -1
node: 13 --> cost: 2147483647 with predecessor -1
node: 14 --> cost: 12 with predecessor 19
node: 15 --> cost: 19 with predecessor 20
node: 16 --> cost: 2147483647 with predecessor -1
node: 17 --> cost: 5 with predecessor 12
node: 18 --> cost: 6 with predecessor 17
node: 19 --> cost: 7 with predecessor 18
node: 20 --> cost: 14 with predecessor 21
node: 21 --> cost: 13 with predecessor 22
node: 22 --> cost: 12 with predecessor 23
node: 23 --> cost: 11 with predecessor 18
node: 24 --> cost: 12 with predecessor 19

Shortest path Dijkstra: 12 17 18 19 14 9 4 3 2 1 0
```

- *If you have a **problem like 'find path'**: model your problem as a **graph**!*
- *A graph **consists of nodes and edges***
- *Searching for the shortest path in an **unweighted graph** is done by **BFS** or **DFS**!*
- *If we are searching in a **weighted** graph, we use **Dijkstra**!*

How to Organize Data

- **Purpose**: Choosing the right way to store and access information
- **Why it matters**: The way you organize data determines what you can do with it efficiently. Wrong choice = slow program.
- **Real-world impact:** How databases store millions of records, how social networks map connections, how file systems organize your computer.

Greedy

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Greedy algorithms

- What is greedy?
- Optimal Substructure
- Activity selection problem
- Coin Change Problem

What is greedy?

- Greedy is an **algorithmic technique**
- Build a solution piece by piece, choosing the next piece with **most immediate**
- Greedy algorithms **optimize locally,** hoping to end up with a **global optimum**.
- Key characteristics:
  - Once a **choice** is made, it's **never changed**
  - Only **consider** the **current step**, not the big picture
  - **Incremental construction** - Builds the solution one element at a time
- **Not guaranteed best solution**, but good is good enough
- *Note: greedy algorithms like Dijkstra's shortest path algorithm do guarantee optimal solutions = optimal greedy algorithms*

Why would you go greedy?

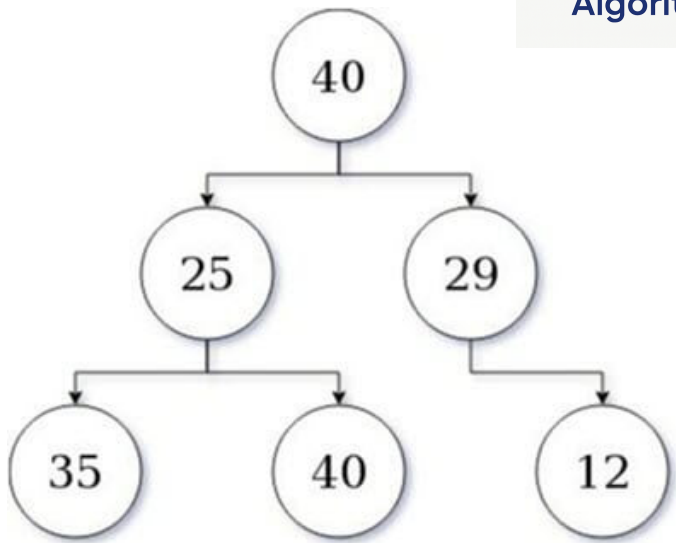
- **Efficiency:** much faster than other approaches
  - time complexity is typically O(n log n) or better
- **Simplicity:** easy to implement and understand
- **Good enough solutions:** even when not optimal, acceptable approximations that are sufficient in practice
- **Real-time constraints:** make quick decisions
- **Memory efficiency:** use less memory, not store all possible solutions

Greedy in real life

- **Positive algorithmic examples**
  - Dijkstra's algorithm - Shortest path
  - Huffman coding - Data compression
  - Kruskal's/Prim's algorithms - Minimum spanning trees
  - Activity selection - Optimal meeting scheduling
  - Fractional knapsack - Resource allocation
- **Problematic real-world greedy behavior:**
  - Traffic routing - Everyone choosing the fastest route leads to congestion
  - Resource allocation - Companies optimizing locally can disrupt entire markets
  - Investment decisions - Short-term gains can undermine long-term stability
  - CPU scheduling - Shortest job first can starve longer processes

When Greedy fails

- Not suitable for every problem, like problems where you need to have a solution for each subproblem (sorting)
- You don't know if you have the optimal solution because you don't know what lies ahead at the current greedy state!
  - *find the biggest sum*
    greedy path: 40 + 29 + 12 = 81
    optimal solution: 40 + 25 + 40 = 105



We are greedy!

- Much **human behavior** is inherently **greedy**
  - Humans make decisions based on immediate benefits without considering all future consequences
- Greedy algorithms feel intuitive

- *Recognize when a greedy approach is suitable for a specific problem versus when you need more sophisticated techniques!*

Basics steps of a Greedy algorithm

- If **x gives a local optimal solution**, it is included in the partial solution set (else it is discarded)
- **Go** to **next x** and continue until the input set is finished or end is reached!

```
1  Algorithm Greedy(a, n)   //n defines the input set
2  {
3      solution= NULL;     //initialize solution set
4      for i=1 to n do
5      {
6          x = Select(a);  //selection Function
7          if Feasible(solution, x) then
8              solution = Union (solution, x);  //include x in the solution set
9      }
10     return solution;
11 }
```

Greedy algorithms

- ~~What is greedy?~~
- Optimal Substructure
- Activity Selection Problem
- Coin Change Problem

What is an Optimal Substructure?

- **Optimal Substructure = foundation of Greedy** (and DP, remember?)
- An optimal solution contains **optimal solutions** to its **subproblems**
  - **Simple explanation:** If you have the best solution for a big problem, then every smaller step must also be the best for that smaller problem
  - **Building blocks analogy:** Building the tallest tower? Every section must also be optimal. If any section could be taller → replace it for an even taller tower!
- Without optimal substructure, you can't build optimal solutions step by step
- Both DP and Greedy algorithms depend on this property

Why does it matter for Greedy?

- **How Greedy Works:**
  - Make the best local choice at each step
  - Solve the remaining subproblem
  - Repeat until done
- **Requirements for Greedy Success:**
  - Local choice must be part of some optimal solution
  - Remaining subproblem has the same structure
  - Optimal solution = Best local choice + Optimal subproblem solution
- **With optimal substruture :** Can solve subproblems independently
- **Without optimal substructure:** Past choices affect future possibilities and Greedy fails because it can't handle these dependencies

Test for Optimal Substructure

1. **Take** any optimal solution to your problem
2. **Split** it into: first choice + remaining subproblem
3. **Ask:** If I had a better solution for the subproblem, would I get a better overall solution?
4. **Yes?** → Has optimal substructure ✅
5. **No?** → Does NOT have optimal substructure ❌

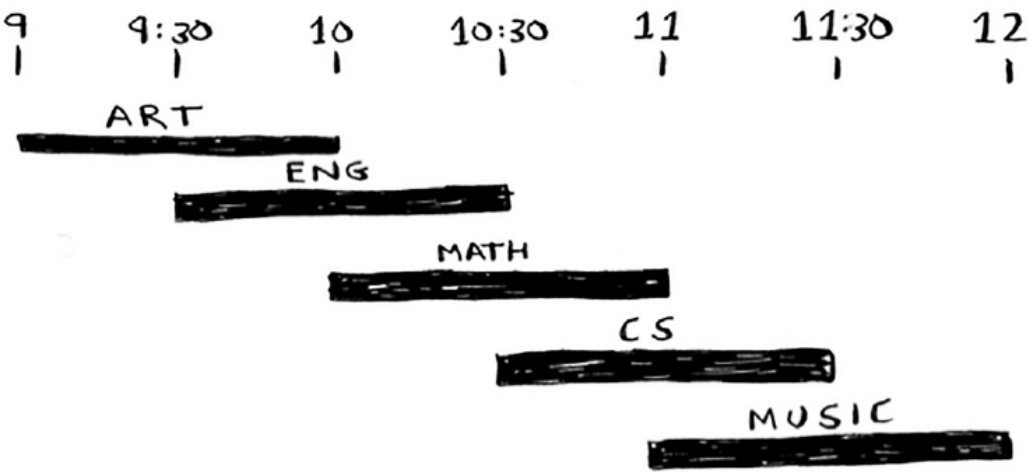
- **Safe indicators**
  - Each choice is independent
  - Remaining problem has same structure
  - No global constraints to track

Greedy algorithms

- ~~What is greedy?~~
- ~~Optimal Substructure~~
- Activity Selection Problem
- Coin Change Problem

Activity selection problem

Suppose you have a classroom and want to hold as many classes here as possible. You can't hold all of these classes in there, because some of them overlap.



CLASS	START	END
ART	9 AM	10 AM
ENG	9:30 AM	10:30 AM
MATH	10 AM	11 AM
CS	10:30 AM	11:30 AM
MUSIC	11 AM	12 PM

How to find the best schedule?

Sounds like a hard problem, but it actually isn't when we think greedy!

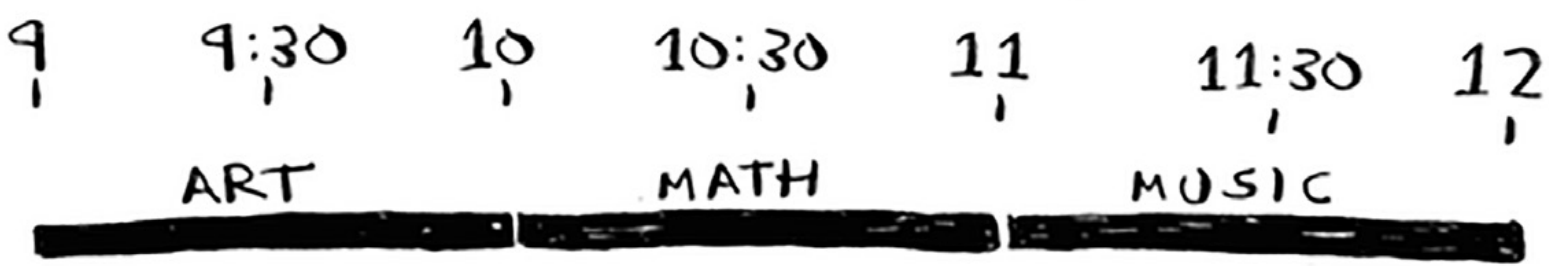
- Pick the class that ends the soonest. This is the first class you'll hold in this classroom.
- Now, you have to pick a class that starts after the first class. Again, pick the class that ends the soonest. This is the second class you'll hold.
- Keep on doing this and you will end up with the answer!

This is how we find the best schedule!

ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	
MATH	10 AM	11 AM	
CS	10:30 AM	11:30 AM	
MUSIC	11 AM	12 PM	

ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	X
MATH	10 AM	11 AM	✓
CS	10:30 AM	11:30 AM	
MUSIC	11 AM	12 PM	

ART	9 AM	10 AM	✓
ENG	9:30 AM	10:30 AM	X
MATH	10 AM	11 AM	✓
CS	10:30 AM	11:30 AM	X
MUSIC	11 AM	12 PM	✓



Why Greedy works

- Our optimal solution: [ART, MATH, MUSIC] = 3 classes, so **do the test**:
- **Split it up:** ART (9-10 AM) <> schedule classes from 10 AM → [MATH, MUSIC]
- **The question**: If I could magically fit MORE classes from 10 AM on, would my total get better? **YES!**
  - *Current: ART + 2 classes = 3 total*
  - *If 10 AM+ had 3 classes: ART + 3 classes = 4 total*
- Activity Selection **HAS optimal substructure**, because improving any part improves the whole! **Greedy works** here because **each choice doesn't mess up the remaining choices!**

The activity selection problem

- How to code this problem?
  - Sort the activities on finishing time (in ascending order)
  - Select the first activity (it ends the soonest due to the sorted finishing times)
  - Selection of new activity when starting time is greater or equal to the previously selected activity
  - Repeat until activities are checked!

```
1  string[] classes = { "art", "eng", "math", "cs", "music"};
2  double[] start = { 9,9.5,10,10.5,11};
3  double[] end = { 10,10.5,11,11.5,12 };
4
5 Scheduling activities = new Scheduling(classes, start, end);
6 activities.Schedule();
```

```
Following classes are selected: art math music
```

Greedy algorithms

- ~~Who is greedy?~~
- ~~Optimal Substructure~~
- ~~Activity selection problem~~
- Coin Change Problem

Coin Change Problem

- Given a set of coin denominations and a target amount, find the minimum number of coins needed to make that amount.

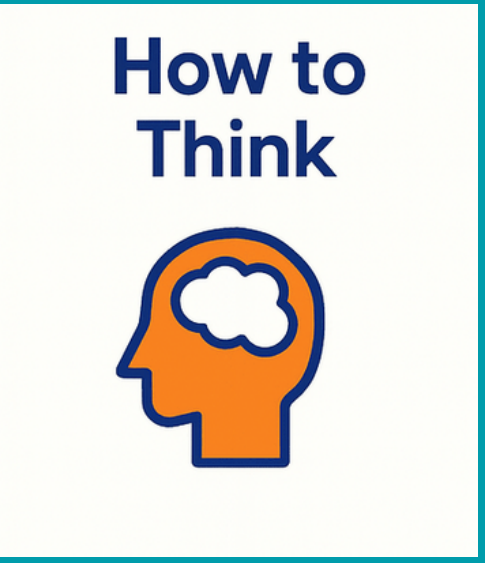
- **Example:**
  - Coins: [50, 20, 10, 5, 2, 1] (euro cents)
  - Target: 67 cents
  - Greedy approach: Take largest coin first
    - 67 → 50 + 10 + 5 + 2 = 4 coins

Extra coding exercises

Why Greedy works

- **Do the optimal substructure test**
  - *Optimal solution: 67 = [50, 10, 5, 2]*
  - *Split: 50 + solve remaining 17*
  - *Test: If 17 had better solution, overall would improve*
  - ***Conclusion: HAS optimal substructure***
- **Note**: only for "canonical" coin systems (Euro coins: [50, 20, 10, 5, 2, 1], US coins: [25, 10, 5, 1]) and f**ails for non-canonical systems:**
  - *Coins: [4, 3, 1] (hypothetical currency)*
  - *Target: 6*
  - *Optimal: 3 + 3 = 2 coins*
  - *Greedy: 4 + 1 + 1 = 3 coins*

- *Greedy algorithms build solutions step-by-step by always making the locally optimal choice at each step, hoping this leads to a globally optimal solution.*
- *They work efficiently (typically O(n log n)) for problems with optimal substructure*
- *If greedy choice is part of an optimal solution, greedy will work perfectly.*



- **Purpose**: Core problem-solving approaches and mental frameworks for computational challenges
- **Why it matters**: These are the fundamental thinking patterns that solve 90% of computational problems. Master these strategies, and you can tackle almost any coding challenge.
- **Real-world impact**: How Netflix recommends movies, how GPS finds optimal routes, how compression algorithms work, how game AI makes decisions.

Trees

Programming advanced

YP0822 - YP0847

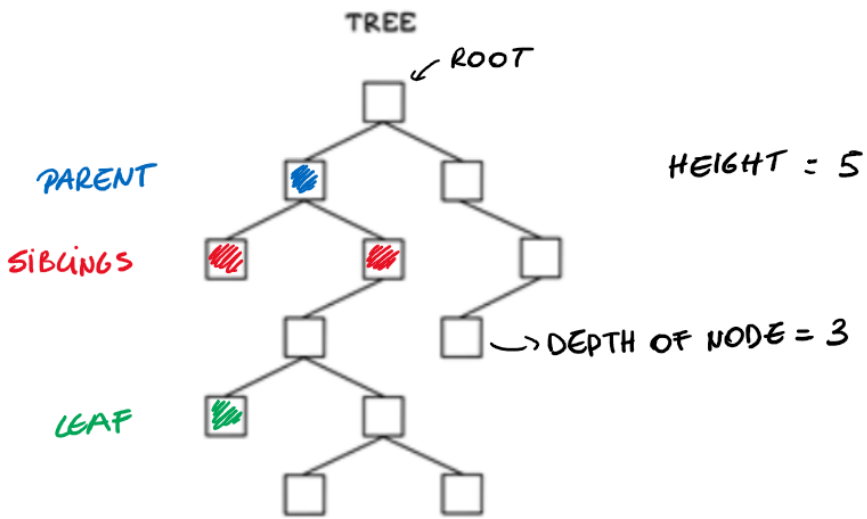
Elke Boonen & Rik Tonnelier

Trees

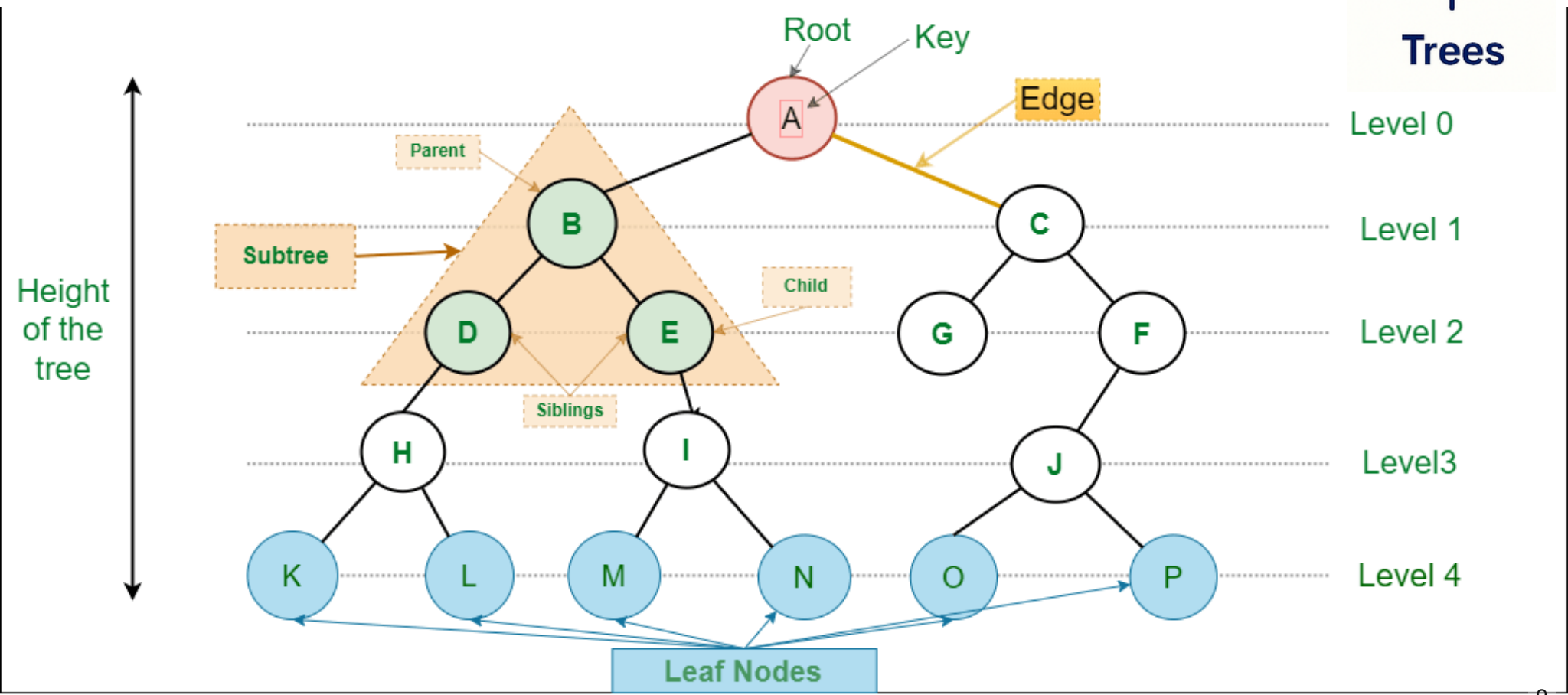
- Types & traversal
- Binary Search Tree
  - add
  - traverse
  - search
  - min/max

What is a tree?

- A **data structure** related to a **graph**
- Starts with the **root node (top)**
  - a node with children: **parent**
  - a node without children**: leaf**
  - nodes with the same parent: **siblings**
- Trees have a **height**
  - number of edges from root to deepest leaf
- The level of a node = **depth**
  - nr of edges from node to the root node.

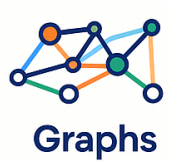


Trees


Trees
Root
Key
Edge
A
Parent
B
C
Subtree
Child
D
E
G
F
Siblings
H
I
J
K
L
M
N
O
P
Level 0
Level 1
Level 2
Level3
Level 4
Height of the tree
Leaf Nodes

Trees are graphs

- **Every tree is a graph, but not every graph is a tree.**
- Trees are a special type of connected, acyclic graph.



Cycles	No cycles allowed	Can have cycles
Paths	Exactly one path between any two nodes	Multiple paths possible
Root	Has a designated root node	No specific root (unless specified)
Hierarchy	Strict hierarchical structure	Can be hierarchical or non-hierarchical
Edges	n-1 edges for n nodes	Can have any number of edges
Direction	Usually directed (parent → child)	Can be directed or undirected
Connectivity	Always connected	Can be disconnected

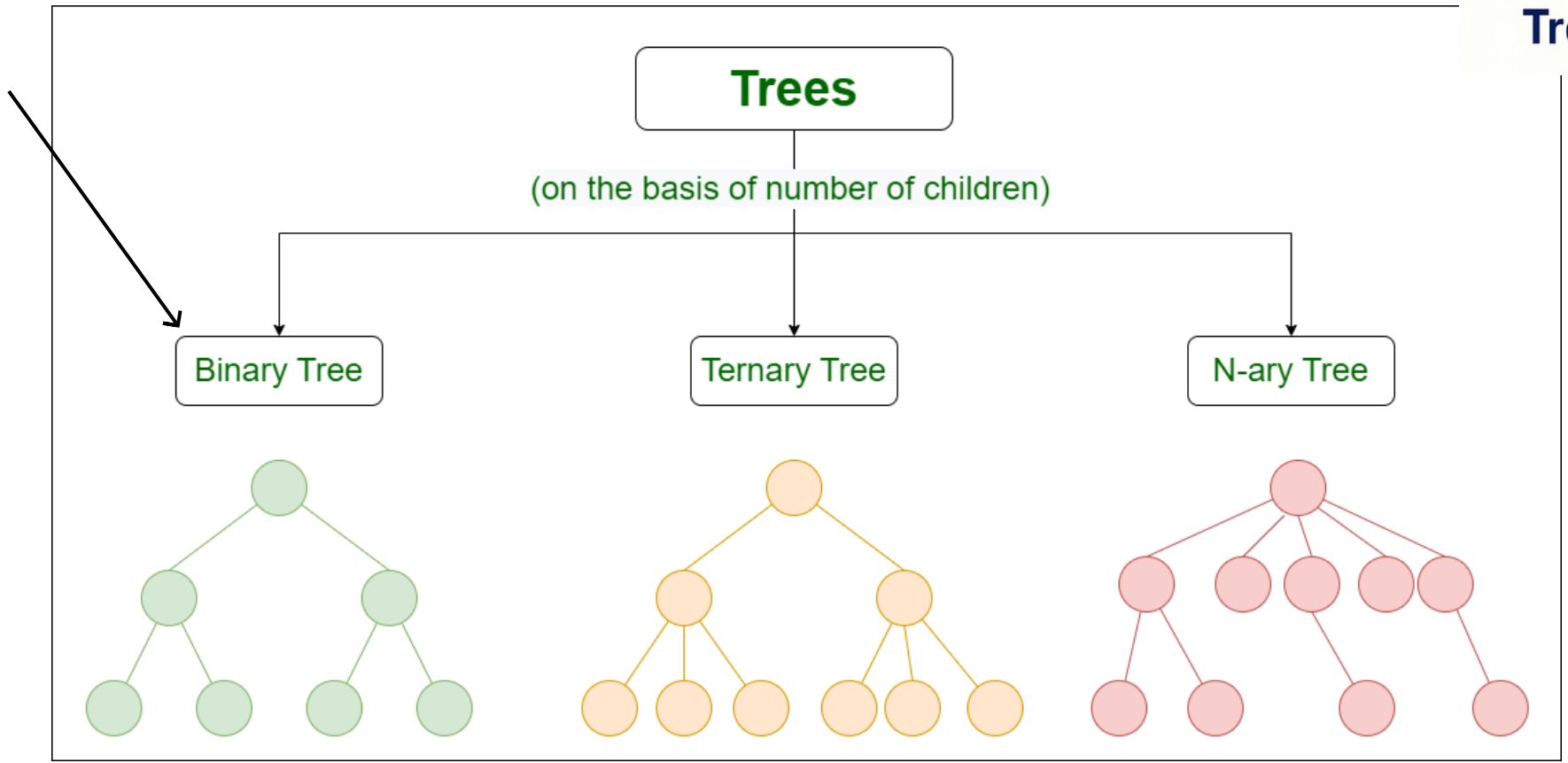
Why do we need trees?

- **Hierarchical Organization:** naturally represent hierarchical relationships
- **Efficient Operations**: many tree structures (like Binary Search Trees) allow for insertion, and deletion operations (much faster than linear structures)
- **Structured Storage:** organize data in a way that reflects real-world relationships
- **Optimization**: efficient algorithms for sorting, searching data.

Trees in real life

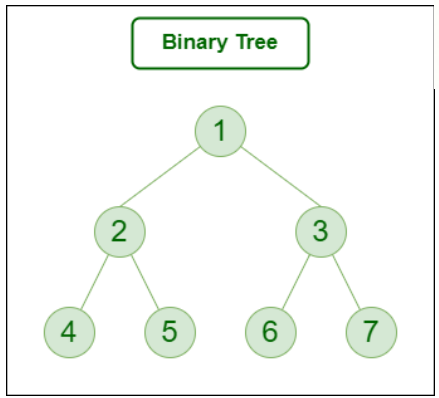
- **File Systems**: computer's directory structure is a tree
  - folders contain subfolders and files, with the root directory at the top.
- **Family Trees**: relationships showing ancestors and descendants.
- **Decision Trees**: in machine learning and decision-making processes, where each internal node represents a decision point.
- **XML/HTML Structure**: web pages and markup documents form tree structures with nested elements.
- **Game Trees**: In game theory and AI, representing possible moves and outcomes in games like chess.
- **Compiler Design**: syntax checking and constructing syntax trees

Types of trees



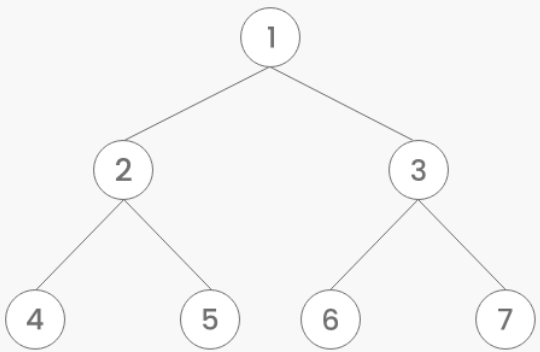
Binary tree

- Tree structure with **at most 2 children**: **left** & **right** child.
- **Types** of Binary Tree:
  - Based on **nr of childeren**
    - Full (each node has 0 or 2 children)
    - Degenerate (each node only 1 child)
  - Based on the **completion of levels**
    - Complete (all levels are completely filled except last level, filled left to right)
    - Perfect (all internal nodes have two children and all leaf nodes are at the same level)
    - Balanced (difference between height of left and right subtree for each node is either 0 or 1)
  - Based on the **value of nodes**
    - **Binary Search: Left subtree of a node < node < right subtree of node**
    - And many more AVL, Red Black Tree, B Tree, B+ Tree, Segment Tree...



Tree traversals

- Trees can be traversed in different ways
- Depth First Search or DFS
  - **In-order** traversal visits nodes in ascending order, **pre-order** traversal visits the root first, and **post-order** traversal visits the root last



Inorder Traversal

4	2	5	1	6	3	7

Preorder Traversal

1	2	4	5	3	6	7

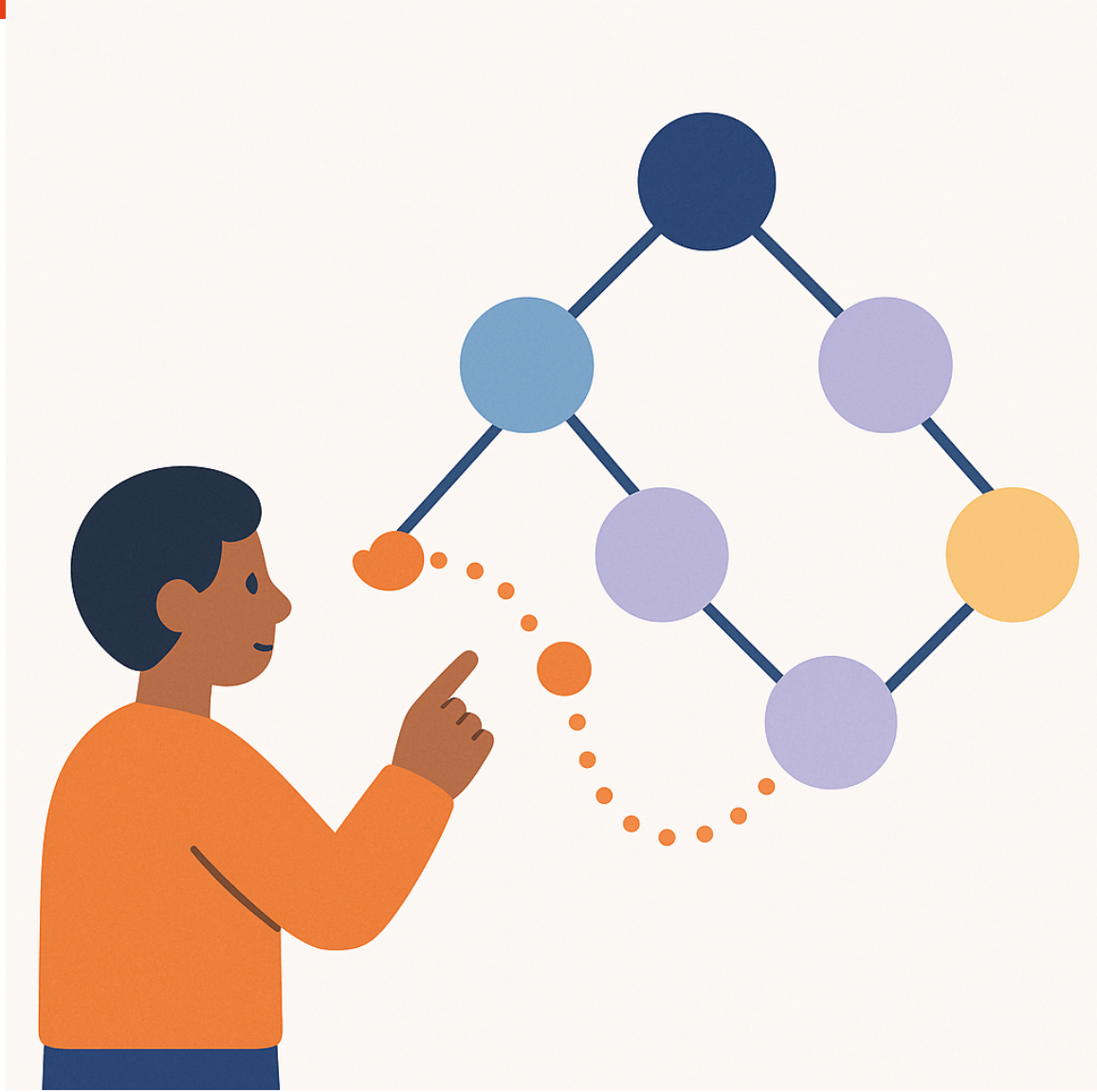
Postorder Traversal

4	5	2	6	7	3	1

- Level Order Traversal or Breadth First Search or BFS
- Boundary Traversal
- Diagonal Traversal

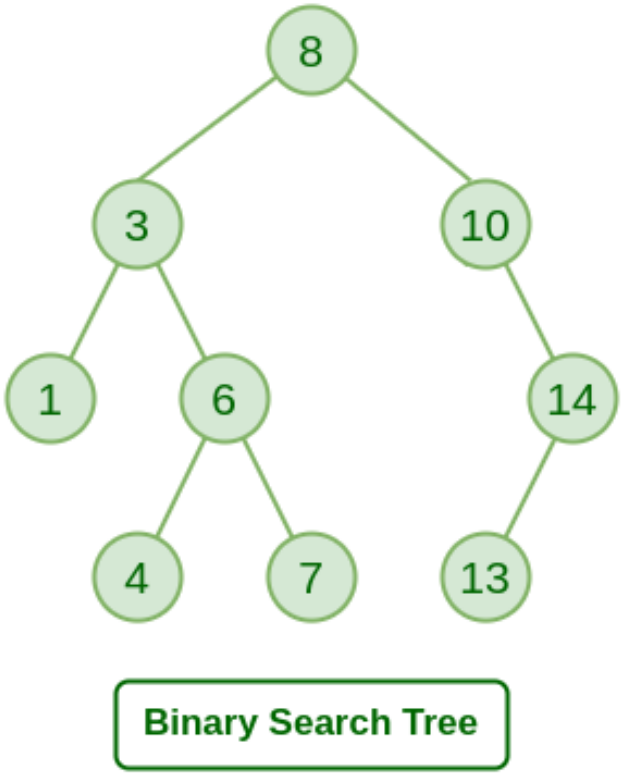
Trees

- ~~Types & traversal~~
- Binary Search Tree
  - add
  - traverse
  - search
  - min/max



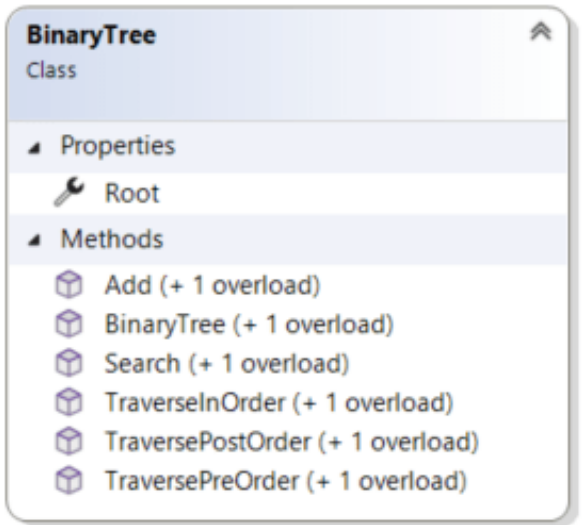
Binary Search Tree

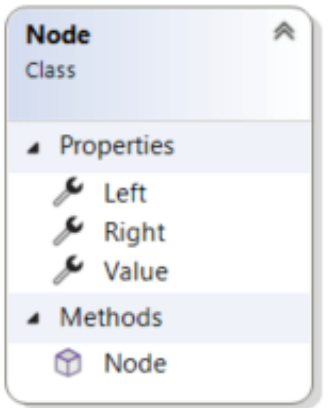
- The **left subtree** of a node contains only nodes with keys **lesser than the node's key.**
- The **right subtree** of a node contains only nodes with keys **greater than the node's key**.
- The left and right subtree each must also be a **binary search tree**.



Code!

- Create 2 classes:
  - Node
    - Left & Right child!
  - BinaryTree
    - Add & Search
    - 3 traversal methods

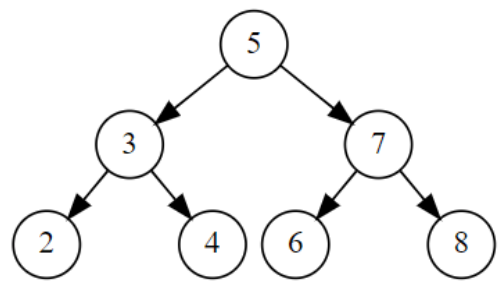




Add

- The Add method adds a new value to the tree
  - Compare the value to the current node and insert it into the appropriate child node
  - Let's work recursively
- Check https://visualgo.net/en to help to understand

```
1  //in BinaryTree
2  public void Add(int value)
3  {
4    if (Root == null)
5    {
6      Root = new Node(value);
7    }
8    else
9    {
10     Add(Root, value);
11   }
12 }
13
14 private void Add(Node node, int value)
15 {
16   //let's code
17 }
18
19 //in main
20 BinaryTree tree = new BinaryTree();
21
22 tree.Add(5);
23 tree.Add(3);
24 tree.Add(7);
25 tree.Add(2);
26 tree.Add(4);
27 tree.Add(6);
28 tree.Add(8);
```



Add

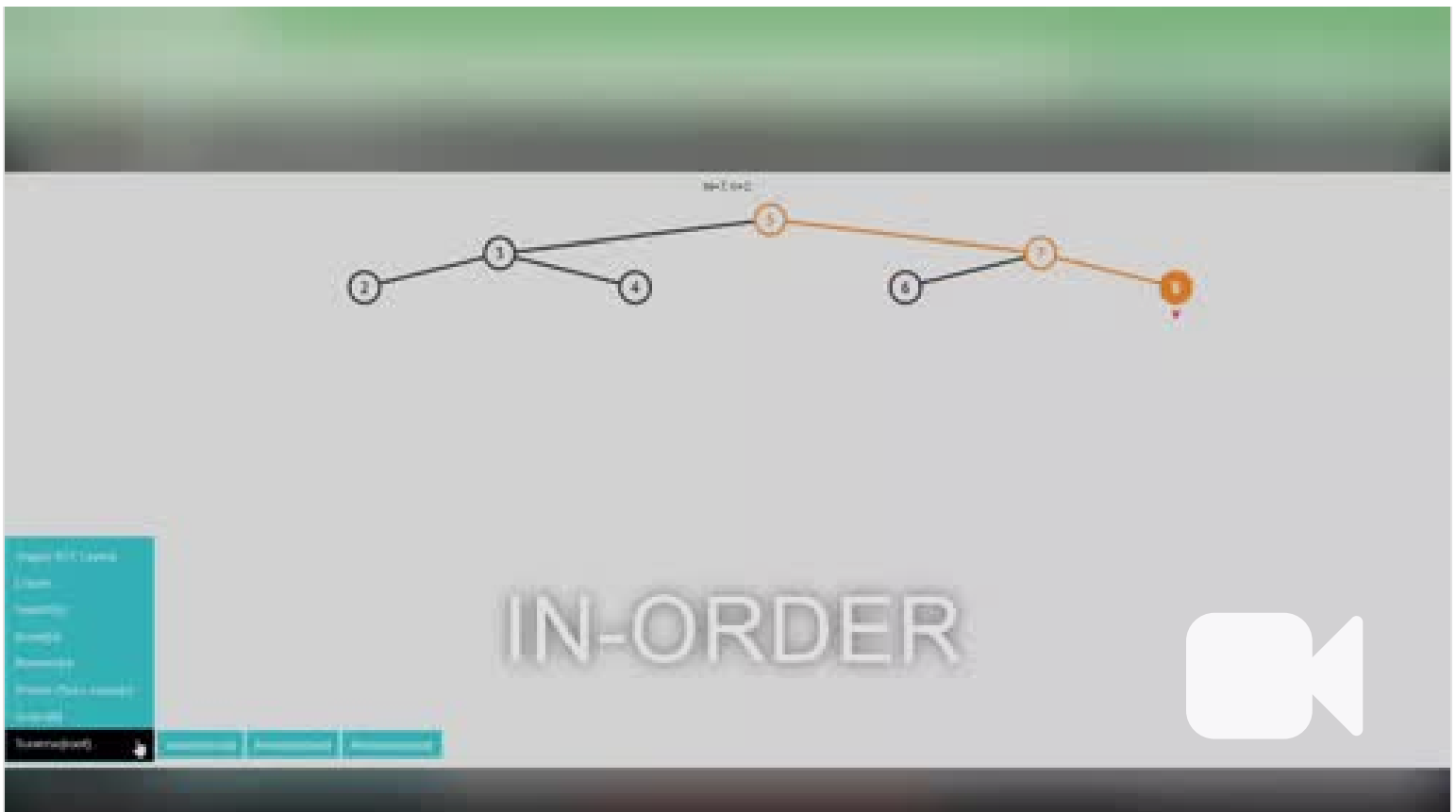
- **Base Case**: If the root is null (i.e., the tree is empty or we've reached a leaf), create a new node with the given value and return it.
- **Recursive Case**:
  - If the `value` to be inserted is less than the value of the `root` node, proceed to the left subtree:
    - If the left child is null, create a new node with `value` and set it as the left child of the root.
    - If the left child is not null, recursively call the `insert` function for the left child.
  - If the `value` is greater than the value of the `root` node, proceed to the right subtree:
    - If the right child is null, create a new node with `value` and set it as the right child of the root.
    - If the right child is not null, recursively call the `insert` function for the right child.

Trees

- ~~Types & traversal~~
- Binary Search Tree
  - ~~add~~
  - traverse
  - search
  - min/max

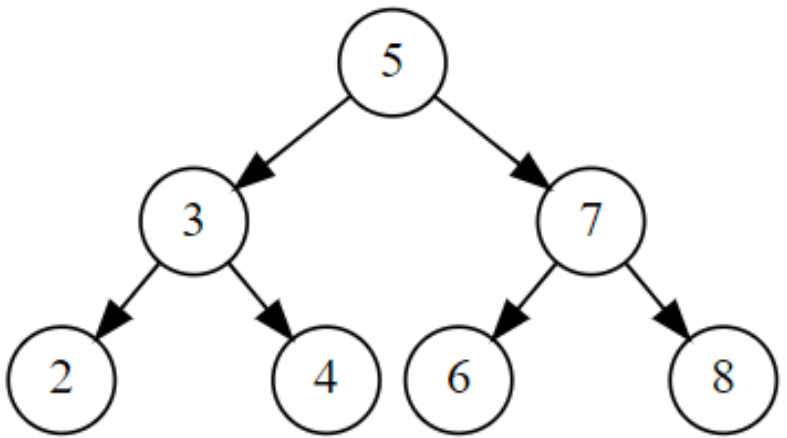
Traverse

- Binary trees can be traversed in three different ways!



Traverse

- Binary trees can be traversed in three different ways:
- **In-order**: visit left subtree, **node**, right subtree
  - **2 3 4 5 6 7 8**
- **Pre-order**: visit **node**, subtree, right subtree
  - **5 3 2 4 7 6 8**
- **Post-order**. visit left subtree, right subtree, **node**.
  - **2 4 3 6 8 7 5**

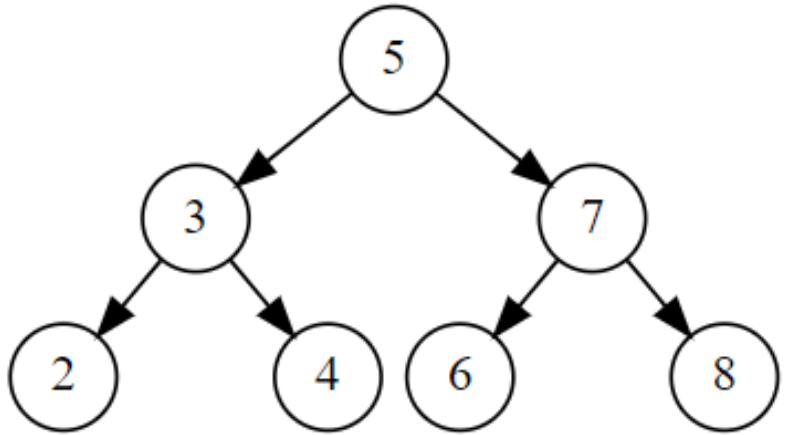


Which traversal method?

- **Inorder** (Left → Root → Right)
  - Visits nodes in sorted order --> **getting data** in ascending **order**
- **Preorder** (Root → Left → Right)
  - Visits parent before children --> **copying**/cloning trees - need parent first to create children
- **Postorder** (Left → Right → Root)
  - Visits children before parent --> **deleting** trees - must delete children first before parent
- *It's about **dependencies** - when copying, you need the parent to create children, when deleting, you must remove children first to safely delete the parent.*

Traverse

```
1  //in BinaryTree
2  public void TraverseInOrder()
3  private void TraverseInOrder(Node node)
4
5  public void TraversePreOrder()
6  private void TraversePreOrder(Node node)
7
8  public void TraversePostOrder()
9  private void TraversePostOrder(Node node)
10
11 //in main
12 Console.WriteLine("In-order traversal:");
13 tree.TraverseInOrder();
14
15 Console.WriteLine("\nPre-order traversal:");
16 tree.TraversePreOrder();
17
18 Console.WriteLine("\nPost-order traversal:");
19 tree.TraversePostOrder();
```



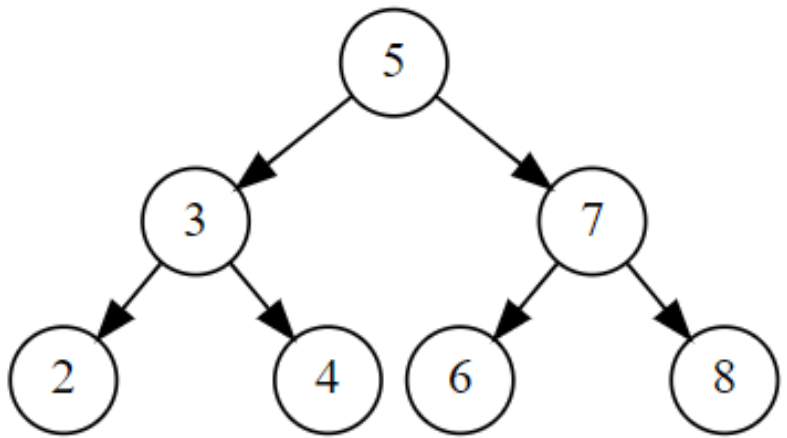

```
In-order traversal:
2 3 4 5 6 7 8
Pre-order traversal:
5 3 2 4 7 6 8
Post-order traversal:
2 4 3 6 8 7 5
```

Trees

- ~~Types & traversal~~
- Binary Search Tree
  - ~~add~~
  - ~~traverse~~
  - search
  - min/max

Search

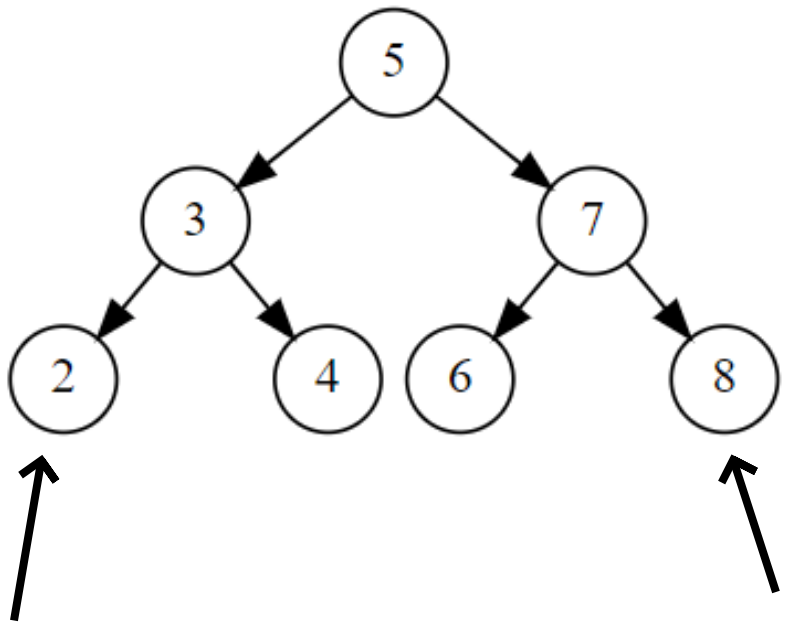
```
1  //in BinaryTree
2  public bool Search(int value)
3  {
4    return Search(Root, value);
5  }
6
7  private bool Search(Node current, int value)
8  {
9    //code
10 }
11
12 //in main
13 Console.WriteLine("\nFound 4? " + tree.Search(4));
14 Console.WriteLine("Found 10? " + tree.Search(10));
```




```
Found 4? True
Found 10? False
```

Min and Max

- Minimum = the leftmost node
- Maximum = the rightmost node




```
1  //in BinaryTree
2  public int FindMax()
3  public int FindMin()
4
5  //in main
6  Console.WriteLine("Min: " + tree.FindMin());
7  Console.WriteLine("Max: " + tree.FindMax());
```

```
Min: 2
Max: 8
```

- Add extra methods:
  - deleting nodes
  - couting nodes
  - summing nodes
  - balancing tree (check if balanced
  - finding height of tree
  - ....

- *Every tree IS a graph, not every graph is a tree*
- *Tree Structure: Root at top, internal nodes with children, leaves at bottom.*
- *Binary trees have max 2 children per node.*
- *Traversal Methods:*
  - *Inorder: Left → Root → Right (sorted order)*
  - *Preorder: Root → Left → Right (good for copying)*
  - *Postorder: Left → Right → Root (good for deletion)*

How to Organize Data

- **Purpose**: Choosing the right way to store and access information
- **Why it matters**: The way you organize data determines what you can do with it efficiently. Wrong choice = slow program.
- **Real-world impact:** How databases store millions of records, how social networks map connections, how file systems organize your computer.

Hash Tables

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Hash tables

- Searching & data structures (again! 🙄😄)
- Aha, so that's a hash function/table
- Collisions
- Making hashing work!
  - Hash table size
  - Hash function
  - Collision handling
- Vulnerabilities in Hash Tables

Searching, again...

Suppose we are buying groceries and the seller must look up every price in a book

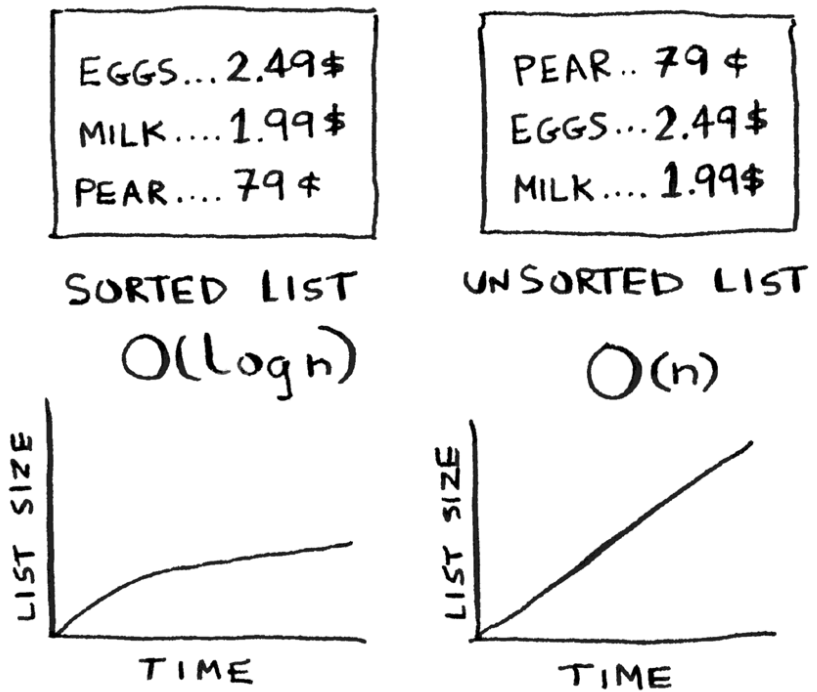
How long will it take

- in an unalphabetized book?
- in an alphabetized one?

Searching, again...

It will take...

- O(n) in an unalphabetized book
- O(log n) in an alphabetized one



Searching, again...

Still... looking things up in a book is a pain, the customer line is growing and the cashier is getting nervous.

He/she actually needs a buddy, someone who instantly knows every selling price!

Searching, again...

Maggie is perfect for the job, no matter how big the book is, she always gives the price in O(1) time! She's even faster than binary search!

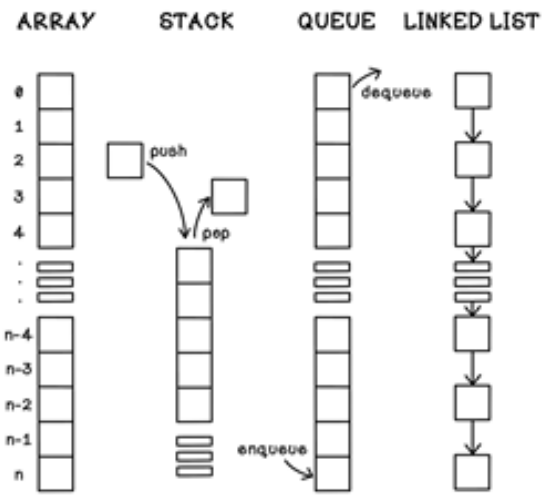
# OF ITEMS IN THE BOOK	SIMPLE SEARCH O(n)	BINARY SEARCH O(log n)	MAGGIE O(1)
100	10 sec	1 sec	INSTANT
1000	1.6 min	1 sec	INSTANT
10000	16.6 min	2 sec	INSTANT

Datastructures, again...

You know a couple of data structures so far

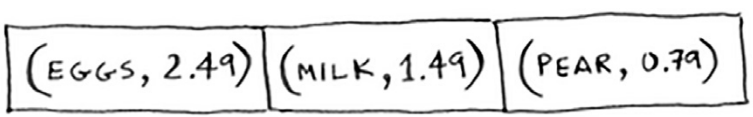
- array
- lists
- queue & stack

  we only take array and lists into account because a queue and stack aren't build for searching data!



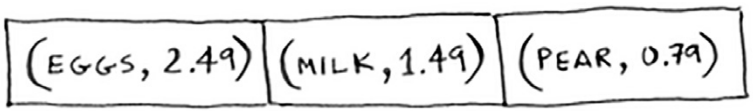
We could implement this book as a key/value array:

- key = product
- value = price



Datastructures, again...

- Implement this book as a **key/value array!**
- Think of sorting & searching: quicksort & binary search!




```
1  static void Main(string[] args)
2  {
3    Book_KVP book = new Book_KVP(20);
4    book.AddItem("apple", 0.67);
5    book.AddItem("pear", 0.79);
6    book.AddItem("eggs", 2.49);
7    book.AddItem("milk", 1.49);
8    book.AddItem("avocado", 1.49);
9    Console.WriteLine(book.ToString());
10   Console.WriteLine("Price of eggs: " + book.GetPrice("eggs"));
11   Console.WriteLine(book.ToString());
12 }
```

```
apple: 0.67
pear: 0.79
eggs: 2.49
milk: 1.49
avocado: 1.49

Price of eggs: 2.49

apple: 0.67
avocado: 1.49
eggs: 2.49
milk: 1.49
pear: 0.79
```

Datastructures, again...

But...we still have to sort & search before we get the right price back... and we want the items in O(1) time!

We want to make a "Maggie"!

This is where hash-functions come in!

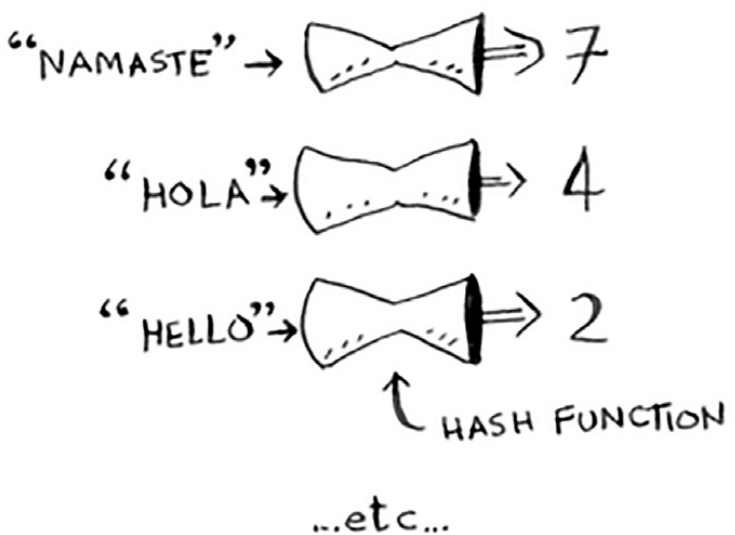
Hash table

- ~~Searching & data structures (again! 🙄😄)~~
- Aha, so that's a hash function/table
- Collisions
- Making hashing work!
  - Hash table size
  - Hash function
  - Collision handling
- Vulnerabilities in Hash Tables

Aha, that's a hash function!

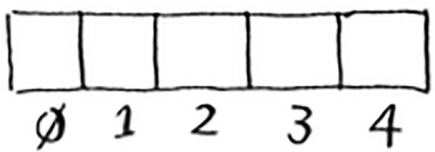
- A hash function is a function where you put in a string (any kind of data = sequence of bytes) en you get a number back!
- A hash function maps strings to numbers

- Requirements for a 'good' hash function
  - needs to be **consistent**
  - should **map different words** to **different numbers**

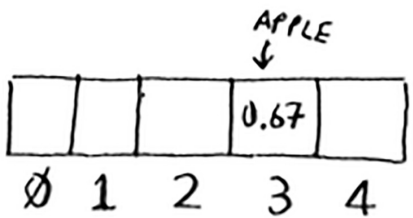


Hash function

- Oh, but how are we going to build 'a Maggie'?
- Start with an empty array:

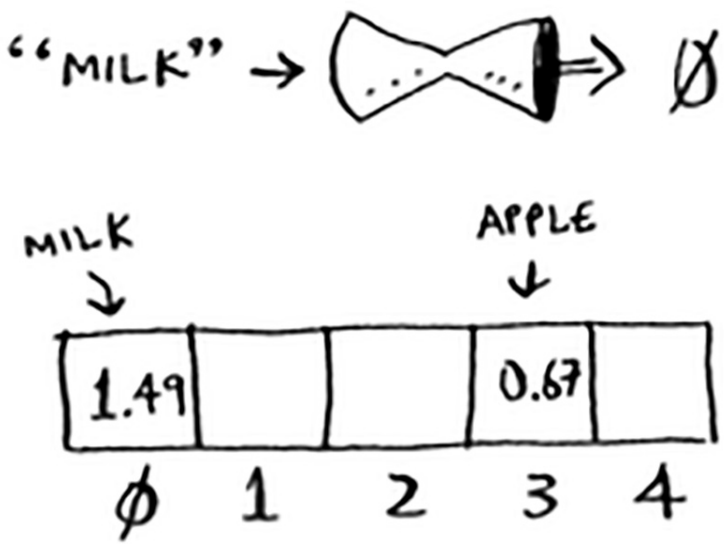


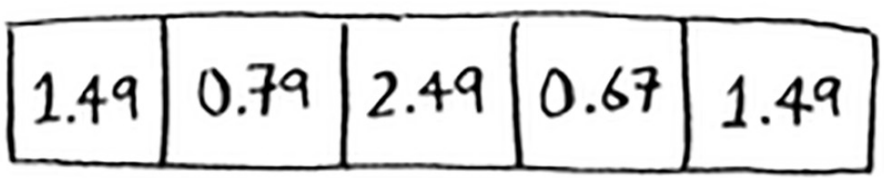
- You'll store all of your prices in this array.
- Let's add the price of an apple, feed "apple" into the hash function



Hash function

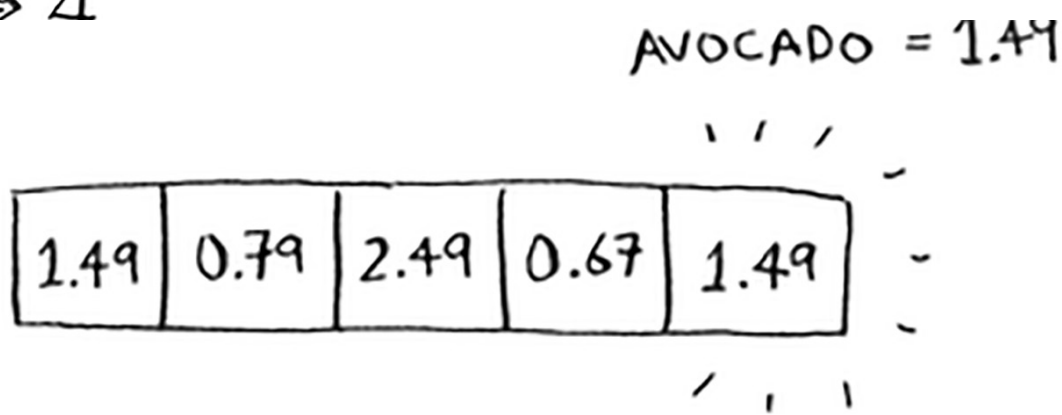
- And so on untill the array is full!



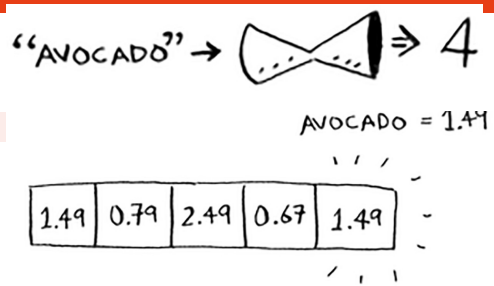


Hash function

- Now ask 'what's the price for an avocado?' You don't need to search for it in the array. Just feed "avocado" into the hash function.
- The hash function tells you where to look for the price!

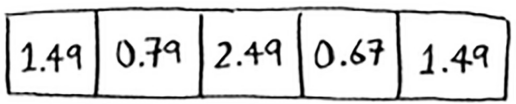


The hash function...



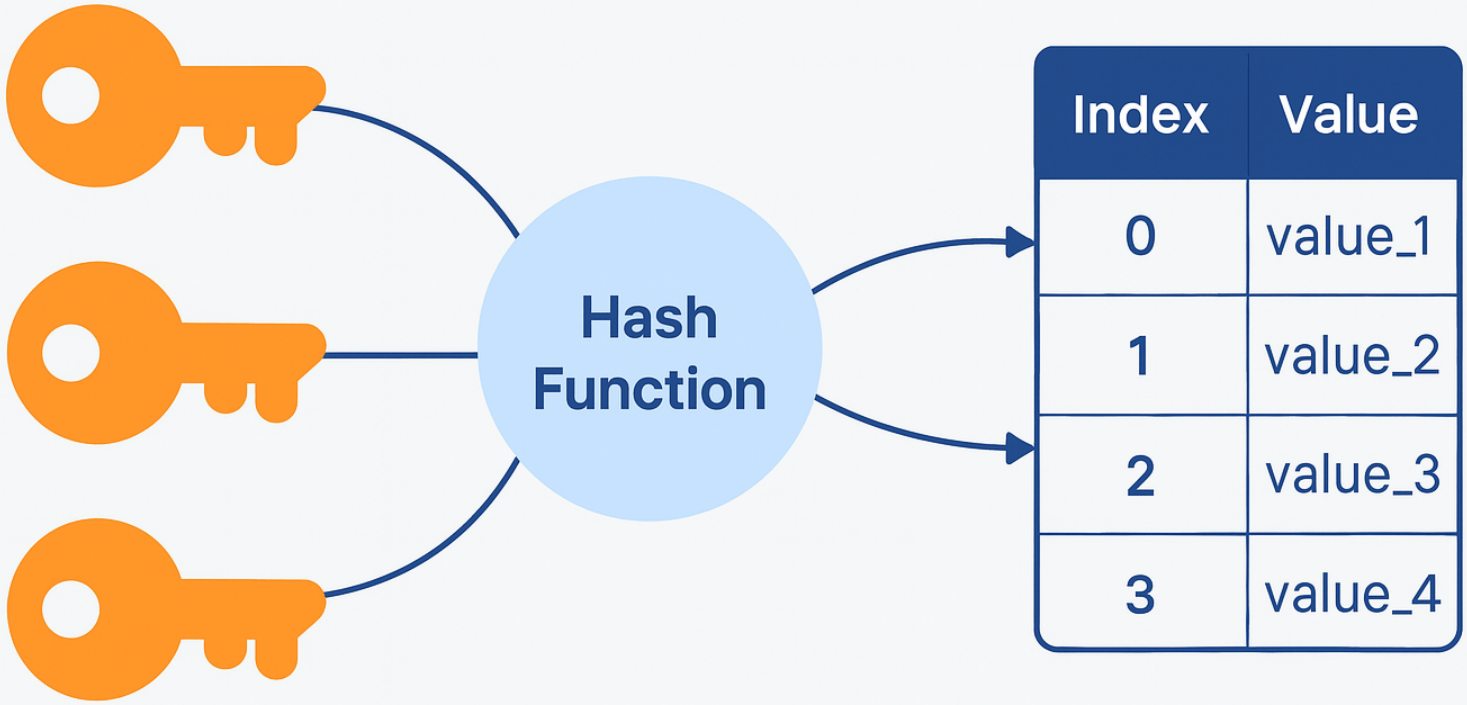
- **Consistently** maps a name to the **same index**
  - every time you put in "avocado", you'll get the same number back!
  - you first use it to find a good spot, afterward use it to find that spot again

- Maps **different strings** to **different indexes**.

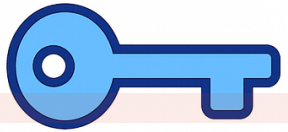


  - avocado maps to index 4, milk to index 0... everything maps to a different slot

- **Knows the length of your array** and only returns valid indexes!
  - if your array is 5 items, the hash function doesn't return 100…


Hash
Function
Index
Value
0
value_1
1
value_2
2
value_3
3
value_4

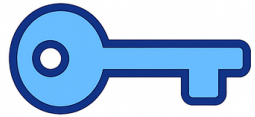
What is a hash table?



Hash Table

- **Hash function + Array = Hash table**
- **Data structure** that **maps keys to values** using a hash function to compute an index
  - *Also known as: Hash maps, maps, dictionaries, associative arrays*
  - **Hash function**: Takes a key and converts it to an array index
  - **Array**: Storage locations where the actual data is kept
  - **Key-Value pairs**: The key is used for lookup, the value is actual data
- **What about speed?**
  - Array/Linked List search: O(n) - might check every single item
  - Sorted Array search: O(log n) - binary search cuts search space in half repeatedly
  - Hash Table search: O(1) - jump directly to the answer!

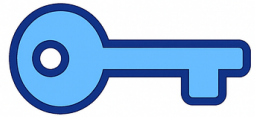
Why do we need hash tables?



Hash Table

- **Fundamental Problem: Speed vs Scale**
  - Real-world data is huge & traditional search methods take too long
  - Modern applications demand instant responses (Google?)
- **Alternative Data Structures fails**
  - Arrays: Only fast if you know the exact position
  - Sorted Arrays: Adding new data requires shifting everything
  - Linked Lists: Must start from the beginning every single time
- **What Hash Tables Solve**
  - **Direct access**: Use the key itself to find the data instantly
  - **No searching required**: Turn "where is it?" into "it's right here!"
  - **Scales beautifully**: Works just as fast with 10 items or 10 million items

Hash tables in real life



Hash Table

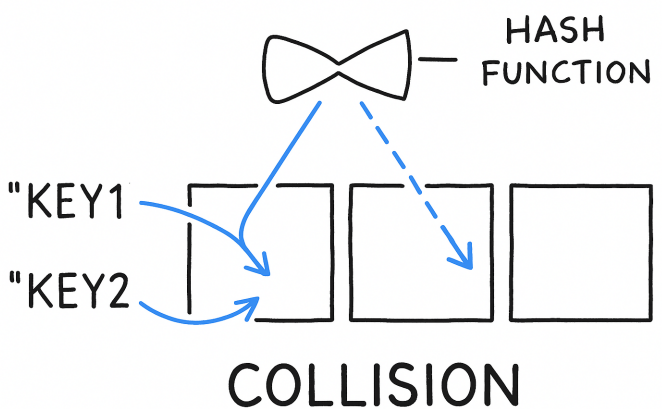
- **Your Phone:**
  - Contacts: Type "Mom" → instant phone number (not searching 500 contacts)
  - Apps: Tap Instagram → OS instantly finds and launches it
- **The Internet**
  - TikTok login: Verify your password among 1.5 billion users instantly
  - Netflix: Search "Marvel" → instantly find all Marvel content
- **Your Computer**
  - File system: Double-click file → OS instantly knows where it's stored
  - Memory: Programs get exact memory they need without searching
- *Every search, login, and file access would be painfully slow without hash tables, modern digital life works exactly as you expect - instantly!*

07 Hash table

- ~~Searching & data structures (again! 🙄😄)~~
- ~~Aha, so that's a hash function/table~~
- Collisions
- Making hashing work!
  - Hash table size
  - Hash function
  - Collision handling
- Vulnerabilities in Hash Tables

Collisions

- Before we can think of creating a hash table: think about collisions!
  - **Two keys crashing into each other at the same location**
- How so? A hash function maps a large set (all possible keys) to a smaller set (hash table locations)... so there is the **possibility of collisions!**
- What is the chance of this happening?
  It's more likely than you might think!



Collisions

- Suppose we have a hashtable with M slots and we have N keys to insert

- The probability of a collision when a Nth key is inserted is calculated by probability theory as follows:
  - prob(collision) = 1 - prob(no collisions)
  - prob(collision) = 1 - prob(first key has no collision) * prob(second key has no collision) * prob(third key has no collision) * ... * prob(Nth key has no collision)

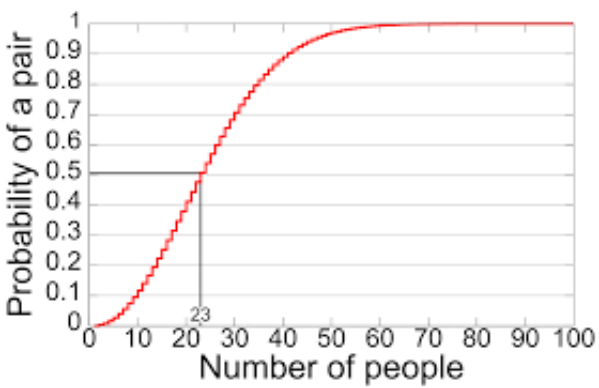
$$P_{N,M}(collision) = 1 - \frac{M-1}{M} * \frac{M-2}{M} * \ldots * \frac{M-N-1}{M}$$

Birthday paradox

- You would think the probability is very low, but that's not quite right!
- Suppose there are 365 slots in the hash table: M=365
- *What is the probability that there will be a collision when inserting N keys?*
  - For N = 60, probN,M(collision) = 99+%
  - For N = 50, probN,M(collision) = 97%
  - For N = 40, probN,M(collision) = 89%
  - For N = 30, probN,M(collision) = 71%
  - For N = 20, probN,M(collision) = 41%
  - For N = 10, probN,M(collision) = 12%

Birthday paradox

- This means that in a relatively small group of people (from 23 people) there is a 50% chance that two people will celebrate their birthday on the same day!
- How crazy is that? Really, think about it! There are 365 days to fill and with 60 people there is almost 100% certainty two people have the same birthday!



Collisions

So In general: collisions are likely to happen, unless the hash table is quite sparsely filled!

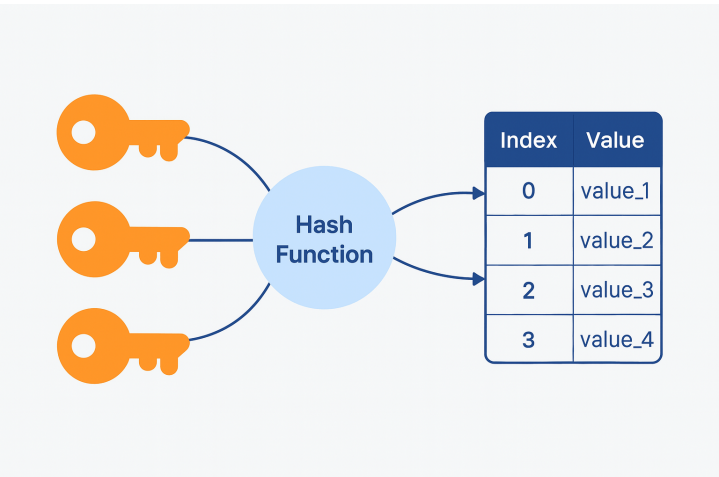
Hash table

- ~~Searching & data structures (again! 🙄😄)~~
- ~~Aha, so that's a hash function/table~~
- ~~Collisions~~
- Making hashing work!
  - Hash table size
  - Hash function
  - Collision handling
- Vulnerabilities in Hash Tables

Making hashing work

The main issues in implementing hashing are

- Deciding on the **size** of the hash table
- Deciding on the **hash** function
- Deciding on the **collision resolution strategy**



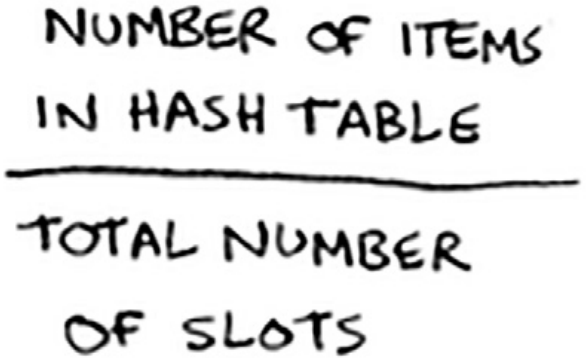
Hash table size

- Choice of hash table size depends on
  - choice of the hash function
  - choice of collision strategy
- General “rule of thumb” about the size is
  - should be 1.3 times the max number of keys actually be stored in the table
  - should be a prime number

    Let M be the next prime larger than 1.3 times the number of keys
- If you **underestimate** the number of keys, you may have to **create** a **larger table** and rehash when it gets too full; if you **overestimate** the number of keys, you will be **wasting** some **space**

Load factor

- How do you know when your table is getting too full?
- A good measure is the load factor
- The load factor measures how many empty slots remain in your hash table.
- Another rule of the thumb is to resize the table when the load factor is greater than 0.7.



Hash table

- ~~Searching & data structures (again! 🙄😄)~~
- ~~Aha, so that's a hash function/table~~
- ~~Collisions~~
- ~~Making hashing work!~~
  - ~~Hash table size~~
  - Hash function
  - Collision handling
- Vulnerabilities in Hash Tables

Let's start coding again

- Create a class to hold a hash table
  - Use an array of doubles
  - Search in the constructor for the perfect size of the hashtable

```
1  Book_Hash book = new Book_Hash(5);
2  book.AddItem("apple", 0.67);
3  book.AddItem("pear", 0.79);
4  book.AddItem("eggs", 2.49);
5  book.AddItem("milk", 1.49);
6  book.AddItem("avocado", 1.49);
7  Console.WriteLine(book.ToString());
8  Console.WriteLine("Price of eggs: " + book.GetPrice("eggs"));
9  Console.WriteLine(book.ToString());
```

Hash function

- A hash function should be **very fast to compute**

  we need it twice: once to store data, once to lookup data
- A hash function should **distribute keys as uniformly as possible** in the hash table, to avoid collisions as much as possible
- A hash function can depend on the type of keys and the size of the table
- We will take a look at hash functions for
  - integers
  - strings

Hash functions for integers

- Hashtable of size M, the value of key K
- A good fast hash function is H(K) = K mod M
  - mod = remainder after division!
- Note: size M of the table must be a prime number!
  - if not: a lot more possible collisions!

key	hash (M = 100)	hash (M = 97)
212	12	18
618	18	36
302	2	11
940	40	67
702	2	23
704	4	25
612	12	30
606	6	24
772	72	93
510	10	25
423	23	35
650	50	68
317	17	26
907	7	34
507	7	22
304	4	13
714	14	35
857	57	81
801	1	25
900	0	27
413	13	25
701	1	22
418	18	30
601	1	19

Modular hashing

Hash functions for strings

- There is no recognized single "best" hash function for strings! However most of the time you will have strings as keys
- The basic approach is to use the characters in the string to compute an integer, and then take the integer mod the size of the table!

  Think about the ASCII values of your characters!

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Hash functions for strings

- When you add an item, you add an item on the hash value of the key!

```
1    public void AddItem(string key, double value)
2    {
3      book[HashFunction(key)] = value;
4    }
```

- What about the hash function?

  Calculate the sum of every ASCII value!

```
1  private int HashFunction(string item)
2  {
3    int h = 0;
4    foreach(char c in item) h += (int)c;
5    return h % book.Length;
6  }
```

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Hash functions for strings

- What do you notice when executing the code?
- Does it give you the right values back? No!
- Our hash table is not filled correctly!

```
1  Book_Hash book = new Book_Hash(5);
2  book.AddItem("apple", 0.67);
3  book.AddItem("pear", 0.79);
4  book.AddItem("eggs", 2.49);
5  book.AddItem("milk", 1.49);
6  book.AddItem("avocado", 1.49);
7  Console.WriteLine(book.ToString());
8  Console.WriteLine("Price of eggs: " + book.GetPrice("eggs"));
9  Console.WriteLine(book.ToString());
```

```
0
0
1.49
0
0.79
1.49
0
Price of eggs: 1.49
0
0
1.49
0
0.79
1.49
0
```

Hash functions for strings

- A more effective approach is to compute a polynomial whose coefficients are the integer values of the chars in the String!
- We will use Horner's method for this!

$$a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots + a_n x^n$$
$$= a_0 + x\left(a_1 + x\left(a_2 + x\left(a_3 + \cdots + x(a_{n-1} + x\, a_n)\cdots\right)\right)\right)$$

a = character values, x = some prime number (it is very common to use 31!)

- Why do we need x to be a prime number? Multiplying by a prime number generates a nicer spread of numbers compared to multiplying by, let's say, an even number, or the number 10!

Hash functions for strings

- Create a new Hash function!

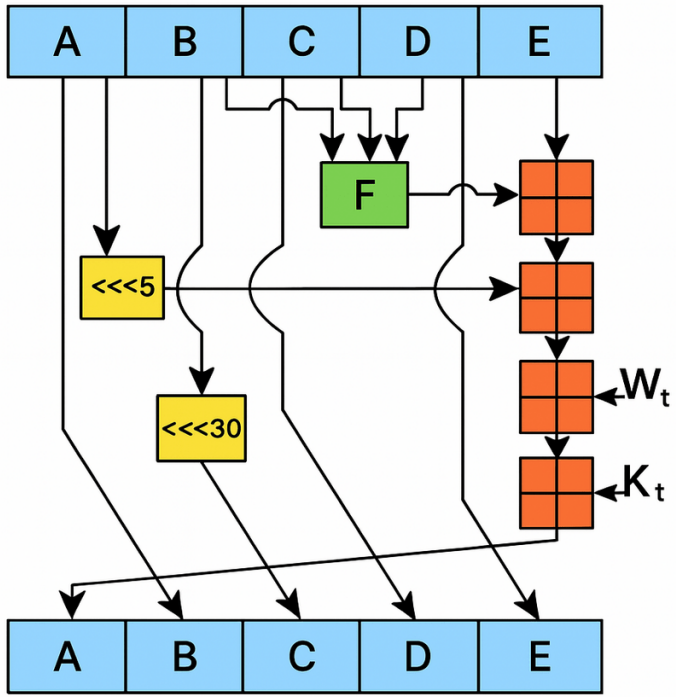
```
1 private int HashFunctionHorner(string item)
2 {
3   long h = 0;
4   foreach (char c in item) h = (31 * h) + (int)c;
5   return (int)(h % book.Length);
6 }
```

- What do you notice when executing the code?
- Our hash table is filled correctly!

```
1 Book_Hash book = new Book_Hash(5);
2 book.AddItem("apple", 0.67);
3 book.AddItem("pear", 0.79);
4 book.AddItem("eggs", 2.49);
5 book.AddItem("milk", 1.49);
6 book.AddItem("avocado", 1.49);
7 Console.WriteLine(book.ToString());
8 Console.WriteLine("Price of eggs: " + book.GetPrice("eggs"));
9 Console.WriteLine(book.ToString());
```

Other hash functions

- These are simple understandable examples of hash functions! Of course, in practice much 'heavier' algorithms will be used, such as those from the SHA family (**Secure Hash Algorithm**)! This is a collection of related cryptographic hash functions designed by the US National Security Agency!
- **But it always comes down to translating a key to a spot in an array so you can only store the value!**



Hash table

- ~~Searching & data structures (again! 🙄😄)~~
- ~~Aha, so that's a hash function/table~~
- ~~Collisions~~
- ~~Making hashing work!~~
  - ~~Hash table size~~
  - ~~Hash function~~
  - Collision handling
- Vulnerabilities in Hash Tables

What about the collisions?

- Unless you are doing "perfect hashing" (impossible!)

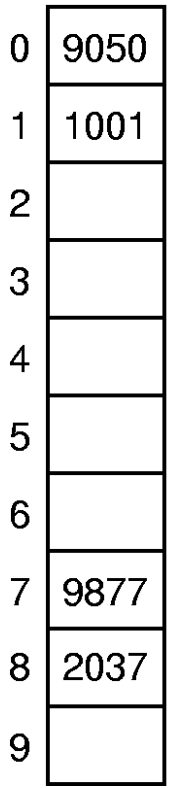
  --> we need collision resolution strategy

- Two possible strategies:
  - Open addressing
  - Separate Chaining

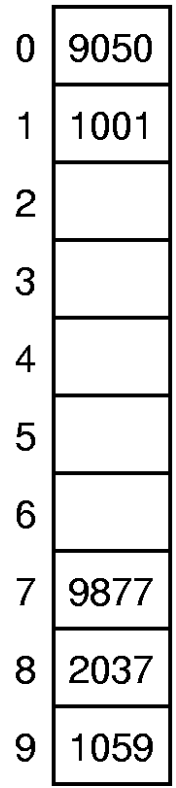
What about the collisions?

Open addressing (or probing): search for other slots using a second function and store item in the first empty slot that is found.

The **disadvantage** is that clusters can form when searching for the next free space, so our values are not evenly distributed across the table.



(a)

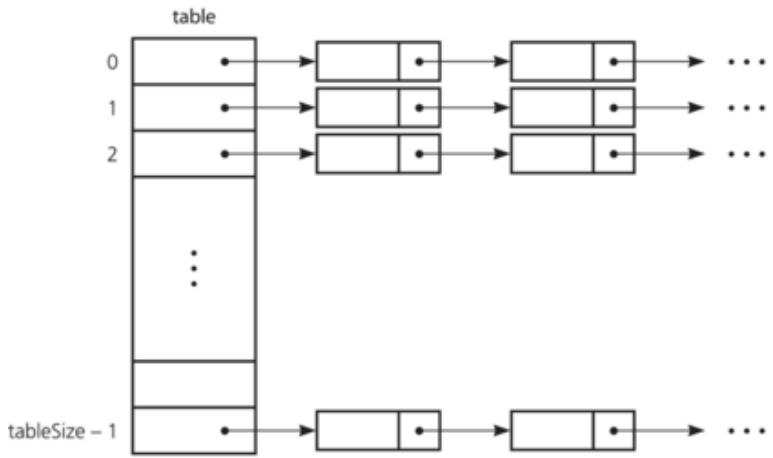


(b)

What about the collisions?

Separate chaining: use another data structure such as a linked list to store multiple items that hash to the same slot

The **disadvantage** is that we are not holding on to the perfect O(1) performance, because we have to loop lists again to search for the right key!



What about the collisions?

- Transform your latest implementation of a hash table to a hash table with separate chaining as a collision strategy!
  - Use an array of Lists
  - Your list is again a key/value-pair (because we need to store the keys as well!)

    otherwise, we don't know if we need to return the price for banana or bread

```
1   Book_HashSep book = new Book_HashSep(5);
2   book.AddItem("apple", 0.67);
3   book.AddItem("pear", 0.79);
4   book.AddItem("eggs", 2.49);
5   book.AddItem("milk", 1.49);
6   book.AddItem("avocado", 1.49);
7   book.AddItem("bread", 2.49);
8   book.AddItem("banana", 1.09);
9   Console.WriteLine(book.ToString());
10  Console.WriteLine("Price of eggs: " + book.GetPrice("eggs"));
11  Console.WriteLine(book.ToString());
```

```
- milk 1.49
- apple 0.67
null
- bread 2.49 - banana 1.09
- eggs 2.49
- pear 0.79
- avocado 1.49
Price of eggs: 2.49
- milk 1.49
- apple 0.67
null
- bread 2.49 - banana 1.09
- eggs 2.49
- pear 0.79
- avocado 1.49
```

Hash table

- ~~Searching & data structures (again! 🙄😄)~~
- ~~Aha, so that's a hash function/table~~
- ~~Collisions~~
- ~~Making hashing work!~~
  - ~~Hash table size~~
  - ~~Hash function~~
  - Collision handling
- Vulnerabilities in Hash Tables

RAINBOW
TABLE

Dark side of Hashing

- **The Password Storage "Solution"**
  - **The problem**: Storing passwords in plain text is obviously terrible
  - **The "brilliant" solution**: Hash passwords before storing them
  - **The logic**: If someone steals our database, they only see hash values
  - ***The reality**: This created a whole new attack surface*
- **The Fatal Flaw: Predictability**
  - Same input = same output
    - *"mypassword" = a665a45920422f9d417e4*
  - This consistency is normally hash tables' greatest strength
  - Predictability becomes exploitable when dealing with secrets

```
User enters: "mypassword"
Hash function: SHA-256("mypassword")
In database: "a665a45920422f9d417e4"
```

Dark side of Hashing

- **Rainbow Tables**: the Hacker's dream tool!
- What Are Rainbow Tables?
  - **Pre-computed lookup tables**: massive databases mapping common passwords to hash values
  - **Time-space tradeoff**: months computing hashes once, then crack passwords instantly
  - **Comprehensive coverage**: includes millions of common passwords, dictionary words, and variations

```
Password          → Hash Value
"password"        → "dd62a11ef721d1542d8"
"password1"       → "183d32937b851835442"
"password123"     → "a44a4e166c066599118"
"123456"          → "aff3ca12020c923adc6"
```

Rainbow table attacks

- **Step 1: Database Breach**
  - Hackers break into company database
  - Steal hash table containing all user password hashes
  - Can't see actual passwords, but have the hashed versions
- **Step 2: Hash Comparison**
  - Compare stolen hashes against rainbow table entries
  - Match found: Hash "8d969eef6ecad..." matches "123456" in rainbow table
  - Password cracked: User's password is "123456"
- **Step 3: Mass Compromise**
  - Process can crack thousands of accounts simultaneously
  - Common passwords are cracked instantly
  - Even moderately complex passwords can be vulnerable

Real-World Rainbow Table attacks

- **LinkedIn (2012)**
  - Breach: 6.5 million password hashes stolen
  - Vulnerability: Used simple SHA-1 hashing without salt
  - Result: Millions of passwords cracked within hours using rainbow tables
  - Impact: Users had to change passwords on LinkedIn and other sites where they reused passwords
- **Yahoo (2013-2014)**
  - Breach: 3 billion accounts compromised
  - Vulnerability: Weak hashing algorithms made passwords vulnerable
  - Result: Massive password database became available to attackers
  - Impact: Entire email accounts compromised, leading to further breaches

Add some salt!

- **Add unique random data before hashing**
  - Each user gets a different salt
  - Salt is stored alongside the hash (not secret, just random)
- **How Salting Works**
  - Without Salt:
    - User 1: hash("password123") → "a665a45920422f9d..."
    - User 2: hash("password123") → "a665a45920422f9d..." (same hash!)
  - With Salt:
    - User 1: hash("password123" + "x7k9m2") → "b889c42f7e891a3c..."
    - User 2: hash("password123" + "p3n8q1") → "d421f8c9a2b47e1f..." (different hashes!)

Salt breaks Rainbow tables

- Even identical passwords produce different hashes
- Pre-computed tables don't include salt combinations
  - Attackers must compute hashes individually for each user
  - *What took seconds now takes weeks or months!*

Hash Tables aren't magic

- **Performance vs Security**
  - **Speed is the enemy**: Hash tables are designed to be fast
  - **Security needs slow**: Password hashing should be deliberately slow
  - **Wrong tool**: Fast hash functions (MD5, SHA-1) are terrible for passwords
  - **Right tool:** Slow, purpose-built functions (bcrypt, scrypt, Argon2)
- **Other** Hash Table **vulnerabilities**
  - Hash Collision Attacks: different passwords identical hashes --> attackers can find alternative passwords that hash to the same value
  - Information leakage: attackers can deduce information from response times, Even without seeing data, timing reveals secrets
  - ...

- *Hash table = hash function + array*
- *Collisions are bad*
- *Hash tables are really fast to search, insert, and delete!*
- *Hash tables have their vulnerabilities!*
- *Every technology has appropriate and inappropriate uses & security is hard!*

How to Organize Data

- **Purpose**: Choosing the right way to store and access information
- **Why it matters**: The way you organize data determines what you can do with it efficiently. Wrong choice = slow program.
- **Real-world impact:** How databases store millions of records, how social networks map connections, how file systems organize your computer.

Tools in the knapsack!

Programming advanced

YP0822 - YP0847

Elke Boonen & Rik Tonnelier

Knapsack problem

- The Knapsack problem
- Start solving a problem
- Solve the Knapsack problem
  - Greedy
  - Brute force
  - Backtracking
  - Divide & Conquer
  - Dynamic programming

The knapsack problem

- The knapsack problem is a **problem in combinatorial optimization!**
- Suppose you're a thief and you're in a store with a knapsack. There are all these items you can steal, but you can **only fit a certain amount in the knapsack** and you are trying to **maximize the value** of items in the knapsack.
  - *each item can only be selected once*

“ *What algorithm do you use?*

Knapsack class & Item class!

```
1  class Item: IComparable<Item>
2  {
3    public int Weight { get; set; }
4    public int Value { get; set; }
5
6    public Item(int value, int weight)
7    {
8      Weight = weight;
9      Value = value;
10
11   }
12 }
13
14 class Knapsack
15 {
16   public List<Item> Items { get; set; }
17   public int MaxWeight { get; set; }
18
19   public Knapsack(List<Item> items, int maxweight)
20   {
21     Items = items;
22     MaxWeight = maxweight;
23   }
24 }
```

Let's start!

ITEM 1 → V: 10
W:1

ITEM 2 → V: 5
W:2

ITEM 3 → V: 40
W:8

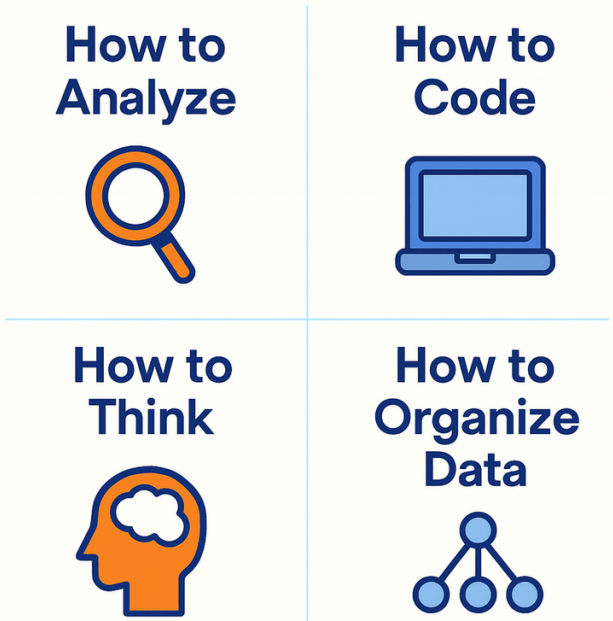
ITEM 4 → V: 45
W: 10

```
1  List<Item> list = new List<Item>();
2  //item value - weight
3  list.Add(new Item(10,1));
4  list.Add(new Item(5,2));
5  list.Add(new Item(40,8));
6  list.Add(new Item(45,10));
7
8
9  Knapsack sack = new Knapsack(list, 10);
10
11 //and now solve the problem!!
```

Knapsack problem

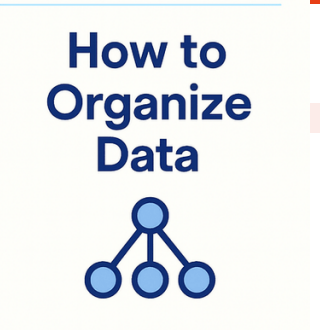
- ~~The Knapsack problem~~
- Start solving a problem
- Solve the Knapsack problem
  - Greedy
  - Brute force
  - Backtracking
  - Divide & Conquer
  - Dynamic programming

How to start?



- **How to Analyze**: We learned to analyze problems with Big O and various search algorithms
- **How to Organize Data**: We built an arsenal of data structures
- **How to Code**: We implemented sorting and recursion techniques
- **How to Think**: We learned different thinking strategies for complex problems

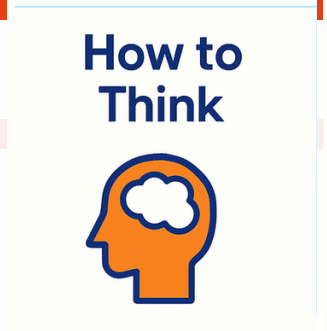
We have data structures!



- *So many data structures to organize data, to store intermediate results, to structure data to perform a specific strategy*

Data Structure	Access	Search	Insert	Delete	Best used for
Array	O(1)	O(n)	O(n)	O(n)	Fixed size, fast access
LinkedList	O(n)	O(n)	O(1)	O(1)	Dynamic size, frequent insert/delete
Stack	O(1) top	-	O(1)	O(1)	LIFO, recursion, undo functionality
Queue	O(1) front	-	O(1)	O(1)	FIFO, breadth-first search
Hashtable	O(1) avg	O(1) avg	O(1) avg	O(1) avg	Key-value lookup, caching
Binary Tree	O(log n)	O(log n)	O(log n)	O(log n)	Hierarchical data, sorted access
Graph	-	O(V+E)	O(1)	O(1)	Networks, relationships, paths

We have strategies!



- ***Recursion = Implementation Tool***
  - *Can be used in every strategy (and sometimes needed in a strategy like Backtracking)*
  - *NOT a strategy itself - it's HOW we implement the strategy*
- *Don't forget **pruning** to cut unnecessary steps!*

Strategy	**Time**	**Space**	**Use When**
Greedy	Varies (often O(n log n))	O(1)	Fast heuristic, local choices
Brute Force	Varies (often exponential)	O(n)	Small datasets, need certainty
Backtracking	Varies (often exponential)	O(n)	Constraint problems
Divide & Conquer	Varies (O(n log n) to O(n^2))	O(log n) to O(n)	Natural recursive splitting
Dynamic Programming	Varies (O(n^2) to O(n^3))	O(n) to O(n^2)	Overlapping subproblems

Ok, let's start!

- **Analysis Phase:**
  - What's the input size?
  - What are the constraints?
  - Is a perfect solution needed or is "good enough" OK?
  - How critical is performance?
- **Strategy Selection:**
  - Do I see overlapping subproblems? → Consider DP
  - Are local choices globally optimal? → Try Greedy
  - Do I need all solutions? → Backtracking
  - Can I prune bad branches early? → Backtracking with pruning
  - Is this a network problem? → Graph algorithms
- **Data Structure Choice:**
  - Do I need frequent lookups? → HashMap
  - Must data stay sorted? → Tree structures
  - Many insertions/deletions? → LinkedList
  - Fixed size with fast access? → Array
- **Implementation Wisdom:**
  - Start with the simplest working solution
  - Test with small datasets first
  - Optimize only when necessary
  - Document your choices!

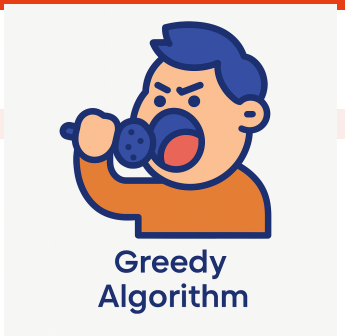
Let's solve the Knapsack problem

- **The Problem:** *4 items, knapsack capacity 10kg, maximize value*
- **Our Analysis:**
  - **Greedy**: Simple to implement, but suboptimal
  - **Brute Force**: Works, but gets slow quickly
  - **Backtracking** + **Pruning**: Smarter than brute force, can eliminate bad branches early
  - **Divide** & **Conquer**: Elegant recursive solution
  - **Dynamic Programming**: Optimal AND efficient for larger datasets
  - ~~**Graph algorithms**: Possible but overkill~~
  - ~~**Sorting algorithms**: Not directly applicable~~
- **Data Structures we use:**
  - Array/List: To store items
  - 2D Array: For DP table

Knapsack problem

- ~~The Knapsack problem~~
- ~~Start solving a problem~~
- Solve the Knapsack problem
  - Greedy
  - Brute force
  - Backtracking
  - Divide & Conquer
  - Dynamic programming

Greedy



- Be greedy, **easy way out!**
- Sort your list of items on value (descending order)
  - can also be done by inheriting from IComparable<Item>

```
1  Items.Sort();
2  Items.Reverse();
3
4  //if you just want to use Sort-method of List!
5  class Item: IComparable<Item>
6  {
7    public int CompareTo(Item other)
8    {
9      return Value.CompareTo(other.Value);
10   }
11 }
```

- Pick the first one and keep picking items until the knapsack is filled!

Greedy

ITEM 1 → V: 10 W:1

ITEM 2 → V: 5 W:2

ITEM 3 → V: 40 W:8

ITEM 4 → V: 45 W: 10

OUPUT

- What is the output? **45**
- Is this the best solution?
  - No! Because we were greedy, we didn't take a look at other possible combinations... the optimal solution is actually **50** (value of item 1 + value of item 3)
- **What about run time?**
  - **n items and only one loop over 2 items** $O(n) = n$ **(linear time)**

Knapsack problem

- ~~The Knapsack problem~~
- ~~Start solving a problem~~
- Solve the Knapsack problem
  - ~~Greedy~~
  - Brute force
  - Backtracking
  - Divide & Conquer
  - Dynamic programming

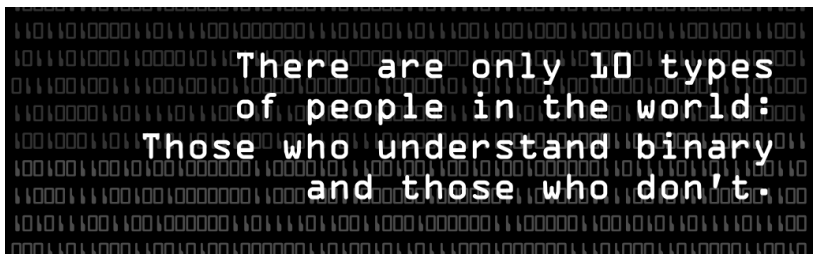
Brute force

- Find all possible combinations!
- 4 items = $\mathbf{2^4 = 16}$ combinations
- If you have two options: it's always a good idea to think binary (because binary you know, you know ;)) either you take an item or you leave it

COMBINATION	BINARY	W	VALUE
No items	0000	0	0
item 1	0001	1	10
item 2	0010	2	5
item 3	0100	8	40
item 4	1000	10	45
item 12	0011	3	15
item 13	0101	9	50
item 14	1001	11	55
item 23	0110	10	45
item 24	1010	12	50
item 34	1100	18	85
item 123	0111	11	55
item 124	1011	13	60
item 134	1101	19	95
item 234	1110	20	90
item 1234	1111	21	100

Brute force

- It's easy if you think binary and select items from your list on index!
  - 0000 = no items
  - 0001 = first item / or last
  - ...
  - 1001 = first and last item
  - 1111 = all items

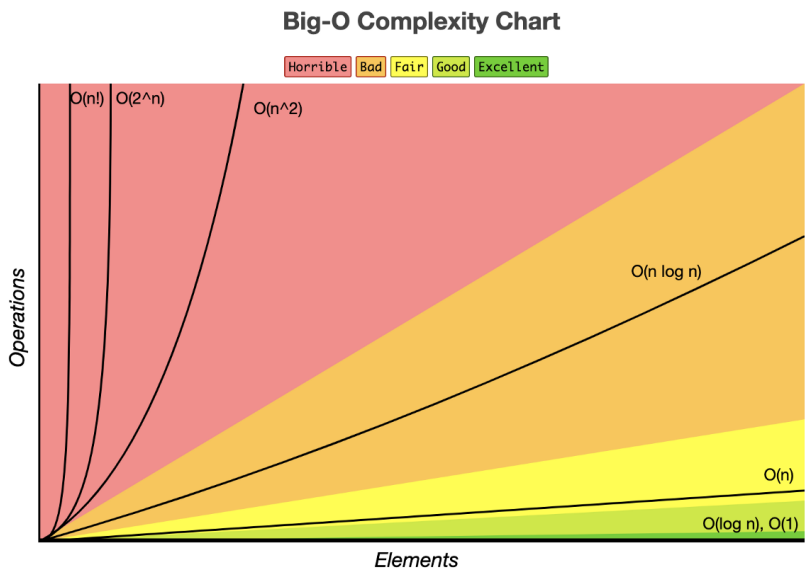


- Find the combination that gives you the highest value within max weight

```
1 for (int i = 0; i < (int)Math.Pow(2, Items.Count); i++)
2 // this can be a start, but maybe you can think of something else!
```

Brute force

- What is the output? **50**
- Is this the best solution?
  - Yes!! So so happy!
- What about run time?
  - n items, but we run **exponentially**
  - $O(n) = n^2$



Knapsack problem

- ~~The Knapsack problem~~
- ~~Start solving a problem~~
- Solve the Knapsack problem
  - ~~Greedy~~
  - ~~Brute force~~
  - Backtracking
  - Divide & Conquer
  - Dynamic programming

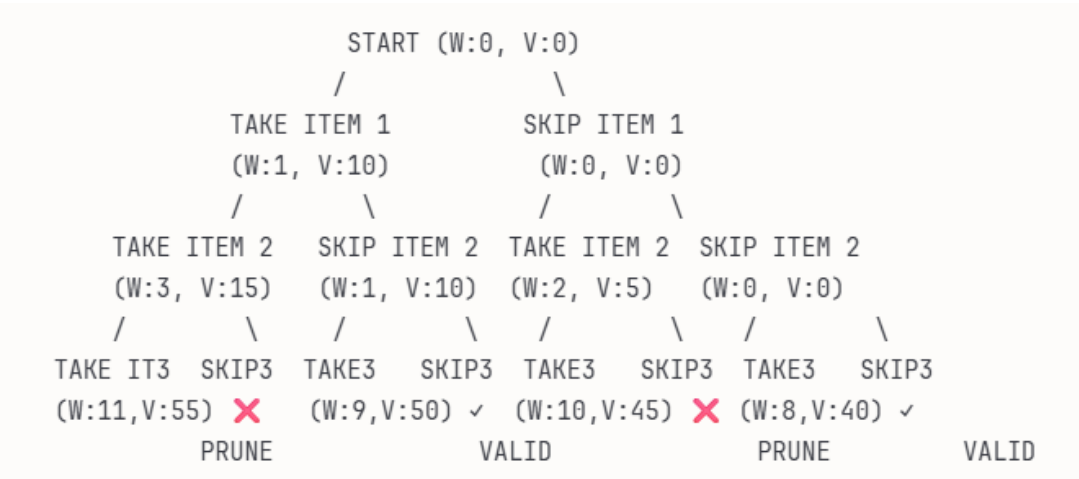
Backtracking

Backtracking Algorithm

- Do we need to calculate ALL combinations? NO!
  - We can stop when we hit constraints; many combinations are invalid
- What are we **building with 'take or leave' decisions**?
  - A **decision tree** - each node represents a **choice**
  - A solution space exploration - **systematically trying possibilities**
  - A binary choice sequence - for each item: include (1) or exclude (0)
- So instead of generating all 2^n combinations, we can:
  - **Build solutions incrementally**
  - **Stop when constraints are violated** and backtrack (go back!)
  - **Prune branches** that can't lead to better solutions

Backtracking

- Build a tree!
- **Prune on weight!**
  - *if > 10 kg = we can't take it!*



COMBINATION	BINARY	WEIGHT	VALUE
No items	0000	0	0
item 1	0001	1	10
item 2	0010	2	5
item 3	0100	8	40
item 4	1000	10	45
item 12	0011	3	15
item 13	0101	9	50
item 14	1001	11	55
item 23	0110	10	45
item 24	1010	12	50
item 34	1100	18	85
item 123	0111	11	55
item 124	1011	13	60
item 134	1101	19	95
item 234	1110	20	90
item 1234	1111	21	100

Backtracking

Backtracking Algorithm

- Advantages of Backtracking:
  - Guaranteed to find optimal solution
  - Can stop early through pruning
  - Can find ALL solutions (not just the best one)
  - Memory efficient (only current path in memory)
  - Easy to understand and debug

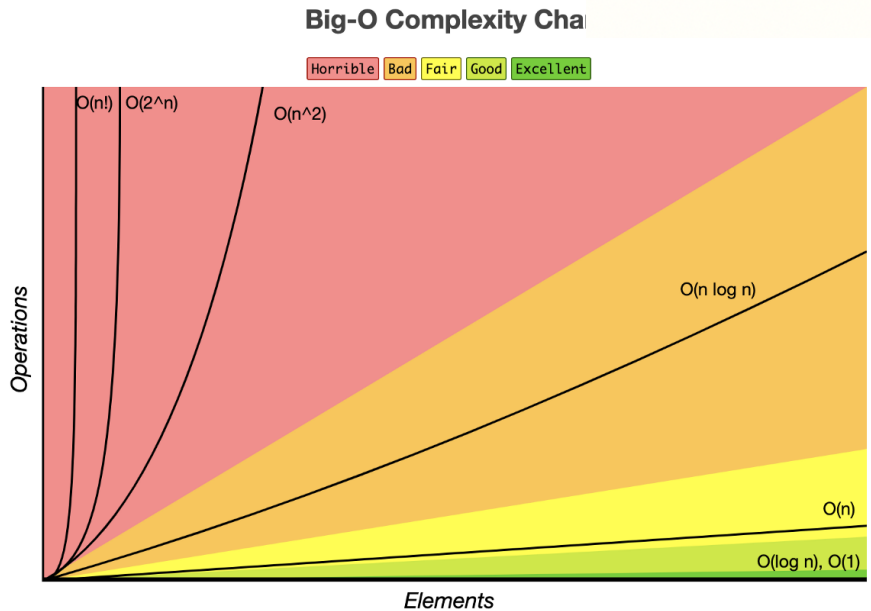
```
For each item you have 2 choices:
1. Take the item (if it fits)
2. Leave the item behind

If a path doesn't work → go back and try the other choice
```

Backtracking

Backtracking Algorithm

- What is the output? **50**
- Is this the best solution?
  - Yes!! So so happy!
- What about run time?
  - n items, we run **exponentially** $O(n) = n^2$ **BUT pruning can reduce actual runtime!**
  - Best case with pruning = much better than brute force!



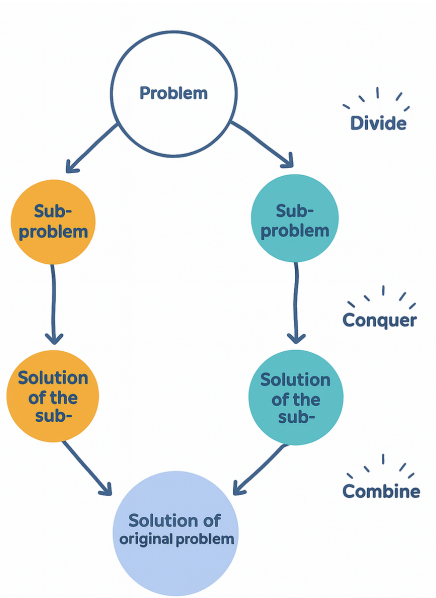
Knapsack problem

- ~~The Knapsack problem~~
- ~~Start solving a problem~~
- Solve the Knapsack problem
  - ~~Greedy~~
  - ~~Brute force~~
  - ~~Backtracking~~
  - Divide & Conquer
  - Dynamic programming

Divide and conquer

Divide & Conquer

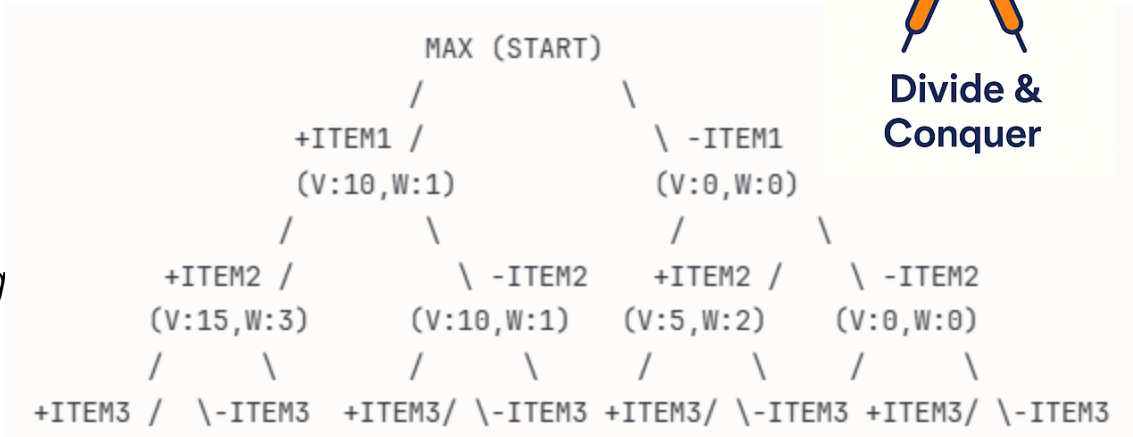
- Find the subproblems, solve the subproblems and merge!
- How can you go from a big problem to a smaller instance of the same problem?
  - Find the base case(s)
  - Find the recursive case = divide the problem until it becomes a base case!
- When write a recursive function involving an array, the base case is often an empty array or an array with one element. If you're stuck, try that first.



Divide and conquer

Divide & Conquer

- **Base case:**
  - No more items left
  - *earlier stop: no capacity remaining*
- **Recursive case:**
  - Split into two subproblems for current item
- **For each item you have 2 choices:**
  - **Include the item** (if it fits) → solve remaining problem
  - **Exclude the item** → solve remaining problem



Divide and conquer

- **Split the problem:**
  - **Subproblem 1**: Best value WITHOUT current item
  - **Subproblem 2**: Best value WITH current item (+ solve rest)
- **Combine the results:** return the **maximum of** both **subproblems**
- **Note**: we **don't backtrack** - we explore both paths and combine their results!

```
1 function knapsack(items, index, capacity):
2     if no more items: return 0
3
4     exclude = knapsack(items, index+1, capacity)
5     include = 0
6     if item fits:
7         include = item.value + knapsack(items, index+1, capacity-item.weight)
8
9     return max(exclude, include)
```

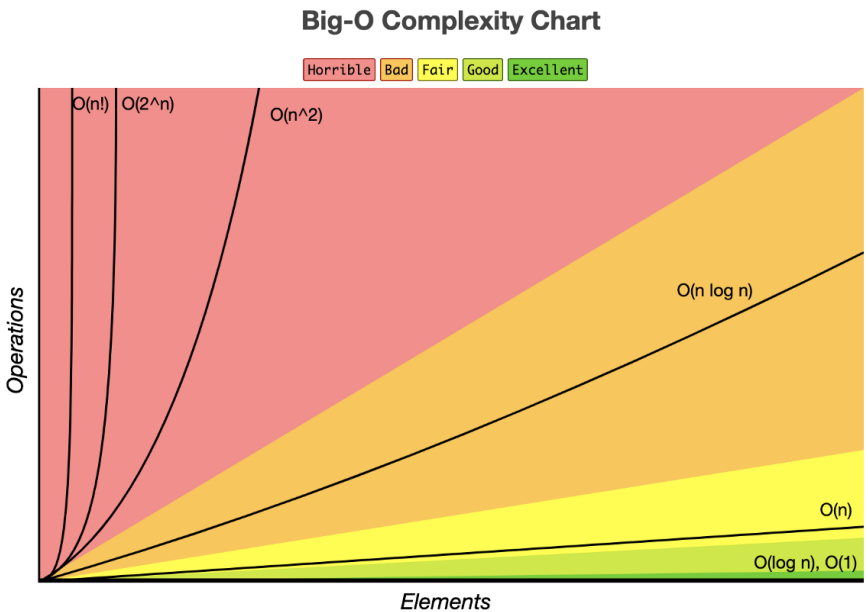
Divide and conquer <> Backtracking

- **Backtracking**: Builds ONE solution path, backtracks when stuck
  - Maintains state (current solution)
  - Explores depth-first with pruning
  - *"Try this path, if it fails, undo and try another"*
- **Divide & Conquer**: Explores ALL possibilities, combines results
  - Stateless - each call is independent
  - Splits the problem into subproblems
  - *"What's the best I can do WITH this item vs WITHOUT this item?"*
- **Same tree structure, different exploration philosophy!**

Divide and conquer

Divide & Conquer

- What is the output? **50**
- Is this the best solution?
  - Yes!! So so happy!
- What about run time?
  - n items, but we run **exponentially** $O(n) = n^2$
  - **We recalculate the same subproblems multiple times! (a lot of recursive calls)**

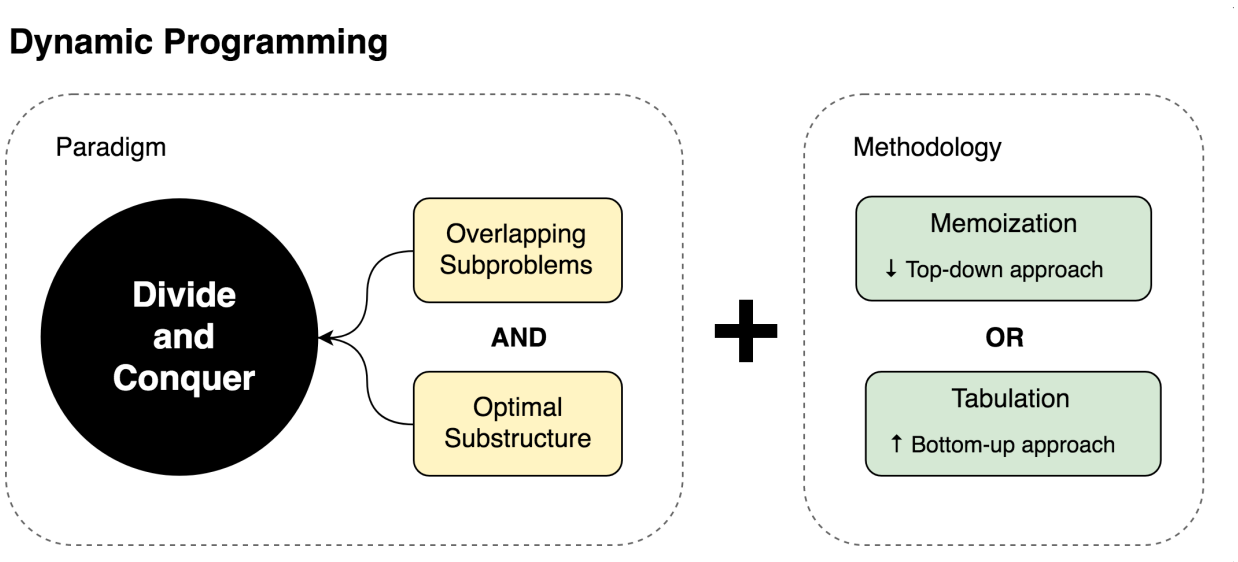


Knapsack problem

- ~~The Knapsack problem~~
- ~~Start solving a problem~~
- Solve the Knapsack problem
  - ~~Greedy~~
  - ~~Brute force~~
  - ~~Backtracking~~
  - ~~Divide & Conquer~~
  - Dynamic programming

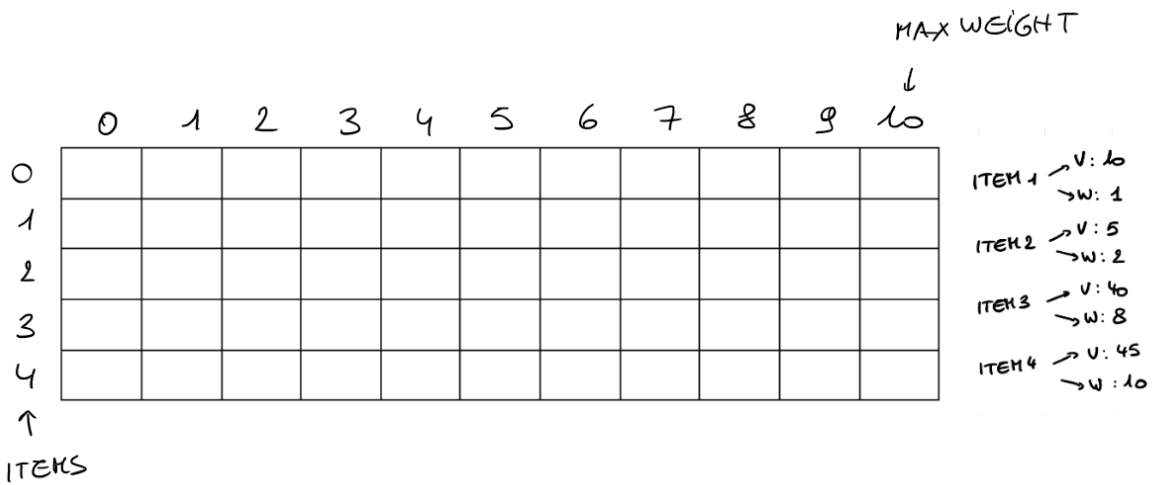
Dynamic programming

- Now it's time to show off ;)
- Divide & conquer = overlapping subproblems & optimal substructure



Dynamic programming

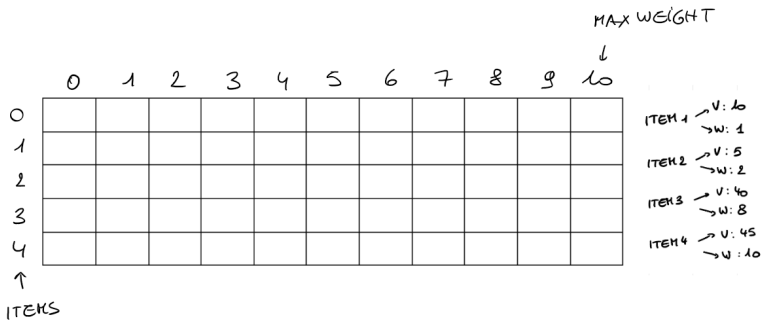
- Let's take tabulation
- We are building from the ground up
- We have two values to take into account
  - value of item
  - weight of item!
- We end up with a 2D Array
- How would you fill the matrix?



Dynamic programming

c[i][M] = value of solution for items 1...i with max weight M

$$c[i][M] = \begin{cases} 0 & \text{If } i=0, M=0 \\ c[i-1][M] & \text{If } w_i > M \\ \max(v_i + c[i-1][M-w_i], c[i-1][M]) & \text{If } i > 0 \text{ and } w_i < M \end{cases}$$



Dynamic programming

MAX WEIGHT

ITEMS	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10	10	10	10	10
2	0	10	10	15	15	15	15	15	15	15	15
3	0	10	10	15	15	15	15	15	40	50	50
4	0	10	10	15	15	15	15	15	40	50	50

ITEM 1 → V: 10, W: 1

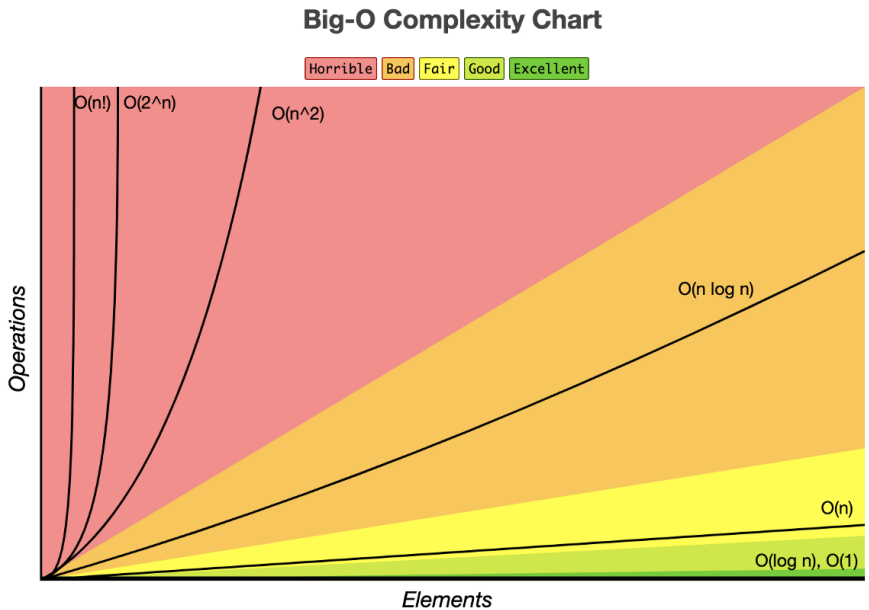
ITEM 2 → V: 5, W: 2

ITEM 3 → V: 40, W: 8

ITEM 4 → V: 45, W: 10

Dynamic programming

- What is the output? **50**
- Is this the best solution?
  - Yes!! So so happy!
- What about run time?
  - n items * weights
  - $O(n) = nW$
  - **n items * weights**
  - for W > n, bad..., but for W < n: there can be a huge difference!



Knapsack problem

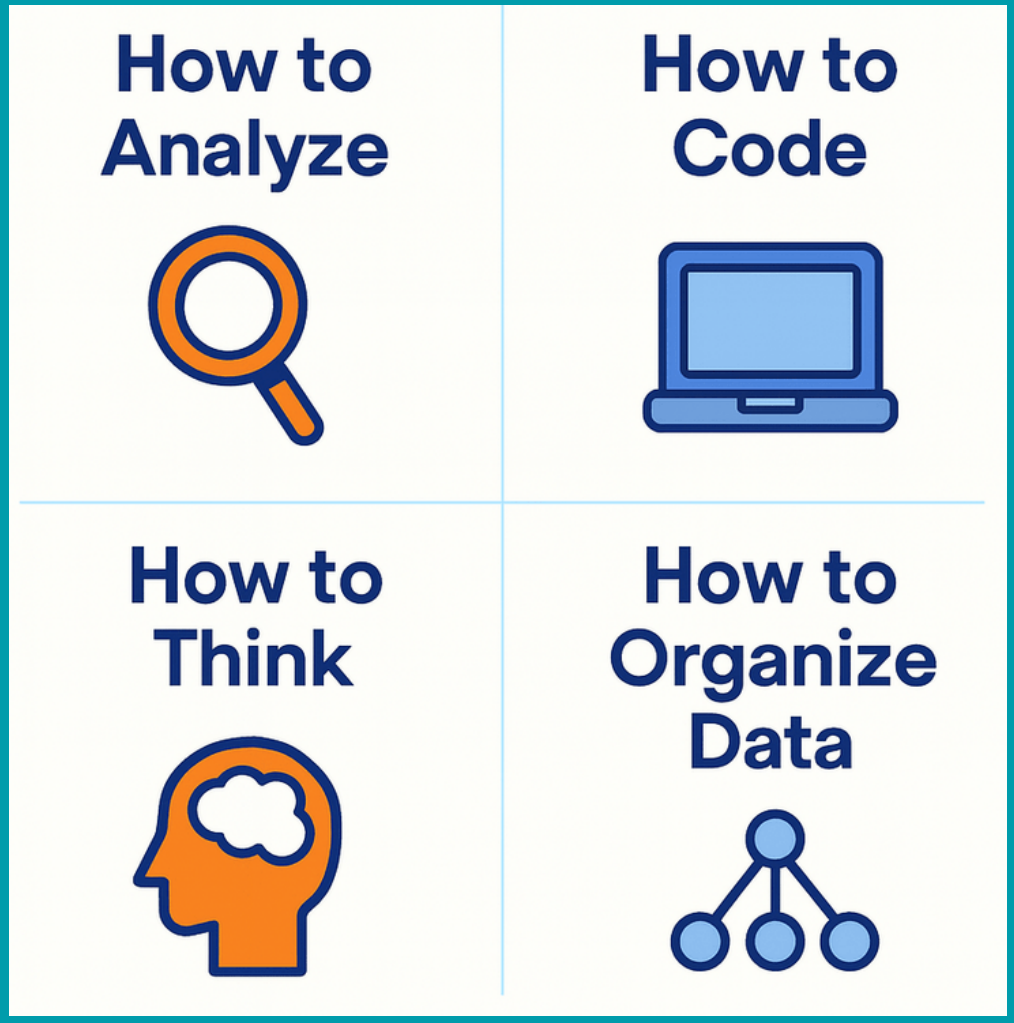
- ~~The Knapsack problem~~
- ~~Start solving a problem~~
- Solve the Knapsack problem
  - ~~Greedy~~
  - ~~Brute force~~
  - ~~Backtracking~~
  - ~~Divide & Conquer~~
  - ~~Dynamic programming~~

So what about the knapsack?

So if you ever go backpacking, consider this:

- **Greedy** lets you carry too much (maybe even unnecessary) stuff
- You really can't handle **brute force** or **divide & conquer**, way too many different combinations
- You can't always **go back** if you need something
- Just be **dynamic** and only pack up to 3 underpants ;)

- *We saw a lot of **different problem-solving strategies** this semester!*
- *They are **all tools in your toolbox**!*
- *It's **not about memorizing algorithms**, but about **trying** to **solve** a **problem** in **different ways** and to be able to **compare** those different solutions!*



“ Let's face it, you are ready and well prepared for the exam! You have this! 💪